Forschungsbericht 2025-16

Rocket Engine Control with Deep Reinforcement Learning

Kai Dresia

Deutsches Zentrum für Luft- und Raumfahrt
Institut für Raumfahrtantriebe
Lampoldshausen

Deutsches Zentrum
DLR für Luft- und Raumfahrt

Forschungsbericht 2025-16

Rocket Engine Control with Deep Reinforcement Learning

Kai Dresia

Deutsches Zentrum für Luft- und Raumfahrt
Institut für Raumfahrtantriebe
Lampoldshausen

153 Seiten
50 Bilder
21 Tabellen
222 Literaturstellen

Deutsches Zentrum
DLR für Luft- und Raumfahrt

Herausgeber:

Deutsches Zentrum
für Luft- und Raumfahrt e. V.
Wissenschaftliche Information
Linder Höhe
D-51147 Köln

ISSN 1434-8454
ISRN DLR-FB-2025-16
Erscheinungsjahr 2025
DOI: 10.57676/bwbm-p528

Triebwerksregelung, Flüssigkeitsraketentriebwerke, LUMEN, Deep Reinforcement Learning, Wiederverwendbarkeit, EcosimPro

Kai DRESIA
DLR, Institut für Raumfahrtantriebe, Lampoldshausen

Regelung von Raketentriebwerken mit Deep Reinforcement Learning
RWTH Aachen

Die Raumfahrtindustrie vollzieht derzeit einen Wandel hin zu wiederverwendbaren und kostengünstigen Trägersystemen. Dies stellt neue Anforderungen an die Auslegung und den Betrieb zukünftiger Flüssigkeitsraketentriebwerke. Wiederverwendbare Trägersysteme erfordern für Landemanöver eine präzise und schnelle Schubregelung über einen weiten Drosselbereich. Eine weitere Herausforderung besteht darin, dass sich Triebwerkseigenschaften aufgrund von Alterungseffekten von Flug zu Flug ändern. Darüber hinaus führt der zunehmende Einsatz kostengünstiger additiver Fertigungsverfahren zu größeren Fertigungstoleranzen und damit zu weiteren Unsicherheiten. Die bevorstehende Einführung von effizienteren, aber komplexeren Triebwerkszyklen wie der gestuften Verbrennung sowie der Einsatz von Triebwerksclustern erhöhen die Anforderungen an den Betrieb zusätzlich. Vor diesem Hintergrund besteht ein Bedarf an neuen, fortschrittlichen Regelungskonzepten, die auf die spezifischen Anforderungen wiederverwendbarer Flüssigkeitsraketenantriebe zugeschnitten sind. Der Einsatz klassischer Regelalgorithmen ist jedoch mit Herausforderungen verbunden. Mehrere gekoppelte Größen wie Schub und Mischungsverhältnis müssen gleichzeitig geregelt werden und weltraumtaugliche Prozessoren haben nur eine begrenzte Rechenkapazität. Neuronale Netze, die mittels Deep Reinforcement Learning anhand von Simulationsmodellen des Triebwerks trainiert werden, stellen hier eine vielversprechende Alternative dar. Sie benötigen im Betrieb nur minimale Rechenleistung und sind in der Lage, optimale Regelstrategien für komplexe, nichtlineare Systeme zu erlernen. Ziel dieser Arbeit ist es, die Eignung von Deep Reinforcement Learning für die Triebwerksregelung anhand des am Deutschen Zentrum für Luft- und Raumfahrt entwickelten LOX/LNG-Triebwerks LUMEN in zwei repräsentativen Testfällen zu untersuchen und mit einer modellprädiktiven Regelung zu vergleichen. Die Testfälle decken verschiedene Herausforderungen ab, wie zum Beispiel die Schubregelung über einen weiten Drosselbereich, die Berücksichtigung von Zwangsbedingungen oder die Maximierung des Triebwerkswirkungsgrades. Durch experimentelle Tests am Prüfstand P8.3 konnte die Eignung des Ansatzes am realen System mit einer Regelgenauigkeit von 1.3 % nachgewiesen werden. In der Arbeit werden weiterhin Stabilitäts- und Robustheitsbetrachtungen für zukünftige Flugtriebwerksregelsysteme diskutiert, bei denen Sicherheit und Zuverlässigkeit von höchster Bedeutung sind.

rocket engine control, liquid propellant rocket engine, LUMEN, deep reinforcement learning, reusability, EcosimPro

(Published in English)

Kai DRESIA
German Aerospace Center (DLR), Institute of Space Propulsion, Lampoldshausen

Rocket Engine Control with Deep Reinforcement Learning

RWTH Aachen University

The space industry is transitioning to reusable and cost-efficient launch vehicles. This significant transformation creates new operational challenges for liquid propellant rocket engines. Reusable launch vehicles require precise and fast thrust control over a wide throttling range, while component aging with each flight can lead to changes in engine system dynamics due to wear and tear. In addition, the increasing use of low-cost additive manufacturing introduces variability in hardware geometry, creating additional uncertainties. The transition to more efficient but more complex engine cycles, such as staged combustion, and the use of clustered engine configurations further complicate operations. Closed-loop control systems offer a promising solution by providing accurate thrust control even as system dynamics change due to engine reuse and manufacturing variability. However, designing classical controllers for liquid propellant rocket engines is challenging because it requires the simultaneous control of multiple coupled variables, while the computational power of space-qualified processors is limited. As a result, there is a growing need for new advanced control algorithms suitable for reusable engines. One promising solution is to use neural network-based controllers trained with deep reinforcement learning on a simulation model of the engine: They require minimal computational power during deployment and can learn optimal control policies for complex nonlinear systems. This thesis therefore investigates the suitability of deep reinforcement learning for liquid rocket engine control and compares the results with state-of-the-art model predictive control. Two test cases around the LOX/LNG expander-bleed LUMEN engine are studied, each presenting different control challenges, including thrust control over a wide throttling range, constraint handling, and maximizing engine efficiency. A controller is also being experimentally tested at the P8.3 test facility in Lampoldshausen, Germany. The controller achieved promising accuracy in controlling multiple variables simultaneously with an average error of only 1.3 %, demonstrating the viability of the approach. This work also discusses stability and robustness considerations for future engine control systems, where safety and reliability are paramount.

Rocket Engine Control with Deep Reinforcement Learning

Regelung von Raketentriebwerken mit Deep Reinforcement Learning

Von der Fakultät für Maschinenwesen der Rheinisch-Westfälischen Technischen Hochschule Aachen zur Erlangung des akademischen Grades eines Doktors der Ingenieurwissenschaften genehmigte Dissertation

vorgelegt von

Kai Dresia

Berichter: Univ.-Prof. Dr. rer. nat. Michael Oschwald
Univ.-Prof. Dr. sc. Sebastian Trimpe

Tag der mündlichen Prüfung: 09. April 2025

Diese Dissertation ist auf den Internetseiten der Universitätsbibliothek online verfügbar.

Abstract

The space industry is transitioning to reusable and cost-efficient launch vehicles. This significant transformation creates new operational challenges for liquid propellant rocket engines. Reusable launch vehicles require precise and fast thrust control over a wide throttling range, while component aging with each flight can lead to changes in engine system dynamics due to wear and tear. In addition, the increasing use of low-cost additive manufacturing introduces variability in hardware geometry, creating additional uncertainties. The transition to more efficient but more complex engine cycles, such as staged combustion, and the use of clustered engine configurations further complicate operations. Closed-loop control systems offer a promising solution by providing accurate thrust control even as system dynamics change due to engine reuse and manufacturing variability. However, designing classical controllers for liquid propellant rocket engines is challenging because it requires the simultaneous control of multiple coupled variables, while the computational power of space-qualified processors is limited. As a result, there is a growing need for new advanced control algorithms suitable for reusable engines. One promising solution is to use neural network-based controllers trained with deep reinforcement learning on a simulation model of the engine: They require minimal computational power during deployment and can learn optimal control policies for complex nonlinear systems. This thesis therefore investigates the suitability of deep reinforcement learning for liquid rocket engine control and compares the results with state-of-the-art model predictive control. Two test cases around the LOX/LNG expander-bleed LUMEN engine are studied, each presenting different control challenges, including thrust control over a wide throttling range, constraint handling, and maximizing engine efficiency. A controller is also being experimentally tested at the P8.3 test facility in Lampoldshausen, Germany. The controller achieved promising accuracy in controlling multiple variables simultaneously with an average error of only 1.3 %, demonstrating the viability of the approach. This work also discusses stability and robustness considerations for future engine control systems, where safety and reliability are paramount.

Kurzfassung

Die Raumfahrtindustrie vollzieht derzeit einen Wandel hin zu wiederverwendbaren und kostengünstigen Trägersystemen. Dies stellt neue Anforderungen an die Auslegung und den Betrieb zukünftiger Flüssigkeitsraketentriebwerke. Wiederverwendbare Trägersysteme erfordern für Landemanöver eine präzise und schnelle Schubregelung über einen weiten Drosselbereich. Eine weitere Herausforderung besteht darin, dass sich Triebwerkseigenschaften aufgrund von Alterungseffekten von Flug zu Flug ändern. Darüber hinaus führt der zunehmende Einsatz kostengünstiger additiver Fertigungsverfahren zu größeren Fertigungstoleranzen und damit zu weiteren Unsicherheiten. Die bevorstehende Einführung von effizienteren, aber komplexeren Triebwerkszyklen wie der gestuften Verbrennung sowie der Einsatz von Triebwerksclustern erhöhen die Anforderungen an den Betrieb zusätzlich. Vor diesem Hintergrund besteht ein Bedarf an neuen, fortschrittlichen Regelungskonzepten, die auf die spezifischen Anforderungen wiederverwendbarer Flüssigkeitsraketenantriebe zugeschnitten sind. Der Einsatz klassischer Regelalgorithmen ist jedoch mit Herausforderungen verbunden. Mehrere gekoppelte Größen wie Schub und Mischungsverhältnis müssen gleichzeitig geregelt werden und weltraumtaugliche Prozessoren haben nur eine begrenzte Rechenkapazität. Neuronale Netze, die mittels Deep Reinforcement Learning anhand von Simulationsmodellen des Triebwerks trainiert werden, stellen hier eine vielversprechende Alternative dar. Sie benötigen im Betrieb nur minimale Rechenleistung und sind in der Lage, optimale Regelstrategien für komplexe, nichtlineare Systeme zu erlernen. Ziel dieser Arbeit ist es, die Eignung von Deep Reinforcement Learning für die Triebwerksregelung anhand des am Deutschen Zentrum für Luft- und Raumfahrt entwickelten LOX/LNG-Triebwerks LUMEN in zwei repräsentativen Testfällen zu untersuchen und mit einer modellprädiktiven Regelung zu vergleichen. Die Testfälle decken verschiedene Herausforderungen ab, wie zum Beispiel die Schubregelung über einen weiten Drosselbereich, die Berücksichtigung von Zwangsbedingungen oder die Maximierung des Triebwerkswirkungsgrades. Durch experimentelle Tests am Prüfstand P8.3 konnte die Eignung des Ansatzes am realen System mit einer Regelgenauigkeit von 1.3 % nachgewiesen werden. In der Arbeit werden weiterhin Stabilitäts- und Robustheitsbetrachtungen für zukünftige Flugtriebwerksregelsysteme diskutiert, bei denen Sicherheit und Zuverlässigkeit von höchster Bedeutung sind.

Contents

List of Figures

List of Tables

List of Acronyms

AAS	American Astronautical Society
BAE	British Aerospace Electronic
BPV	bypass control valve
CALLISTO	Cooperative Action Leading to Launcher Innovation in Stage Toss-back Operations
CNES	Centre national d'études spatiales
DDPG	Deep Deterministic Policy Gradient
DEMO-24	demonstrator test campaign
DLR	German Aerospace Center
DMIPS	Dhrystone Million Instructions per Second
DQN	Deep Q-Network
ESA	European Space Agency
ESPSS	European Space Propulsion System Simulation
FCV	fuel control valve
FPOV	fuel preburner oxidizer valve
FTP	fuel turbopump
FTP-23	fuel turbopump test campaign
GPU	graphics processing unit
HMS	health and usage monitoring system
IAE	integrated absolute error
IGN	igniter
INJ	injector
IPOT	Interior Point Optimizer
I_{sp}	specific impulse
JAXA	Japan Aerospace Exploration Agency
LH2	liquid hydrogen
LNG	liquid natural gas
LOX	liquid oxygen
LQR	Linear Quadratic Regulator
LSTM	Long Short-Term Memory
LUMEN	Liquid Upper Stage Demonstrator Engine

MAPE	mean percentage absolute error
MCC	main combustion chamber
MDP	Markov Decision Process
MFV	main fuel valve
MIMO	multiple-input and multiple-output
MOV	main oxidizer valve
MPC	model predictive control
NN	neural network
OCV	oxidizer control valve
OPOV	oxidizer preburner oxidizer valve
OTP	oxidizer turbopump
OTP-22	oxydizer turbopump test campaign
PPO	Proximal Policy Optimization
PI	proportional-integral
PID	proportional-integral-derivative
RAM	random-access memory
RAV	regenerative cooling valve
ROF	ratio of oxidizer to fuel
RL	reinforcement learning
RWTH	Rheinisch–Westfälische Technische Hochschule
SAC	Soft Actor-Critic
SLM	selective laser melting
SISO	single-input and single-output
SSME	Space Shuttle Main Engine
TCA	thrust chamber assembly
TCA-23	thrust chamber test campaign
TFV	turbine fuel control valve
TOV	turbine oxidizer control valve
TBV	turbine bypass valve
UDP	User Datagram Protocol
XCV	mixer control valve

1. Introduction

Reusability and cost savings are the primary goals of future space transportation systems. Reusing the first stage of launch systems that return to Earth via retropropulsive landing is an important step in this transformation. After the stage and its engines are recovered and inspected, they can be reused, significantly reducing the recurring costs. However, the design and operation of liquid rocket engines faces new challenges caused by the shift to cost efficiency and reusability: Landing maneuvers require fast and precise thrust control over a wider throttling range, especially in the presence of strong atmospheric winds [24]. In addition, as engines are reused for multiple flights, aging effects can degrade performance over time [36]. Performance variations can also result from increased hardware dispersion caused by factors such as mass production and additive manufacturing [36]. The trend towards more efficient but more complex engines, such as those using staged combustion cycles, adds to the operational challenges.

Closed-loop rocket engine control provides precise and continuous throttling even as engine characteristics change. This provides an operational advantage in scenarios such as retropropulsive landings, planetary entry and descent, space rendezvous, and orbital maneuvers [118]. Throttleable rocket engines can also continuously follow the most efficient thrust curve rather than making few discrete thrust changes [30]. The control system can also counteract the changing dynamics as components age with each launch, and respond to perturbations such as variations in propellant temperature and pressure.

In the long term, engine life can be extended by coupling the controller with a health and usage monitoring system [119]. This integration allows closed-loop thrust control while minimizing accumulated engine damage, thereby maximizing engine life and reducing recurring operating costs. The trend towards using engine clusters rather than single engines provides additional optimization opportunities [36]. Finally, coupling the controller with a diagnostic system that can predict catastrophic system failures, such as combustor instabilities, allows such failures to be prevented. This proactive approach minimizes the risk of failure and increases the probability of mission success by actively avoiding critical system conditions [129, 130, 214].

Nowadays, most engines rely on pre-programmed open-loop control sequences rather than closed-loop control [150]. Open-loop control simplifies development,

testing, and validation, and is sufficient for single-use engines in expendable launch systems. However, open-loop control is inadequate to meet the advanced control requirements of future reusable engine developments [150]. The industry recognizes this need for evolution. For example, the new European Vinci upper stage engine was initially designed to operate in open-loop [64]. To improve its performance, a closed-loop control system based on classical proportional-integral (PI) loops for thrust and mixture ratio was developed and is now used in the flight version of Vinci [64]. Additionally, a dedicated rocket engine control system will be included in the Prometheus engine, which is currently under development [152].

The Space Shuttle Main Engine (SSME) is the most extensively documented and researched engine using a closed-loop control system. Designed for human-rated operation, reusability, and high performance, the SSME controller operates at 50 Hz and uses two decoupled PI control loops to regulate thrust and mixture ratio [178]. The controller was highly successful, providing reliable control during all missions. Despite its success, the engine suffered from problems caused by cumulative damage to the main combustion chamber and turbine blades [35, 158]. In response, Lorenzo et al. [117] proposed an intelligent cluster-level control framework to minimize engine wear. The idea is to distribute the total thrust according to the health status of each engine, which is estimated based on the current damage rate as a function of thrust and turbine discharge temperature. According to simulations, a potential reduction in accumulated damage by a factor of 3-4 seemed to be possible.

There are four major challenges for closed-loop rocket engine control: First, thrust and mixture ratio are coupled and must be controlled simultaneously. Rocket engine control is therefore by nature a coupled, multiple-input, multiple-output task. Second, rocket engines operate near the material limits of temperature and stress. Even brief periods of excessive temperature or pressure can cause significant damage. Therefore, a control system must be able to effectively manage constraints. Third, space radiation limits the computing power of processors, requiring computationally efficient control algorithms. Finally, a thorough test and validation of the control system is required. Certification of controllers for rocket engines is particularly difficult because rocket engine test facilities are expensive and rare.

Optimal control methods like model predictive control (MPC) [8, 156, 176] are a potential solution. MPC is the state-of-the-art approach for controlling complex physical systems that naturally handles constraints and multivariable systems. It optimizes control actions in real-time based on a mathematical system model. The optimization uses an objective function that contains the control requirements, such as following a reference trajectory or economic goals, like maximizing efficiency or engine life. Perez-Roca [148] studied MPC

for controlling the transient startup of a gas generator engine and obtained promising results. However, the author also reported challenges with the high computational requirements due to the necessary online optimization.

Neural networks are gaining popularity as an alternative especially for control in resource-constrained settings [13, 20, 142]. They can approximate any continuous control policy [88] and require minimal computing power during deployment [94]. Neural networks can be trained to output the optimal control action at each time step based on the current state of the system. Training can be done by supervised learning, for example to approximate an MPC controller [89], or by deep reinforcement learning [193].

Deep reinforcement learning trains neural networks by directly interacting with a (simulated) environment. It has shown remarkable success in mastering complex competitive board and computer games such as Go [174, 179, 180], Chess [174, 179], or StarCraft [207], surpassing the performance of human experts. In the space domain, reinforcement learning has been studied, among others, for planetary landing, orbit transfer, attitude control, and docking maneuvers [197]. Reinforcement learning uses a reward function to encapsulate control objectives such as following a reference trajectory, handling constraints, and maximizing economic goals.

Reinforcement learning can also train neural networks by directly using non-linear simulation models. Such models (for example in EcosimPro or Matlab Simulink) are typically already available in the space industry, as they are needed for system analysis, component design and sizing, and test preparation. Reinforcement learning therefore directly leverages the effort put into developing and validating such models. This is a major advantage over MPC, which requires the derivation and validation of (often linearized) mathematical models. However, robustness, stability, safety, and certification remain open challenges in reinforcement learning, especially when the neural network is trained only in simulation. Many extensions such as robust, safe, or fault-tolerant reinforcement learning are currently being investigated to address these challenges [34, 127, 141, 222].

In the space domain, reinforcement learning has been studied to enhance the autonomous on-board capabilities of spacecrafts. These studies aim to minimize reliance on human intervention and increase the spacecraft's ability to cope with uncertain and changing environments. In the long term, autonomous on-board systems may enable entirely new types of missions that are not currently feasible [197]. Examples are investigations of guidance, navigation, and control for autonomous landing on celestial bodies [177], space debris removal [160], orbit transfers [85], missile guidance [62], and docking maneuvers with both controlled and uncontrolled spacecraft [51, 57, 91].

The Institute of Space Propulsion of the German Aerospace Center (DLR) in Lampoldshausen conducts research on rocket engine control and health monitoring with the aim of developing an intelligent engine controller [47]. The work includes reinforcement learning for offline optimization of engine startup sequences and the automation of test stand operation [46, 213]. Fault detection and isolation is also explored [108]. Initial research into using reinforcement learning for rocket engine control was conducted in simulation with an expander bleed engine [19, 48, 52]. First experimental tests with a cold gas thruster have also been successfully performed [86, 87]. Finally, research is aimed at providing safety guarantees for reinforcement learning [39].

A major milestone was reached in 2023 with the testing of a neural network-based controller for the Liquid Upper Stage Demonstrator Engine (LUMEN) oxidizer turbopump [43]. The controller was trained with reinforcement learning in a simulated EcosimPro environment and then used for real-time control at the test facility. It effectively tracked a predefined reference trajectory for pump discharge pressure and mass flow by manipulating two flow control valves. Over a total test duration of more than 500 seconds, the mean error remained consistently below 1 percent and no instabilities were observed. This successful test was an important milestone in the development of an intelligent engine controller.

1.1. Research objectives

The primary objective of this thesis is to evaluate the suitability of reinforcement learning for rocket engine control. The study begins with a comprehensive literature review to identify the key control objectives for liquid propellant rocket engines during different mission phases, from engine ignition to throttling and steady-state operation. The review also explores more advanced concepts, such as controlling entire engine clusters and developing fault-tolerant systems. In addition, it analyzes the major challenges of rocket engine control, including the inherent coupling between thrust and mixture ratio, safety concerns, and limited on-board computing resources.

Two representative test cases of thrust and mixture ratio control are studied, including economic objectives and constraint handling. The neural network controllers are trained in simulation only using reinforcement learning. EcosimPro and the ESPSS libraries [55] are used for modeling. These libraries are commissioned by ESA and are widely used in the European space industry. After training, the control performance and robustness to model parameter variations and sensor noise are analyzed. Finally, the control performance is experimentally tested on the P8.3 test bench in Lampoldshausen.

Another goal of this thesis is to compare the neural network controllers with a state-of-the-art MPC approach developed in cooperation with the Institute of Control Engineering at the RWTH Aachen University [205]. The comparison focuses on tracking performance, constraint handling capabilities, and fuel efficiency, and aims to demonstrate that both approaches are well suited for liquid rocket engine control, each having distinct strengths and weaknesses.

All test cases are centered around the LUMEN project. The engine developed in this project is a LOX/LNG expander-bleed engine in the 25 kN thrust range, developed and tested in house [200]. Its extensive instrumentation, precise electrical flow control valves and representative engine cycle make it an ideal platform for studying control algorithms.

In summary, the following research questions are studied:

1. How accurately can a neural network control the transient thrust and mixture ratio of a representative liquid propellant rocket engine? How can the policy be made robust against modeling errors? Is it possible to include advanced control objectives, such as maximizing efficiency or minimizing engine damage, into the control policy?
2. What are the advantages and challenges of using reinforcement learning for rocket engine control compared to state-of-the-art MPC?
3. How does the neural network controller perform on a real rocket engine during experimental tests? Is it feasible to train the neural network only on a simulation model, which may contain modeling errors, and then deploy it directly on the test bench without further fine-tuning?

1.2. Structure of this work

This work is structured as follows:

Chapter 2 provides a comprehensive literature review on rocket engine control, covering historical developments and recent trends towards closed-loop control. The special requirements regarding sensing, actuation, and onboard computers are outlined. The suitability of various control algorithms is then discussed. The example of the Space Shuttle Main Engine Controller offers valuable insights into the challenges of closed-loop control.

Chapter 3 describes the basics of reinforcement learning, the Soft Actor-Critic (SAC) algorithm that is used in this work, and the process of formulating a typical rocket engine control problem as a Markov Decision Process. The challenges of applying reinforcement learning to real-world systems are discussed, along with related work in engineering and space applications.

Chapter 4 presents the LUMEN engine, include the architecture, the operational envelope, and system constraints. A simulation model of LUMEN in EcosimPro using the ESPSS libraries is created and compared to experimental data from component test campaigns. The simulation model is analyzed by exciting it with step functions to the flow control valves and observing the system response.

Chapter 5 covers test case 1, the fuel-efficient thrust and mixture ratio control with thermodynamic and mechanical constraints. The control problem is formulated within the framework of reinforcement learning, detailing the action space, observation space, and reward function. The controller is analyzed in terms of tracking performance, fuel efficiency, and its capability to handle constraints. The robustness against sensor noise and modeling errors is also studied. Finally, the control performance is compared with MPC.

Chapter 6 presents test case 2, the continuous thrust control of LUMEN. The controller is trained in simulation and its tracking performance, robustness to model parameter variations and sensor noise is analyzed. Then, it is tested experimentally at the P8.3 test facility in Lampoldshausen during two consecutive hot-run tests.

Chapter 7 summarizes the major findings of this thesis and discusses future work. The focus is how to deal with the challenges of robustness and safety of reinforcement learning in the safety-critical environment of rocket engine control. Extensions such as safe reinforcement learning or coupling reinforcement learning with traditional low-level controllers such as PI controller is discussed, leading to open research questions.

DeepL and ChatGPT were used throughout this work to assist with English proofreading. Both tools helped improve the language, style, and readability of the text, but were not used for content generation. An example prompt that was used is as follows “Proofread the given paragraph in terms of language and grammar. Do not generate additional content, add sentences, or change the reasoning”.

2. Overview of rocket engine control

Many publications on rocket engine control focus on individual aspects, but a brief overview of liquid rocket engine control is lacking. Therefore, this chapter provides an overview of the various control tasks and challenges during operation. First, the basics of liquid rocket engine operation are discussed. Then, emerging challenges and advanced control objectives related to reusability and clustered engine configurations are addressed. In addition, the influence of the engine architectures and the characteristics of the actuators, sensors, and on-board computers on the engine control are discussed.

Several control algorithms that have either been used in the past or are potentially well suited for future advanced engine control are presented. Finally, the Space Shuttle Main Engine (SSME) controller is presented as an example of closed-loop engine control. The SSME is one of the most studied engines and many studies have been conducted to improve engine safety and life expectancy. The overall objective of the chapter is to provide the reader with an overview of rocket engine control, focusing on the challenges and open issues for the control system, especially in the context of reusability.

2.1. Fundamentals of rocket engine control

Rocket engines generate thrust by ejecting a gas accelerated to high velocity through a nozzle. Efficient engines use the combustion of reactive chemicals, consisting of fuel and oxidizer, in a combustion chamber to provide the necessary energy. According to Newton's third law, the ejected gas pushes the engine in the opposite direction. Changing the thrust angle can be accomplished by gimbaling the engine. The magnitude of the thrust is typically controlled by the amount of oxidizer and fuel that is being combusted [192].

The basic principles of engine control can be illustrated using a simple bi-propellant pressure-fed engine as shown in Figure 2.1. Propellants are delivered to the combustion chamber from pressurized tanks via supply lines and the injector head. Two independent flow control valves upstream of the injector head regulate the mass flow of fuel $\dot{m}_{\text{fuel}}$ and oxidizer $\dot{m}_{\text{ox}}$. The primary goal of engine control is to regulate the combustion chamber pressure p_{cc} and the ratio of oxidizer to fuel (ROF), or mixture ratio. p_{cc} mostly determines

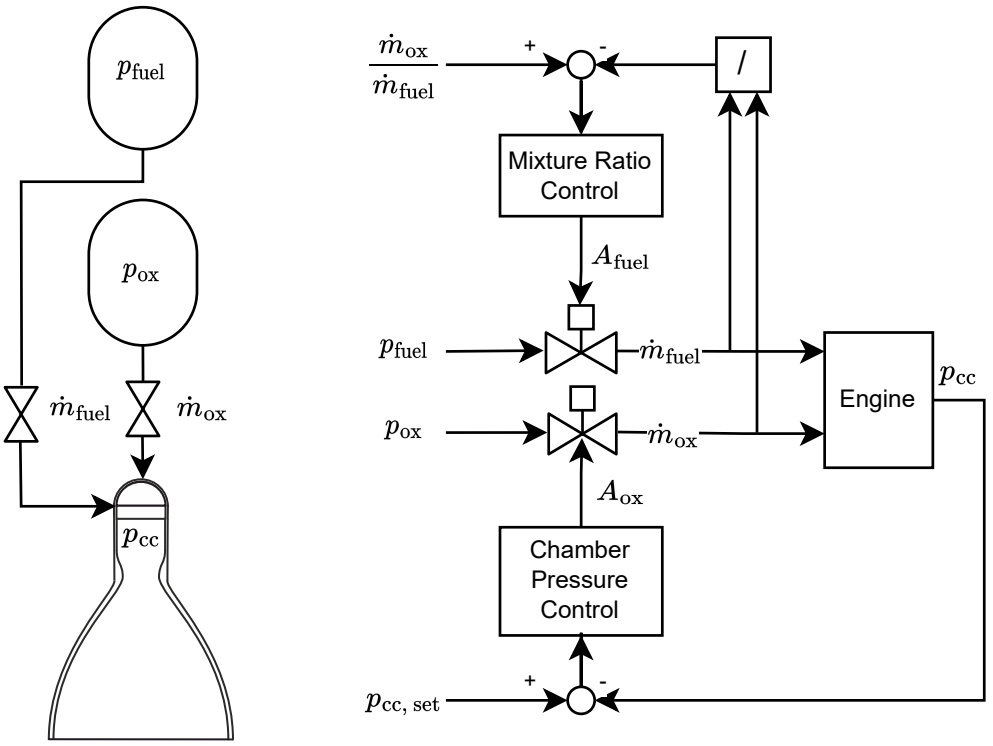


Figure 2.1.: Control example for a pressure fed engine. Reprinted from Lorenzo and Musgrave [118], 1992, with the permission of AIP Publishing

the engine's thrust and can be regulated via the total propellant mass flow $\dot{m}_{\text{T}} = \dot{m}_{\text{ox}} + \dot{m}_{\text{fuel}}$. *ROF* influences the combustion temperature and is significant in terms of propellant utilization and combustion efficiency. It is defined via

$$ROF = \frac{\dot{m}_{\text{ox}}}{\dot{m}_{\text{fuel}}}. \tag{2.1}$$

The example is discussed by Lorenzo and Musgrave [118], who give the transfer function of the system describing the relation between p_{cc} and $\dot{m}_{\text{T}}$:

$$\frac{p_{\text{cc}}(s)}{\dot{m}_{\text{T}}(s)} = \left(\frac{c^*}{A_{\text{t}} g}\right)\left(\frac{e^{-\sigma s}}{\tau s + 1}\right), \tag{2.2}$$

where s is the output variable of a Laplace transformation. All elements on the right side of the equation are parameters of the propellant combination and the combustion chamber geometry. $(c^*/A_{\text{t}} g)$ is a proportional constant depending on the the characteristic exhaust velocity c^* of the propellant combination and the throat area A_{t}. σ is the combustion delay, and τ is the chamber fill time. Therefore, p_{cc} is directly proportional to $\dot{m}_{\text{T}}$.

By adjusting the upstream flow control valves in each feed line, $\dot{m}_{\text{ox}}$ and $\dot{m}_{\text{fuel}}$ can be controlled. However, it follows from equations 2.1 and 2.2 that p_{cc}

and ROF are coupled. p_{cc} is directly proportional to the sum, while ROF is determined by the ratio between $\dot{m}_{fuel}$ and $\dot{m}_{ox}$. To decouple p_{cc} and ROF, two independent control loops are usually used. The mixture ratio loop is often tuned to be faster to avoid harmful stoichiometric conditions, which cause excessive hot gas flame temperatures. The combustion chamber pressure loop is slower and tuned according to thrust response requirements [118].

The fundamental coupling between p_{cc} and ROF persists even in more complex, pump-fed engines: Gas generator cycles use a secondary combustion chamber that produces hot gas for the turbopumps. The main combustion chamber pressure can be modulated by adjusting the total amount of hot gas that is produced. Changing the main combustion chamber mixture ratio is achieved by adjusting the distribution of hot gas to the oxidizer and fuel turbopumps. Both control loops are again coupled. For this engine cycle, an additional constraint is given by the gas generator mixture ratio, which must also be controlled, further complicating the control task.

Conclusion for control: The coupling between p_{cc}, and thus engine thrust, and ROF inherently increases the difficulty of designing appropriate controllers. More complex engine cycles have additional constraints that must be met during operation.

2.2. Basic engine control

The operation of liquid rocket engines can be divided into multiple chronological phases: First, the engine and propellant systems are purged and cooled down to reduce cavitation, boiling, or two-phase flows during ignition. Next, the combustor is ignited and the pressure is increased to nominal operating conditions. The engine then operates at a constant thrust level or is throttled, depending on the mission scenario. Finally, the engine is shut down. [192]

Start and shutdown control

The primary objective of start and shutdown control is to ensure safe ignition of the combustion chambers and to bring the engine to its intended operating condition. A typical start sequence consists of several steps, including preconditioning, chilldown and, if necessary, activation of a turbine starter. Propellants are then injected and ignited, and the combustion chamber pressure is gradually increased to reach the intended rated conditions [92]. In some cases, there may be a brief hold at moderate chamber pressures to verify nominal operation before ramping to higher pressures [23]. The ignition and

ramp-up phase typically lasts up to 5 s for most pump-driven cycles [148]. However, for expander cycle engines, this phase can extend up to 15 s until the system reaches its equilibrium state [106].

The engine ignition sequence is typically controlled in open loop [148]. It is one of the most challenging phases, requiring precise timing of the engine's flow control valves, often with an accuracy of a few milliseconds. In addition, this phase involves discrete events such as the activation of the igniter or turbine starter. Factors such as two-phase flow, priming, and boiling add to the complexity. Traditionally, the development and refinement of startup sequences has relied heavily on experimental testing and extensive experience.

Closed-loop control during ignition is prohibited due to limited observability and controllability. Sensors and actuators do not function properly due to low pressures, low mass flow rates, and two-phase flow conditions [135]. As a result, closed-loop control is typically activated after all discrete events, such as ignition and turbine starter operation, have been completed and higher pressure and mass flow levels have been established. For example, thrust control on the Space Shuttle main engine is activated 2.2 seconds after ignition, followed by mixture ratio control after 3.7 seconds. [23].

Shutting down the engine is handled similarly, using a fixed sequence. The shutdown of single-use engines is less challenging as it only requires a safe shutdown without causing any damage to the launch vehicle. However, reusable engines need to be protected from damage caused by high temperatures, oxygen-rich combustion, or overspeeding the turbopumps.

Conclusion for control: Complex open-loop sequences are required for engine startup and shutdown, which are experimentally established and heavily rely on experience. After this initial phase, closed-loop control of thrust and mixture ratio can be activated.

Thrust control

The most common method of thrust control is to adjust the combustion chamber pressure [21]. The thrust F can be calculated by adding the momentum and pressure forces:

$$F = \dot{m}_\mathrm{T} u_\mathrm{e} + (p_\mathrm{e} - p_\mathrm{amb}) A_\mathrm{e}, \tag{2.3}$$

where $\dot{m}_\mathrm{T}$ is the total propellant mass flow rate, u_e and p_e are the velocity and pressure at the nozzle exit, p_amb is the ambient pressure, and A_e is the nozzle exit area. Assuming isentropic nozzle flow, a constant specific heat ratio κ,

and ideal gas behavior, the ideal thrust equation can be derived [192]:

$$F = A_{\mathrm{t}} p_{\mathrm{cc}} \sqrt{\frac{2\kappa^2}{\kappa - 1} \left(\frac{2}{\kappa + 1}\right)^{\frac{\kappa+1}{\kappa-1}} \left[1 - \left(\frac{p_e}{p_{\mathrm{cc}}}\right)^{\frac{\kappa-1}{\kappa}}\right]} + (p_{\mathrm{e}} - p_{\mathrm{amb}}) A_{\mathrm{e}}. \quad (2.4)$$

For this ideal engine, the thrust is a function of the combustion chamber pressure p_{cc}, the throat area A_{t}, the specific heat ratio κ, and flow conditions at the nozzle exit. Since the effect of the mixture ratio on κ is limited, thrust control becomes equivalent to p_{cc} control. Other techniques such as throat area adjustment have been studied but are not common [49].

Historically, orifices have been adjusted during pre-flight test campaigns to fine-tune the combustion chamber pressure to the desired thrust level [150]. When multiple thrust levels are required, predefined valve sequences are programmed without relying on sensor feedback. Continuous throttling, however, requires closed-loop feedback [150]. Varying tank pressures and fuel temperatures during long missions might also require closed-loop control [192].

Mixture ratio control

Mixture ratio control is the regulation of the oxidizer-to-fuel ratio supplied to the combustion chamber. This control is essential for maximizing propellant utilization, for operating the engine at peak efficiency, and for preventing harmful stoichiometric conditions associated with high combustion temperatures. To ensure that the maximum material temperature of the combustion chamber wall is not exceeded, operating conditions close to the stoichiometric mixture ratio needs to be avoided.

The mixture ratio control loop is often executed faster than the thrust control loop to prevent damaging high temperatures [118]. Without active control, external factors such as temperature and pressure changes in the propellant tanks can cause the mixture ratio to shift during the mission. This can lead to suboptimal performance and non-ideal propellant utilization, reducing the payload capacity of the launch vehicle. During transient phases such as throttling, the mixture ratio needs to be kept constant.

Safety control

Engine components have mechanical and thermal limits that must be respected during operation. Given the essential high efficiency and minimal weight of rocket engines, most components operate at the limit of what is technically feasible, requiring precise control and monitoring. For example, in gas generator

engines, turbine gases reach temperatures of up to 1000 K, which is close to the thermal limit of the turbine blades. Excessive temperature spikes or prolonged operation at higher temperature can result in significant damage to the turbine blades, which can reduce their service life or lead to fatal damage.

Critical parameters are typically monitored by rule-based systems that can safely shut down the engine or switch to an emergency power level if an anomaly is detected [150]. This approach may be appropriate for expendable engines operating at few, fixed thrust levels. However, reusable engines may require more sophisticated control systems to monitor constraints in order to extend engine life. A suitable control algorithm should therefore be able to handle constraints naturally. Even short constraint violations can cause fatal damage, so constraint handling becomes increasingly challenging for complex cycles, such as staged-combustion engines, that are operated in a wide operational envelope with fast transients.

Example of a basic engine controller

Figure 2.2 gives an overview of the basic rocket engine control tasks. A controller receives thrust $|F|_{\text{target}}$ and mixture ratio ROF_{target} commands from a high-level vehicle controller that is responsible for guidance and navigation.

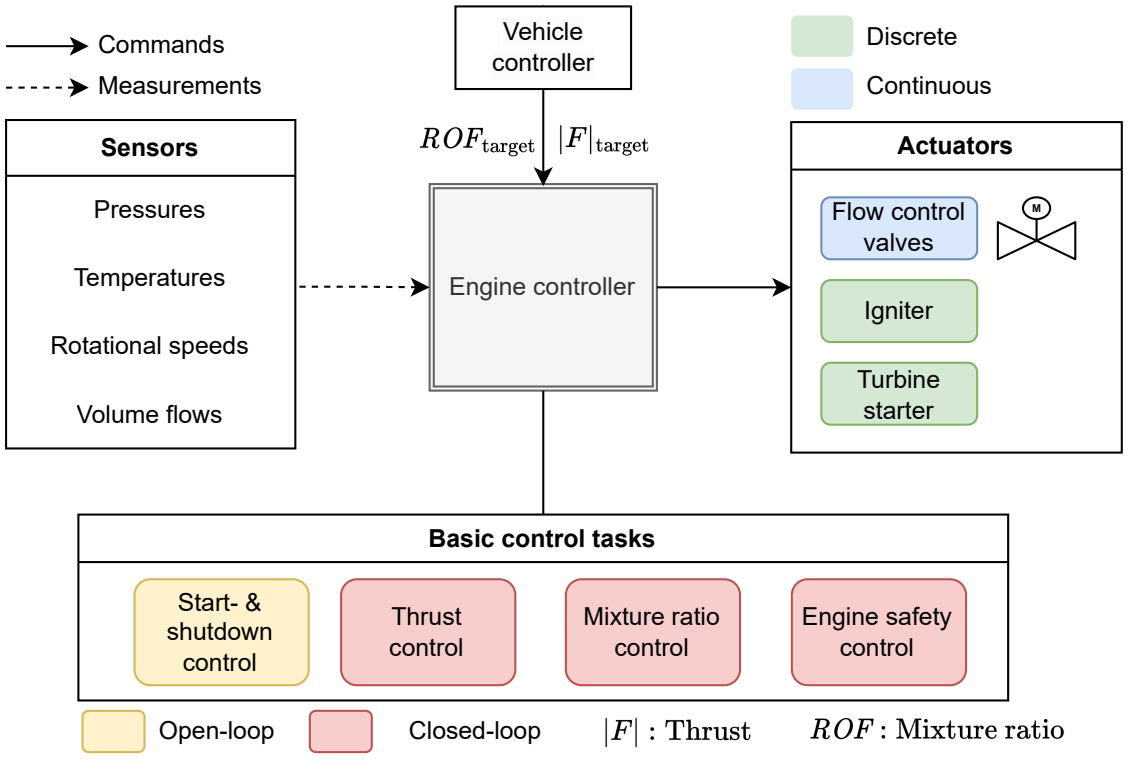


Figure 2.2.: Basic engine controller concept

Using real-time sensor values of the combustion chamber pressure and an estimated mixture ratio, the controller performs thrust and mixture ratio control via the engine's adjustable flow control valves. To ensure engine safety, the controller also monitors turbine inlet temperatures and rotational speeds. Finally, the controller is responsible for starting and shutting down the engine safely using an igniter and turbine starter system.

2.3. Advanced engine control

Advanced engine control can improve the safety, performance and reusability of rocket engines. The goal is to move beyond thrust and mixture ratio control: **Life-extending control** [118] addresses challenges of reusability by reducing engine damage. **Cluster control** [36, 136] treats a cluster of multiple engine as a single system rather than individual ones. **Fault-tolerant control** [25] is designed to increase reliability in the event of sensor, system, or actuator faults. **Active combustion control** [125] is a promising development that combines the fields of combustion dynamics and control. The goal is to forecast combustion instabilities and use actuators to prevent their occurrence.

2.3.1. Life-extending control

Life-extending control refers to control methods designed to minimize wear and damage, thereby extending operational life and reducing costs. Minimizing wear is critical for reusable engines to ensure high performance over multiple flights without extensive maintenance. One idea for life-extending control is an integrated health and usage monitoring system (HMS). The HMS estimates damage rates for the combustion chamber and the turbopumps, taking into account pressure levels, temperatures, and rotational speeds. The damage rate is then fed back into the control algorithm to find the best trade-off between performance and damage. One challenge is that most damage models are nonlinear. This requires high computing power when the damage rate is calculated in real time [119].

During the Space Shuttle era, life-extending control has been studied to reduce the recurring costs of the program [118]:

Ray et al. [158] proposed a life-extending control concept at engine level. The idea is to find optimal open-loop sequences before the mission to reduce turbine blade wear during critical transient phases. The damage rate is computed using nonlinear damage models [116] and fed back into the optimization algorithm. Nonlinear optimization then yields open-loop sequences with

minimized damage while maintaining fast transients. Simulations indicate a potential reduction in accumulated damage by a factor of 3-4 compared to a baseline sequence.

The approach was limited to offline optimization prior to flight due to the high computational demands of performing the nonlinear optimization. To overcome this challenge, the authors suggested the use of neural networks to replace the nonlinear damage models. Neural network based damage models could bring life extending control to real-time execution. Preliminary work in this direction was done by Troudet and Merrill [204] as early as 1990.

Lorenzo et al. [119] investigated real-time life-extending control. The concept involves two distinct control loops: an inner performance loop for thrust and mixture ratio control based on a linear lumped parameter model with 20 state variables. The outer loop is responsible for damage mitigation. It uses a nonlinear damage model to estimate damage rates using the inner loop as the plant model. By using nonlinear optimization techniques, the damage rate to the engine was reduced in simulation. The authors emphasize the need to minimize the numerical cost of the damage models, since the necessary real-time nonlinear optimization limited execution speeds.

Prognostics involves predicting future system performance and subsequently forecasting its remaining useful life. This technique has recently received significant research attention with the goal of reducing the recurring costs of reusable launch vehicles. The idea is to predict the remaining useful life of engine components to facilitate planned replacement or maintenance. Studies have focused on the main combustion chamber [32, 69, 90, 105], turbopumps [61, 70, 143], and refinement of the methodology itself [168, 182]. However, the direct coupling of prognostic methods to engine control algorithms is still little explored. One reason may be the necessary real-time optimization, which is often prevented by the nonlinear nature of the prognostic methods.

Conclusion for control: Most damage models are nonlinear and therefore cannot be included in standard control algorithms. However, increasing the life of the engine promises a significant reduction in recurrent costs. Thus, life-extending control should be a focus of future controller developments.

2.3.2. Cluster control

Cluster control refers to methods for the simultaneous control of multiple rocket engines in a cluster. One idea is that an intelligent cluster controller could distribute the total required thrust to each engine based on the current health of each engine. By taking into account the health status, the cluster controller could increase the overall reliability of the entire cluster and ensure

mission success even in the presence of anomalies or degraded performance of individual engines. The cluster controller is also responsible for throttling, which can be achieved by continuously adjusting the thrust of each engine or by shutting down entire engines completely.

Nemeth et al. [136] studied an intelligent cluster control framework for the Space Shuttle, which is powered by three engines. The goal was to minimize maintenance, extend service life, and maximize mission success. The framework integrates real-time diagnostics, engine cluster control, and adaptive reconfiguration modes for hardware faults. It provides capabilities to adjust the engine control-loop in response to sudden engine faults or degradation.

Lorenzo et al. [117] developed an intelligent control framework for all three Space Shuttle engines. The authors aimed to minimize wear by estimating the current damage rate for each engine based on its current thrust level and turbine exhaust temperature. The total vehicle thrust requirement is then distributed among all three engines so that the damage rate of each engine is equal. If an engine shows signs of degradation in the form of higher damage rates, the thrust of that engine is automatically reduced.

Colas et al. [36] analyzed the challenges and benefits of cluster control for reusable engines with the goal of minimizing total recurrent cost. The authors explored different architectures for cluster control - a bottom-up and a top-down approach: Both approaches consist of three levels: the launcher level, the propulsion cluster level, and the engine level itself. They further propose to couple the engine control to a dedicated health and usage monitoring system, including fault detection and isolation algorithms. By detecting faults, the propulsion cluster controller could reconfigure the system. In addition, damage to engines could be minimized by allocating thrust requirements to the healthiest engine. On the other hand, they note that more research and testing is needed due to the high complexity of cluster control.

Exemplary architecture: Figure 2.3 illustrates an example architecture for cluster level control. The system consists of three primary components: the high-level vehicle controller, a dedicated cluster controller, and multiple low-level engine controllers. The **vehicle level controller** is the highest in the hierarchy and responsible for executing the guidance, navigation, and control. It calculates the required total vehicle thrust $|F|_{\text{target}}$ based on the current and desired trajectory. The **cluster controller** distributes $|F|_{\text{target}}$ to each engine, taking into account the potentially limited operating envelope of the engines due to aging or faults. It also distributes the global mixture ratio among the engines to maximize propellant utilization and efficiency.

Engine controllers are located on each engine. They are responsible for meeting the thrust and mixture ratio requirements of the cluster controller.

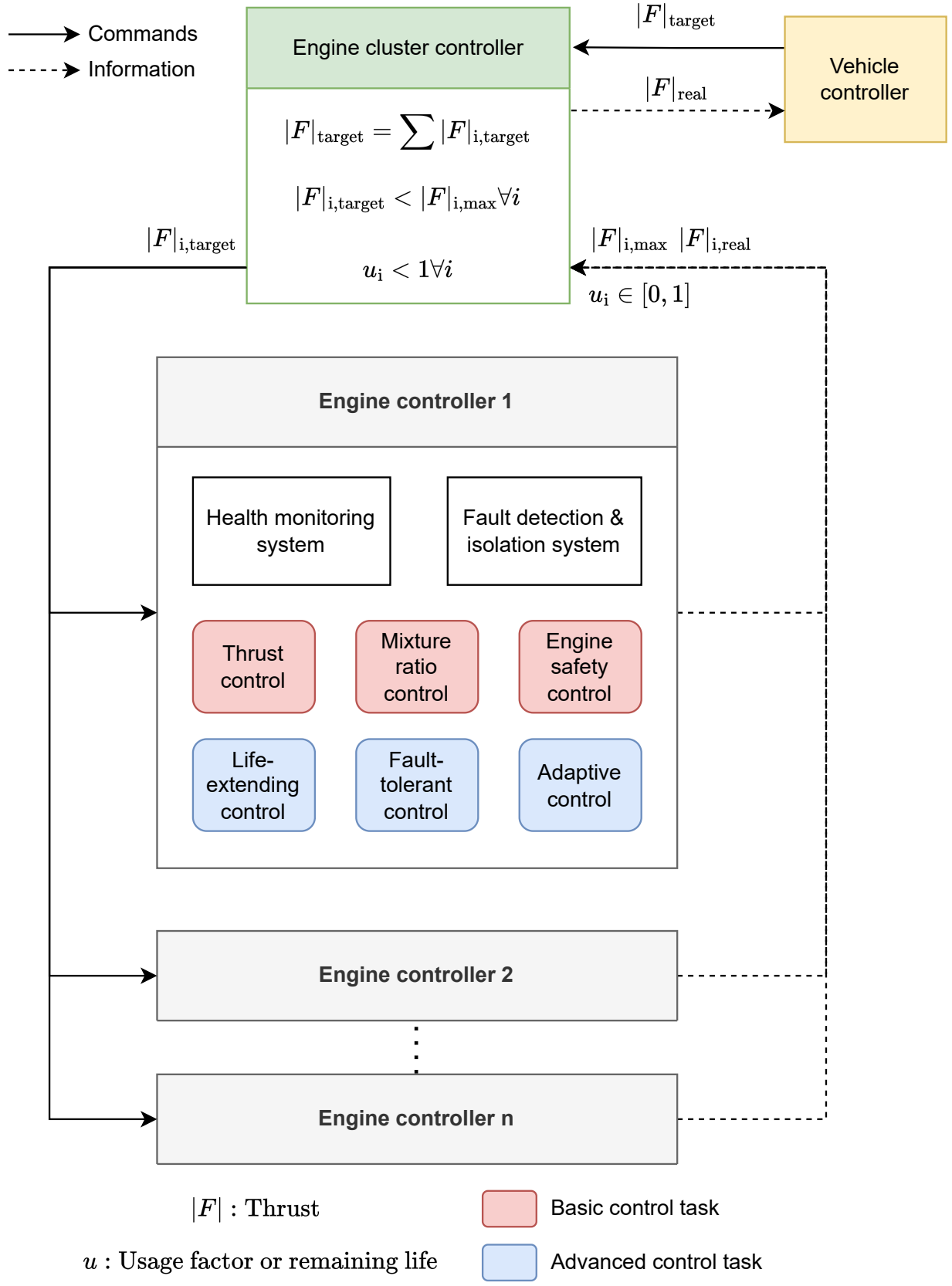


Figure 2.3.: Advanced engine controller concept for a rocket engine cluster [47]

They further report the current thrust level $F_{\mathrm{i,real}}$, the maximum attainable thrust of the engine $F_{\mathrm{i,max}}$, and a damage indicator $u \in [0, 1]$. $u = 0$ is specified for a new engine, $u = 1$ correlates with the engine reaching the end of its service life. The damage indicator u is derived by a health and usage monitoring system based on the engine's operating time and historical data on pressure measurements, rotational speeds, temperatures or vibrations. Depending on the degree of degradation, $F_{\mathrm{i,max}}$ may also be reduced.

A fault detection system is integrated to identify and isolate faults at the engine level. Faults can be caused by malfunctioning actuators, sensors, or engine components. When faults occur, the engine controller can safely shut down the engine or reconfigure the system while accepting degraded performance with reduced thrust. In addition, the controller is adaptive, adjusting its control law based on the current health status of the engine.

Conclusion for control: Cluster control extends control beyond the individual engine, making the design and testing of such systems even more complex. On the other hand, it offers an appealing opportunity to increase performance and safety while reducing recurring costs. A hierarchical approach to engine cluster control seems logical to simplify the design and testing of such a system, as opposed to having a single algorithm responsible for handling all tasks simultaneously. Future engine control developments should already consider the integration into an engine cluster control framework and provide building blocks for a life-extending and fault-tolerant control.

2.3.3. Fault-tolerant control

Fault-tolerant control [25] addresses the inherent risk of sensor, actuator, or system faults. System faults can arise from several sources, including injector blockage, combustion instability, cavitation, or leakages [92]. In addition, valves can become stuck or operate with reduced bandwidth [92]. Due to weight and space constraints, redundancy by design, which is common in the aviation industry, is impractical: it is not feasible to have a backup valve that could take over if the main valve fails. Control algorithms that can handle faults, e.g. sensor faults, are therefore becoming increasingly important.

Faults are significant changes in the dynamics of a system. They exceed small external disturbances that inherently exist in any system. While controllers are generally designed to be robust with regard to disturbances, traditional control algorithms cannot compensate for faults. Dedicated fault-tolerant architectures must be used to limit the effects of faults, as unresolved ones can lead to total system failure [2]. Fault-tolerant control techniques can be divided into passive and active systems, as shown in Figure 2.4.

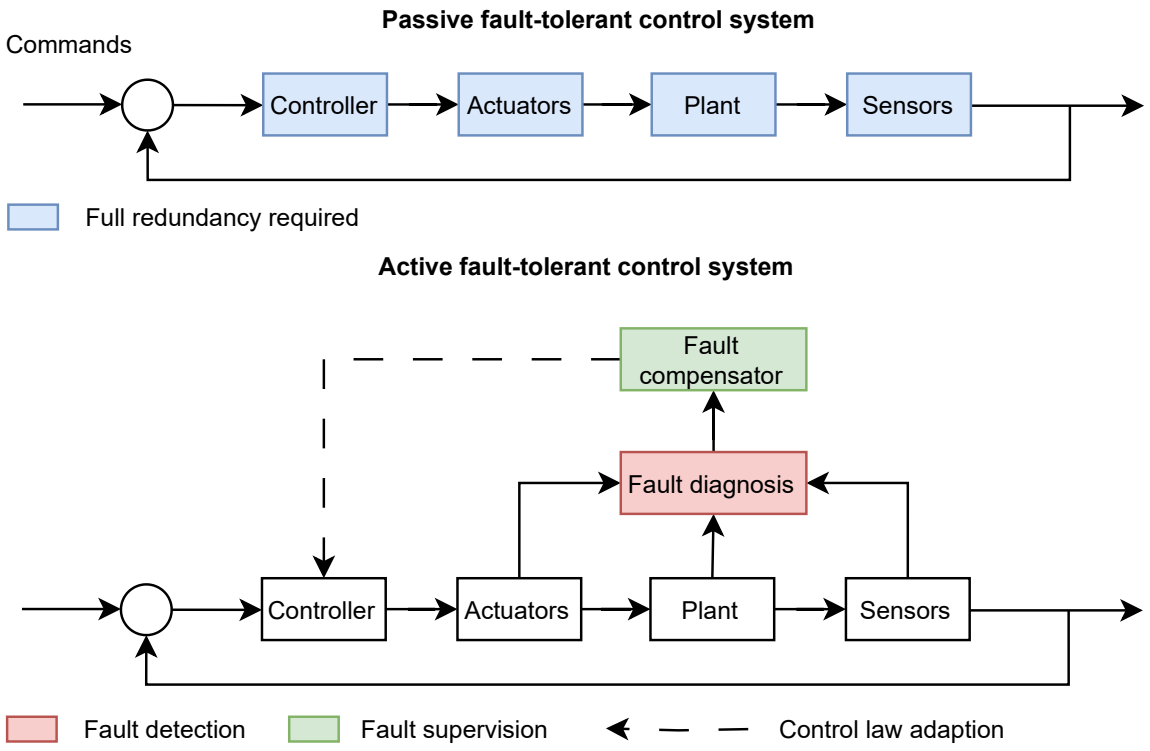


Figure 2.4.: Overview of fault-tolerant control systems. Adapted from Abbaspour et al. [2]

Passive systems use hardware redundancy to handle faults. However, achieving full passive fault tolerance means ensuring redundancy in all electronics, actuators, system components, and sensors, which is typically not feasible for flight engines. **Active systems** use real-time fault diagnosis and fault compensation to adjust or reconfigure the control law. Various active fault tolerance strategies exist, including switching-based methods, hierarchical structures, or analytical feedback compensation. Abbaspour et al. [2] provides a detailed overview of the different techniques.

Active fault-tolerant systems used in the space industry are typically switching-based approaches due to their simplicity. This approach involves switching between a set of predefined control strategies based on the type and severity of the fault. This usually means incorporating a backup controller or open-loop sequence that is activated in the event of a fault, e.g. used for Vinci [64].

Space Shuttle: Challenger's STS-51-F mission is an example of multiple sensor faults during the mission: 5:43 minutes after liftoff, the center engine shut down due to the failure of both temperature sensors in the turbine exhaust. By extending the burn time of the two remaining engines, the ascent into orbit could be continued [111]. However, the second engine also experienced a sensor failure. Only a manual deactivation of the temperature

sensor monitoring loop prevented the second engine shutdown, which would have resulted in loss of crew and vehicle. This anomaly highlights the need for a more sophisticated fault tolerant control system that responds autonomously to such faults. Typically, engine shutdown should be a last resort; operating at reduced power is preferable. In this context, Musgrave et al. [129] conducted a study on how to deal with actuator faults in real time and possibly continue operation with reduced thrust.

Conclusion for control: Dealing with sensor, actuator, and system faults is a key challenge for future intelligent rocket engine control systems due to safety requirements. However, the occurrence of faults poses a unique challenge to the control system because they fundamentally change the dynamics of the system. Control algorithms based on linearized models cannot work in these circumstances. Potential solutions are either to create multiple controllers for different fault scenarios and switch between them as needed, or to use control algorithms that can deal with faults naturally.

2.3.4. Active combustion control

Active combustion control aims to suppress combustion instabilities by actively disrupting the growth of resonant oscillations [17, 31, 81, 125]. It is still an open field with many questions regarding the selection of appropriate sensors, actuators, and real-time capabilities. Given these challenging requirements, no operational engine is known to use such a system. Yet, experiments have demonstrated the use of feedback control to suppress combustion instability in laboratory combustors [151, 153]. In recent years, research has further explored methods to predict the onset of combustion instabilities in real time [103, 132, 214]. In the future, early warning precursors and active combustion suppression may be coupled with control systems to provide increased reliability against combustion instabilities.

2.4. Characteristics of liquid rocket engines

The characteristics of the controlled system strongly influence the suitability of different control algorithms. Questions are whether the system is inherently stable and what is the system's response to disturbances: Does the system have large time delays, nonlinearities, or non-minimum phase behavior? In addition, the deviations between two systems caused by manufacturing variations, potential parameter changes during operation, and the level of expected disturbances influence which control algorithm can be used. The available sensors, actuators, and computing power may further limit the choice.

Some answers to these questions can be found in literature:

1. Liquid propellant rocket engines are inherently open-loop stable, i.e. they return to their original state in the event of perturbations [192]. This inherent stability reduces the need for active stabilization [148]. Potential unstable conditions such as combustion instabilities or critical turbopump eigenfrequencies are mitigated by design choices such as using passive elements like baffles or Helmholtz resonators [92].

2. Some engines are non-minimum phase systems where the initial system response to changes in input is reversed [148]. For example, thrust may initially decrease when the engine is throttled up by opening the flow control valve. This initial reverse response is caused by two opposing processes with different time constants and gains [104]. First, the process with the smaller time constant dominates, causing the reverse response. Then the second process with higher gain becomes dominant. Non-minimum phase systems limit the use of simple control schemes [104].

3. System uncertainties are caused by the manufacture and operation of the engine itself. Uncertainties are due to sensor inaccuracies, machine tolerances of injector elements, or changing valve dynamics due to temperature effects [92]. Hardware variation can also increase with additive manufacturing. During operation, propellant density and therefore pressure losses changes with tank pressure and temperature. The acceleration of the vehicle introduces additional disturbances.

The engine architecture itself further influences the dynamic characteristics of the system. Simple pressure-fed engines typically have time constants of a few milliseconds, while larger turbopump cycles may require several seconds to reach steady-state conditions [92]. In expander cycle engines, phenomena with extremely different time scales are present: While the combustion processes react quickly to changes in mass flow, the heat transfer processes between the combustion chamber, the regenerative cooling circuit, and the turbopump output take longer due to the thermal masses involved [44].

The degree of coupling between different variables also depends on the engine cycle. Coupling refers to the interdependence of different components. In gas generator cycles, the gas generator and the main combustion chamber are not directly coupled because the gas generator can operate independently of the main combustion chamber. In contrast, the preburners of staged combustion engines are directly coupled to the main combustion chamber. Expander engines are also highly coupled due to the coupling between heat transfer and turbopump performance.

Characteristics of flow control valves

Since flow control valves are the main actuators used for rocket engine control, it is important to study their characteristics. Valves must be sized correctly to ensure **control authority**, which refers to how much of the total system pressure drop is caused by the valve when fully open [53]. Undersized valves provide excellent control authority at the expense of high pressure drop. Oversized valves sacrifice control accuracy as they need to be operated in a limited range. Engineering guidelines recommend that control valves contribute 35 % to 75 % of the total pressure drop when fully open [196]. Valves are sized by their flow coefficient k_v, which defines the water flow through the valve in $\mathrm{m^3\,h^{-1}}$ at a pressure drop of 1 bar at a given water temperature [206].

The second important parameter is the **valve characteristic curve**: It is an inherent property of the valve geometry and defines how k_v changes as a function of valve position. For valves with a linear characteristic, k_v changes proportionally with valve position, while equal-percentage valves open progressively more. Linear valves are typically used when the pressure drop across the valve contributes to a large portion of the total pressure drop in the system. If piping or other downstream components cause significant pressure drop, equal-percentage valve characteristics are usually used. [53]

Consider a simple hydraulic example to illustrate the effect of different valve characteristics: A valve with a flow coefficient of $k_{\mathrm{v},1} = 10\,\mathrm{m^3\,h^{-1}}$ regulates the flow through an orifice with a flow coefficient of $k_{\mathrm{v},2}$ between two reservoirs with constant pressure. The size (and therefore the flow resistance) of the orifices is varied between $k_{\mathrm{v},2} = 2\,\mathrm{m^3\,h^{-1}}$ to $20\,\mathrm{m^3\,h^{-1}}$. Figure 2.5 compares the resulting normalized flow rate as a function of valve opening for a linear and an equal percentage valve. For a large orifice with $k_{\mathrm{v},2} = 20\,\mathrm{m^3\,h^{-1}}$, the linear valve results in a preferable overall linear system curve. For smaller orifice diameters, the pressure drop across the orifices becomes dominant, resulting in a sharp and nonlinear system curve. Using the linear valve for control would cause problems because it is very sensitive at small opening angles and has almost no control authority at large opening angles. In this case, an equal-percentage valve can help linearize the system curve, thereby increasing the usable valve opening range.

Conclusion for control: Future engines are likely to incorporate more sensors for improved on-line monitoring and fault detection. New control algorithms will also benefit from the transition to all-electric valves due to the increased valve position accuracy, reduced dead times and faster response times. The selection of appropriate valves is critical when designing a control system.

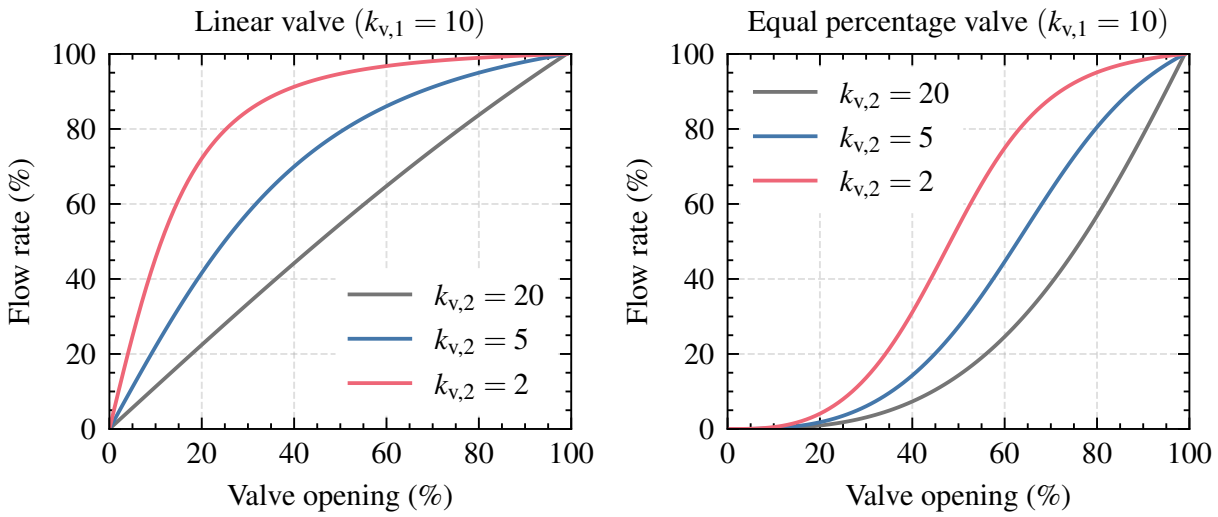


Figure 2.5.: Influence of the valve characteristics on the system curve

2.4.1. Computing hardware for space applications

Computing hardware for space applications is exposed to the harsh environment of space. In the absence of Earth's protective magnetic field, computer circuits are directly exposed to cosmic radiation. This radiation can cause miscalculations or permanently destroy the circuits. Processors on rocket engines also have to deal with severe vibrations and temperature gradients.

Space-qualified hardware is specifically designed to withstand such harsh conditions over a long period of time [113]. It uses radiation-hardened processors and comes with built-in low-level error correction mechanisms. The computing circuits are designed for extreme reliability and low power consumption, which is required due to tight energy budgets and the lack of convective cooling in space. But space hardening comes at a cost: Space-qualified processors are 1-2 orders of magnitude slower than their commercial counterparts and extremely expensive:

A common processor used for space applications is the RAD750 from BAE Systems [72]. It has been used on several missions such as the James Webb Telescope [124] or the Curiosity rover [217]. The processor has a clock speed of 200 MHz and 256 MB random-access memory (RAM). It costs about $200,000 in 2002 [161]. Its successor, the RAD5545, is probably the most powerful radiation-hardened processor available today. It is a quad-core processors, operating at 466 MHz with a power dissipation of 20 W. In Europe, the European Space Agency (ESA) uses Gaisler LEON processors. The latest addition to this family is the quad-core GR740 operating at 250 MHz [84].

If the processors are exposed to space for a limited period of time, commercial off-the-shelf components may be an option. Such processors from ARM are

currently being considered for future projects in Europe [82]. One example is the Ariane 6, which uses an ARM Cortex-R5 processor in the on-board computer for monitoring operations and telemetry [6, 137].

Table 2.1 compares space-qualified processors, denoted by a superscript 1, with commercial off-the-shelf processors, denoted by superscripts 4 and 5. The table also includes a processor that has been analyzed by Berger [19] for a neural network rocket engine controller. Computing power can be measured in Dhrystone Million Instructions per Second (DMIPS). It is the result of a standardized speed test with a specific workload mix, representative of scientific tasks [215].

Table 2.1.: Comparison of space-qualified and commercial proccesors.

Processor	Core count	Frequency	DMIPS
BAE RAD750[1] [72]	1	200 MHz	400
BAE RAD5545[1] [67, 112]	4	466 MHz	5200
LEON GR740[1] [84]	4	250 MHz	425
ARM Cortex R5[2] [98]	1	480 MHz	800
ARM Cortex M7F[3][187]	1	480 MHz	1027
ARM Cortex-A72[4] [181]	4	1500 MHz	$\approx$ 10.000
Intel i9-12900K[5] [33]	16	4800 MHz	576.000

[1]: space-qualified processors
[2]: considered for the Ariane 6 onboard computer [137]
[3]: analyzed by Berger [19] for a neural network engine controller
[4]: Raspberry Pi 4 or Samsung Galaxy S7 phone from 2016
[5]: high-end consumer processor in 2023

The lack of performance is a problem for computationally intensive applications (e.g. image processing or nonlinear optimization for real-time trajectory planning and control) [18, 120]. Ideas for using commercial off-the-shelf components to increase performance are therefore being studied intensively. One idea is to mix radiation-hardened hardware with such components [120]. Even radiation tolerant graphics processing units (GPUs) are being investigated as they enable numerically expensive, image-based machine learning [28].

Conclusion for control: Radiation and vibration are significant problems for processors used in space. Rocket engine control algorithms must deal with this inevitably low computing power. As a result, algorithms using real-time (nonlinear) optimization can only achieve a limited control frequency [148]. Numerically efficient methods, such as embedding the controller in a small neural network, thus become an interesting alternative.

2.5. Control algorithms for rocket engine control

Different control algorithms have been studied for rocket engine control in the past [148]. The choice of a particular algorithm depends on the desired performance, the available computing power, and a trade-off between simplicity and accuracy. More complex controllers can provide higher accuracy, but are usually more difficult to design and test [58]. An appropriate controller must be robust to modeling errors, parameter variations over time, disturbances, and sensor noise. This chapter introduces several techniques, namely proportional-integral (PI) controllers, full state feedback, robust control, and model predictive control (MPC). References to historical applications in space propulsion have been extracted from the survey by Perez-Roca [148].

2.5.1. System modeling

Control engineering often relies on models of the system to be controlled. Creating such models is often time-consuming and requires making assumptions about the complexity of the model. Linear control assumes a linear relationship between inputs and outputs. Linear time invariance implies that all coefficients of the underlying differential equations of the controlled system do not change with time. An example of a time-varying system is the attitude control of a spacecraft whose mass changes with fuel consumption. Several tools have been developed to study the dynamics, stability, and robustness of linear systems, such as Bode diagrams and Nyquist plots. [139]

However, real physical systems are nonlinear [58]. Causes of nonlinearities include physical processes such as friction, chemical reactions, or turbulence. Discontinuities due to physical constraints such as valve saturation or discrete events are also not captured by linear systems. For situations where nonlinear effects dominate, linear control may be inadequate [139]. If a nonlinear model of the process is available, nonlinear control techniques can be used, such as nonlinear model predictive control [8].

Linear control is often preferred, as its theory is more developed [139]. Nonlinear systems can be linearized around an equilibrium point using a truncated Taylor series expansion. Near the equilibrium point, the second and higher order terms are negligible and the linearized model approximates the nonlinear system with reasonable accuracy. This simplification introduces modeling errors that are often outweighed by the simplicity of linear models. However, as the system moves away from the equilibrium point, the higher order terms can no longer be truncated, and the linearization errors increase. This often limits the operating range of controllers using linear models. [7]

Application in rocket engine control: The survey by Pérez-Roca et al. [150] emphasizes that most rocket engine control studies have focused on linear system models for steady-state thrust and mixture ratio control. Goals include countering disturbances and manufacturing deviations. Nonlinear models have mostly been used for analysis but not for control. Notable exceptions are life-extending control with nonlinear damage models [119] or transient control of the startup based on nonlinear models [149].

2.5.2. PID control

Single-input, single-output proportional-integral-derivative (PID) controllers are widely used and accepted in industrial processes because of their simplicity and versatility [15]. The control action u is based on the error between the desired setpoint and the actual process variable $e(t) = y(t) - y_{\mathrm{ref}}(t)$. The control action is calculated using proportional (P), integral (I), and derivative (D) terms weighted by individual gains K_p, T_i, and T_d. The input/output relationship for an ideal PID controller in standard form is given as

$$u(t) = K_p \left(e(t) + \frac{1}{T_\mathrm{i}} \int_0^t e(\tau)\, d\tau + T_\mathrm{d} \frac{de(t)}{dt} \right). \tag{2.5}$$

A high proportional term results in a large change in the output for a given error. The integral term eliminates steady-state errors. The derivative term accounts for the rate of change of the error. This makes it possible to predict future error behavior and reduce overshoot, therefore improving the settling time and stability of the system. The gains can be adjusted manually or by using heuristics such as the Ziegler-Nichols rules. [15]

PID controllers are designed for linear systems. If the system has nonlinearities, large time delays, or non-minimum phase behavior, extensions are required: Gain scheduling by changing K_p, T_i, and T_d as a function of the setpoint and anti-windup techniques to deal with actuator saturation are possible modifications. Another improvement is to incorporate feedforward control and use the PID controller only for error correction. [15]

When using PID controllers for systems with multiple inputs and outputs, multiple decentralized PID controllers can be used. The idea is to divide the control problem into independent controllers for each variable, with no information exchange between them. Each controller attempts to control a single output by manipulating a single actuator. Problems arise when the variables are cross-coupled, i.e. a single actuator affects multiple variables at the same time. In this case, control loops with different bandwidths can be used, but reduced performance and stability must be expected. [7]

Application to rocket engine control:

PI controllers (without the derivative term) have been studied for thrust and mixture ratio control:

- The **Space Shuttle Main Engine** [178] uses two decoupled PI control loops at 50 Hz for thrust and mixture ratio by controlling the amount of oxygen to both preburners via two flow control valves. The control system exceeds the required accuracy of ±1.3 % in thrust and ±1.0 % in mixture ratio over a wide throttle range of 50 % to 109 % of rated thrust. The controller has an additional safety loop that can artificially reduce the thrust reference if high turbine exhaust gas temperatures are detected due to high oxygen mass flow rates in the preburners. The controller is described in more detail in Section 2.6.2.
- PI controllers are being studied for Japan's **LE-9** engine [4, 123, 189] and are used in ground tests for development and acceptance testing. The objective is to increase the number and accuracy of setpoint changes during testing to achieve significant cost benefits [123]. A control system with two independent loops is being studied for thrust and mixture ratio using feedforward models and PI controllers for error correction. During initial experimental testing, the control systems followed commands effectively, although some overshoots were observed [188]. The authors believe that this is due to the coupling of the control loops.
- Raposo [155] studied thrust and mixture ratio control for **Vinci** at the nominal operating point and during throttling. Two PID controllers were tuned on a linearized state-space model and improved with feedforward control and anti-windup techniques. The controller showed sufficient performance in maintaining operating points and transitioning to the low-thrust regime when being tested on a nonlinear simulation model. Robustness to parameter uncertainty was also evaluated. PID controllers are also used in the flight version of Vinci [64].

2.5.3. Multivariable control

Multivariable control refers to control methods for multiple-input and multiple-output (MIMO) systems. The idea is to use a single controller with multiple actuators, instead of individual controllers for each output. For coupled systems, multivariable control offers superior performance because it directly accounts for the coupling. It is also usually more robust to uncertainties and disturbances [7]. However, designing and testing can be challenging [117].

Since rocket engine control is inherently a MIMO problem, algorithms that can handle MIMO systems will naturally outperform multiple decoupled single-input, single-output control loops. Musgrave [130] developed an multivariable controller for the Space Shuttle's main engine to increase its robustness to changes in system parameters due to aging. In Europe, engineers studied multivariable control for ground testing to accelerate development. They studied a three-input, three-output controller for the Vulcain gas generator engine and a two-input, two-output controller for Vinci [64, 148].

2.5.4. Full state feedback

Full state feedback uses the entire system state to compute the control vector instead of relying only on the error term as PID controllers do. Assuming that the system state is measurable and the system is controllable, full state feedback can arbitrarily change the characteristics of the closed-loop system [42]. Full state feedback is particularly useful for MIMO systems with strong coupling and outperforms multiple decoupled single-input and single-output (SISO) controllers [7].

State space representation

Full state feedback uses system models in state space form, which are defined by a set of first order differential and algebraic equations. The state space form of a continuous, linear, time-invariant system is given as follows:

$$\dot{x}(t) = Ax(t) + Bu(t), \tag{2.6}$$

$$y(t) = Cx(t) + Du(t). \tag{2.7}$$

Equation 2.6 is the state equation, which describes the derivative of the state vector $\dot{x}$ as a linear combination of the current state x and control inputs u. A is the state matrix, which contains the system dynamics. B describes how the control inputs affect the state variables. Equation 2.7 is the output equation, which relates the system output y to x and u via the matrices C and D. The eigenvalues of A, also called the poles of the system, determine the stability. If any eigenvalue of A is positive, the system is unstable. Eigenvalues with an imaginary part indicate oscillatory properties. If all poles of the system are in the left half-plane, the system is stable. [42]

Pole placement

Pole placement is a method to design controllers by manipulating the eigenvalues of the system to achieve stable closed-loop performance. Figure 2.6

illustrates the general idea: The full system state x is fed back and multiplied by a gain matrix K to compute the control input $u = -Kx$. Substituting u back into the state equation 2.6 for the regulator case with $y_{\text{ref}} = 0$ yields

$$\dot{x}(t) = (A - BK)x(t). \tag{2.8}$$

$(A - BK)$ is the new system matrix of the closed-loop system. By choosing K such that all eigenvalues of $(A - BK)$ are in the left half plane, even potentially unstable systems with positive eigenvalues can be stabilized.

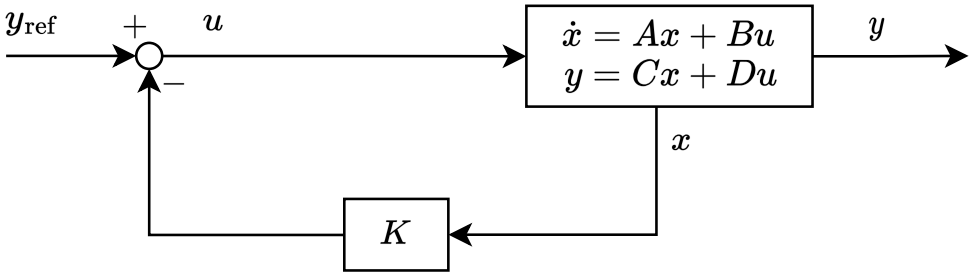


Figure 2.6.: Full state feedback control loop

The downside of pole placement, especially for MIMO and complex systems, is that it is often not clear where exactly to place the system's poles. Instead, control engineers want to tune the controller based on practical guidelines such as *"a tighter control of output y_1 is required"* or *"reduce the control usage of actuator u_1"*. [7]

Optimal control via LQR

The Linear Quadratic Regulator (LQR) [7] is a special form of full state feedback that provides a more intuitive way to compute the gain matrix K. LQR seeks to minimize a quadratic cost function

$$J = \int_0^\infty \left(x(t)^T Q x(t) + u(t)^T R u(t) \right) dt,$$

which combines the deviations of the system states and the control effort. The matrix Q is used to weight the importance of state deviations. R weighs the cost of the control effort. The formal solution for minimizing J is a linear time invariant state feedback of the form $u(t) = -Kx(t)$.

LQR results in a control policy with large stability margins. It can naturally work with MIMO systems and provides an intuitive design philosophy by tuning Q and R. However, it is designed for linear systems without constraints.

In the presence of nonlinearities, optimality and robustness are no longer guaranteed. [15]

Application to rocket engine control: Full state feedback was studied for the Space Shuttle Main Engine by Musgrave et al. [129] and Musgrave [130]. By using two additional control valves, the baseline thrust and mixture ratio controller was extended to include the turbine outlet temperature. The goal was to minimize temperature overshoots that lead to turbine blade damage during transients. Simulations demonstrated the high performance of the linear full-state feedback controller on a nonlinear engine model.

2.5.5. Robust control

Many control algorithms or tuning rules use models to approximate the real system. However, the model is always a simplification of the more complex underlying physical processes, and modeling errors are inevitable: External disturbances are present, sensor measurements contain noise, and actuators may behave differently than modeled. Internal system parameters may change over time due to aging. Linearization and deliberately neglected dynamics introduce additional uncertainties. These factors make models only an imperfect representation of the real system [7].

A robust controller maintains stability even in the presence of model uncertainties, external disturbances, and noise [42]. Robust control theory has developed tools to quantify the robustness of a system (e.g. gain and phase margins) and to synthesize robust controllers. The goal is to answer the question of how much uncertainty in system parameters and external disturbances a controller can handle before it becomes unstable.

H_∞ and μ-synthesis are optimization-based methods that include external disturbances and parameters uncertainties into the controller design [109]. H_∞ attempts to minimize the worst-case effect of external disturbances. The optimization minimizes the gain of the transfer function between external disturbances and performance measures, even when the process has uncertainties. μ-synthesis focuses on internal parameter variations of the system. Robust control can also be applied to classical PID controllers by explicitly incorporating system uncertainties into the gain tuning process [63].

The increased robustness is a trade-off with performance and often leads to conservative controllers [176]. When dealing with systems whose parameters vary slowly, e.g. due to aging, robust control may be suboptimal and adaptive control can significantly improve performance beyond robust control [211].

Application to rocket engine control: Lorenzo et al. [119] have studied robust control for the Space Shuttle Main Engine using H_∞ methods based on

desired robustness bounds. According to Pérez-Roca et al. [150], Saudemont and Le Gonidec [170] also studied a robust controller for the Vulcain engine using H_∞ in an internal ArianeGroup study.

2.5.6. Model predictive control

Model predictive control (MPC) uses a mathematical system model to predict future system states over a finite time horizon [8, 156]. This prediction allows MPC to solve an optimization problem at each sampling instant, yielding a sequence of optimal control action that minimize a cost function. The first action is then executed, and the optimization is repeated at the next time step, providing real-time feedback control. MPC is used in various applications, from process industries such as catalytic crackers in oil refineries to power electronics, robotics, or cruise control in the automotive sector [176]. Sampling times vary from seconds to minutes in process industries to sub-milliseconds in power electronics. MPC is particularly successful because it can naturally handle constraints and incorporate economic objectives, such as minimizing fuel consumption, into the cost function.

Figure 2.7 illustrates the block diagram of MPC: At each time step k, a constrained optimization problem is solved to determine the optimal control actions that minimizes a cost function J. J is often given by reducing the tracking error between a reference vector r and the system output y, while satisfying input and output constraints. The result is an vector of optimal control actions, where the first action u is executed and held constant until the next time step.

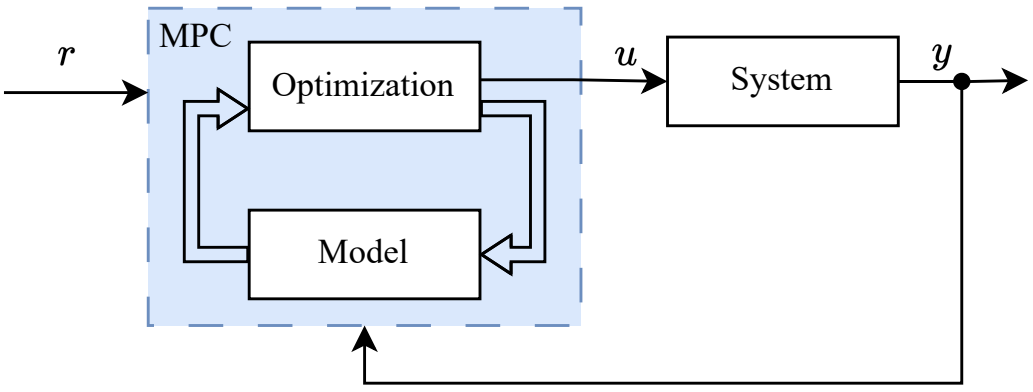


Figure 2.7.: Block diagram of the control loop in MPC (in accordance to [176])

Mathematically, the optimization problem is defined by:

$$\begin{aligned} & \min_{u} \sum_{k=1}^{N2} \|r_k - y_k\|^2 + \|\Delta u_k\|^2 \\ \text{subject to} \quad & x_{k+1} = f_x(x_{k+1}, x_k, u_k) \\ & y_k = f_y(x_k, u_k) \\ & u_{\text{lb}} \leq u_k \leq u_{\text{ub}} \\ & y_{\text{lb}} \leq y_k \leq y_{\text{ub}} \end{aligned}$$

Here, r_k is the reference value at time interval k. $y_{\text{lb}}, y_{\text{ub}}, u_{\text{lb}}, u_{\text{ub}}$ are the lower and upper bounds of the input and output constraints. The prediction horizon N_2 must be long enough to predict the effects of control actions. However, its length is limited by the increasing computational complexity of solving the optimization problem for a large prediction horizon.

Challenges of MPC include the need for an accurate mathematical system model and the high computational demands associated with the real-time optimization process. The model must capture the underlying system dynamics, but it must be simple enough to allow real-time control. Consequently, a trade-off between model complexity and accuracy is inevitable. In many cases, simple linear models will be suitable. But the use of small linear models introduces modeling errors to which the MPC controller must be robust.

A relatively new research direction is to couple MPC and machine learning. **Neural MPC** [167] uses a neural network as the system model, leveraging the capabilities of neural networks in modeling complex physical processes with low computational complexity. Neural networks can also be trained on simulated or experimental data, lowering the burden of model generation. **Approximate MPC** [83, 89] uses a neural network to approximate the control policy. First, a standard MPC scheme is designed. It is then applied to a wide range of inputs and the resulting control actions are saved. Then, a neural network is trained to approximate the standard MPC scheme, significantly lowering the real-time computational demands.

Application to rocket engine control: MPC appears to be well suited to rocket engine control. Since it can control systems with multiple inputs and outputs, and naturally include constraints into the optimization, MPC is appropriate for current and future advanced engine control tasks. Even life-extending control seems to be possible by including an economic term in the cost function that minimizes damage rates. By using more complex, nonlinear system models, it may even be possible to control the challenging phases of engine startup and shutdown. An open question is whether the slow

computing hardware used for space applications is powerful enough to achieve a sufficient control frequency, especially for advanced control scenarios.

A control scheme similar to MPC that uses an internal predictive model was studied for ground tests of the **Vulcain 1 and Vulcain 2** engine [64]. The aim was to control the main combustion chamber pressure, mixture ratio and gas generator temperature. The controller handles setpoint changes during ground testing, allowing a wider operating envelope to be tested in a single hot-fire test. This reduced costs and accelerated the development program. This strategy was later extended to the BOREAS demonstrator, which is scheduled for testing in 2024 [64].

Another notable study of MPC for rocket engine control was conducted by Perez-Roca [148]. The author developed a MPC controller for the **Vulcain 1** and **Prometheus** engine, aiming to control the highly transient phases during engine startup. The goal of the study was to reach predefined reference values of pressure and mixture ratio for the main combustion chamber and the gas generator by manipulating three control valves simultaneously. Closed-loop control starts after the combustion chamber is ignited, 1.5 s after opening the main valves. The sampling time is given with 10 ms and a prediction horizon of $N_2 = 10$ is used.

The MPC controller uses a linear state-space model derived from physical equations. Model generation started by constructing a nonlinear state space model consisting of 12 states and 5 inputs. Compared to a sophisticated simulation, the nonlinear model shows steady-state errors of 5 % to 8 % and transient errors of up to 15 %. These discrepancies are attributed to deliberately neglected effects such as the temperature effect on some thermodynamic properties. The nonlinear model is linearized around an equilibrium point, further increasing modeling errors. Despite these deviations, the author reports promising control accuracy with steady-state errors below 1 %. The MPC controller is able to satisfy all constraints even when parameter uncertainties are taken into account. However, some overshoots are observed, an effect attributed to the nonlinear influence of the turbine starter, which is not captured by the linearized model.

The author concludes that the ability of the MPC controller to handle constraints during the startup phase can minimize engine damage and potentially reduce operating costs by extending the life of the engine. However, the author estimates that the achievable control frequency on state-of-the-art space processors is a factor of ten slower than necessary. In addition, building the mathematical system model was challenging, limiting the flexibility and adaptability of the approach.

2.6. Examples of rocket engine control

This section provides examples of throttling and thrust requirements for different mission scenarios, such as orbital ascent and landing. Historical missions are the baseline for future controller developments. In addition, the Space Shuttle Main Engine (SSME) controller is presented.

2.6.1. Throttling and thrust requirements

Most historical engines were designed to operate at a continuous thrust level [30], but many scenarios benefit from continuous throttling capabilities, such as planetary entry and descent, space rendezvous, or hovering [148]. Examples of throttleable engines are the SSME with a throttling range between 67 % and 109 % of its maximum thrust in 1 % increments [163]. Falcon 9's Merlin M1D vacuum engine can throttle down to about 65 % [186]. The new Prometheus engine will have a throttling range down to 30 % [152].

Retropropulsive landing and powered-descent

Retropropulsive landing of the first stage of reusable launchers and powered-descent maneuvers for interplanetary missions require rapid throttling with significant and immediate thrust changes. Precisely following the optimal flight trajectory can significantly reduce propellant requirements. For example, Scharf et al. [172] highlight that using a sub-optimal trajectory planning algorithm of the Apollo-era for the Curiosity rover's descent would have required replacing 70 % of the rover's mass with propellant.

Blackmore [24], jointly responsible for the first successful landing of the Falcon 9 first stage, provides insights into the challenges of precision landing on a planetary surface using propulsion with an accuracy of a few meters. He argues that these requirements leave little room for error and require high-precision thrust control during to reach the intended target location.

Blackmore also addresses the challenge of calculating the landing trajectory: It has to be planned in real-time on board during the flight and cannot be pre-programmed. This is because, firstly, the state of the vehicle cannot be fully predicted in advance due to atmospheric uncertainties. Secondly, winds during re-entry further disturb the trajectory, with high altitude winds on Earth, for example, often exceeding 100 miles per hour [24]. Thirdly, uncertainties in the exact mass of the vehicle at the start of the powered descent phase introduce additional uncertainties. As a result, thrust requirements must be continuously updated during the flight.

CALLISTO (Cooperative Action Leading to Launcher Innovation in Stage Toss-back Operations) is a sub-scale vertical take-off, vertical landing rocket stage developed in collaboration between JAXA, CNES and DLR [96]. Its purpose is to demonstrate technologies for future reusable launch vehicles. CALLISTO uses a modified 40 kN LOX/LH2 RSR engine with continuous throttling capability, allowing thrust adjustments between 16 kN and 46 kN. CALLISTO also uses active thrust control throughout the ascent, boostback and landing phases. The project team emphasizes the need for precise control to ensure a successful landing, stating: *"A perfect coordination of thrust, position, and velocity is, in fact, required to ensure that the impact does not exceed the structural loads that the legs can withstand"* [166].

Example of ascending into orbit

Figure 2.8 shows the percentage thrust for all three SSME engines during ascent into orbit of the first Space Shuttle flight STS-1. Upon ignition, the engine operates at full thrust. During the phase of maximum aerodynamic pressure, the engine is trottled down to reduce aerodynamic loads on the vehicle. Towards the end of the mission, the thrust is gradually reduced to limit the vehicle's acceleration.[1]

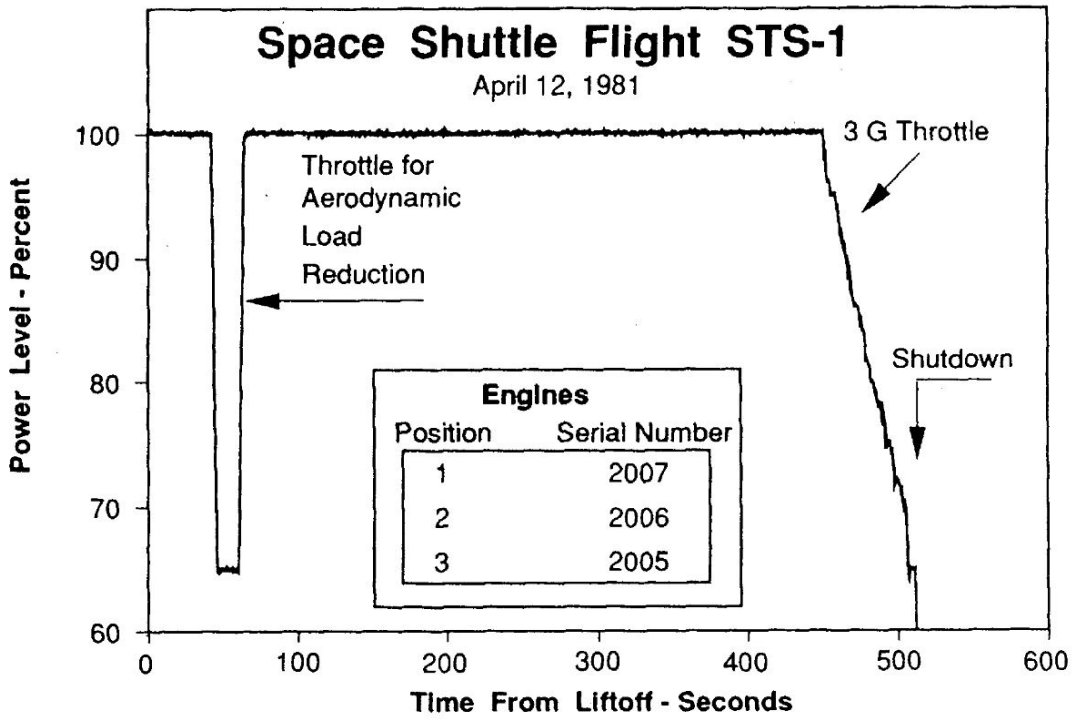


Figure 2.8.: Thrust profile of Space Shuttle flight STS-1. Reprinted with the permission of AAS from Biggs [23], 1992.

[1] It is remarkable how similar the three engines perform in this first flight of the SSME.

Conclusion for control: Continuous throttling capabilities are critical for the success of future missions, and many engines currently under development incorporate such features [184]. Thrust control must be accurate, as non-optimal control can lead to sub-optimal trajectories, requiring large fuel margins. In the case of retropropulsive landing, the propellant needed for re-entry and landing must be transported into space, making it even more important to be as efficient as possible. One can also assume that precise and fast thrust control improves landing accuracy and reduces fuel consumption during powered descent. In addition, a smoother and more precise touchdown reduces the structural requirements for the landing gear, resulting in lower structural weights. Future engine control systems therefore have to deal with all these aspects effectively: the engine needs to be throttleble in a large operational envelope, and throttling has to be as precise and fast as possible while avoiding damage to the engine itself.

2.6.2. The Space Shuttle Main Engine controller

The SSME is a fuel-rich staged combustion engine. It was developed for the Space Shuttle Orbiter, which had its maiden flight in 1981. Now, 40 years later, it is being used again on NASA's Space Launch System [134]. The engine generates 2090 kN of vacuum thrust at a combustion chamber pressure of 206 bar. It is an extremely complicated engine with a fuel-rich staged combustion cycle architecture. All major components, i.e. the main combustion chamber, two preburners, two high-pressure, and two low-pressure turbopumps need to be operated in a controlled manner.

The SSME has an electronic digital control unit responsible for repeatable engine ignitions, closed-loop control of thrust and mixture ratio, and monitoring of critical parameters during operation. Due to the complex engine cycle and the safety requirements of human-rated operation, the safety requirements for stability, redundancy, and performance were immense [16, 23, 130, 178]. The engine was designed for reusability, but various problems led to a shorter than expected service life and the engine suffered from problems due to damage accumulation [35]. After each flight, all three engines were removed from the orbiter, disassembled, and refurbished for the next launch [23].

Start sequence of the SSME

The start sequence is probably one of the most complex start sequences developed for liquid rocket engines. Its development required 37 hot-fire tests, during which 13 turbopump replacements were necessary due to fatal

hardware damage [23]. Major challenges included starting the turbopumps without overspinning, ensuring reliable ignition, and avoiding lean mixture ratios. Figure 2.9 illustrates the startup sequence, which requires simultaneous operation of five control valves to ignite the main combustion chamber and both preburners. Precise adjustments of the fuel preburner oxidizer valve (FPOV) and oxidizer preburner oxidizer valve (OPOV) are seen during the first 2 s. It can be assumed that each small adjustment in FPOV and OPOV was made in response to problems during testing.

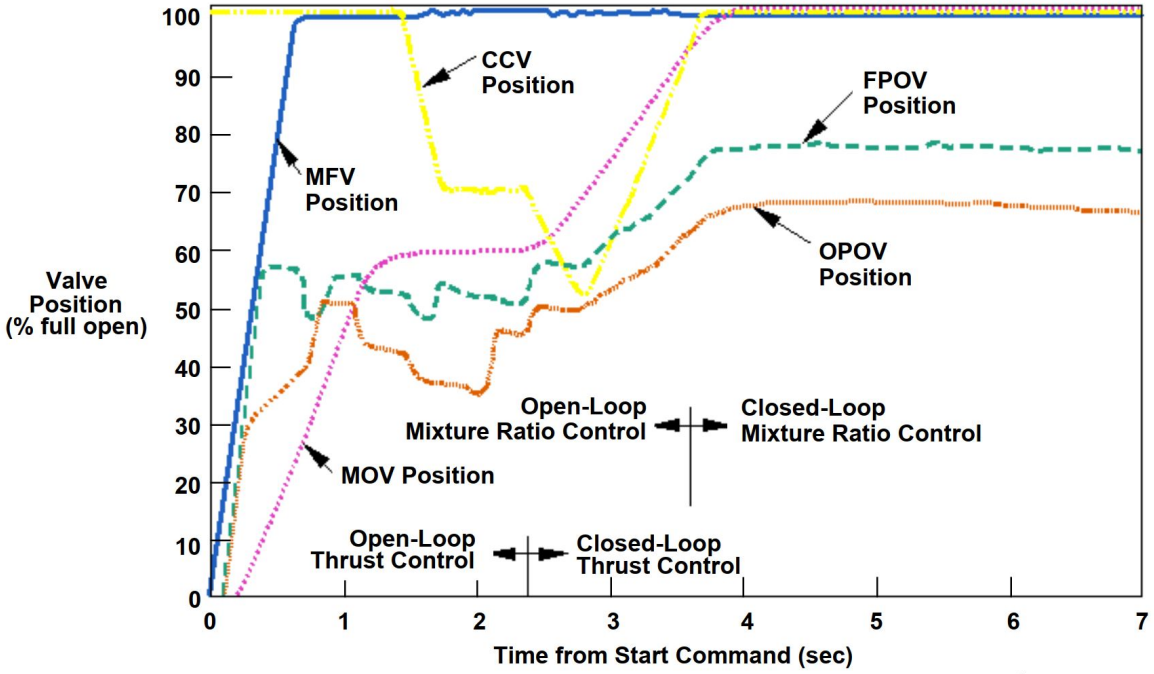


Figure 2.9.: Startup sequence of the Space Shuttle Main Engine from [163]

In his summary of the first ten years of the SSME, Biggs [23] provides insights into the precision required in operating the engine. One notable incident involved the main oxidizer valve (MOV) being opened 2 % more than required. This, combined with an unusually high breakaway torque of the high-pressure fuel turbopump, resulted in lower hydrogen flows and thus oxygen-rich combustion. The consequences were severe, leading to the total destruction of the engine. The author points out that even small valve position errors of 1 % or timing errors of a tenth of a second can cause serious damage.

Baseline SSME engine controller

Closed-loop thrust control is activated 2.2 s after ignition, followed by mixture ratio control at 3.7 s. The controller uses FPOV and OPOV to control the amount of oxygen to both preburners. It has two independent PI loops with a sampling frequency of 50 Hz.

The functional requirements of the controller are as follows:

- thrust control between 50 % to 109 % in 1 % increments.
- mixture ratio control from 5.5 to 6.5 in 0.05 increments.
- thrust and mixture ratio error below ±1.3 % and ±1 %, respectively.
- thrust change rate[2] of 540 kN s^{-1}.
- mixture ratio adjustments at a rate of 1 mixture ratio in 3 seconds.

Interestingly, the required thrust change rate is much higher than for the mixture ratio. Presumably, this helps to decouple the two PI control loops, thereby increasing the stability of the control system. Thrust commands are given by a high level flight computer. Each engine has a safety loop that can override these thrust commands: the controller monitors the turbine exhaust temperature. If it exceeds a predefined threshold, the thrust command is artificially reduced. As Seitz and Searle [178] articulates, the safety loop intervenes primarily during maximum thrust ramp-up transients to reduce temperature damage to the high-pressure turbines.

Studies for advanced engine control of the SSME

One of the main drivers for control research during the Space Shuttle era was the desire to reduce engine damage during operation by incorporating damage models into the controller [116, 119, 157, 158, 204]. Other aspects that have been explored are accommodating faults [129, 135] or treating the Shuttle's three engines as a single system rather than individual engines [136]. The baseline engine controller caused increasing turbine discharge temperatures over time [130, 158]. After prolonged engine operation, the turbine efficiency decreased due to aging, resulting in higher turbine discharge temperatures to maintain thrust requirements. This increased the risk of temperature red lines and elevated damage rates to the turbine blades.

Musgrave [130] therefore developed a multivariable controller using all six engine valves to extend closed-loop control to the high-pressure turbine outlet temperature (6 valves for 4 controlled variables). In simulation, the controller showed similar results for standard engine operation. The advantages over the baseline controller were demonstrated for a degraded engine condition by a 10 % reduction in high pressure turbine efficiency. Here, the multivariable controller reduced the turbine exhaust temperature by over 140 K compared to the baseline controller, accepting a small 3 % reduction in thrust. The authors conclude that this approach could extend turbine blade life, prevent unnecessary shutdowns and allow longer engine operation.

[2]With this rate, transitioning from 100 % thrust to 109 % takes roughly a third of a second.

Conclusion for control: The baseline controller of the SSME achieved high accuracy in regulating thrust and mixture ratio. However, it was prone to causing excessive turbine discharge temperatures resulting in turbine blade damage during transients. The SSME therefore is an excellent example of the potential benefits of advanced control systems: Lorenzo et al. [117] points out that refining the control algorithm was a high priority. Some studies focused on increasing engine life [158] by avoiding excessive temperature peaks or creating a fault-tolerant architecture that would maintain engine operation in the face of aging or faults [135]. Initial studies on intelligent cluster control have also been conducted to increase overall mission success [136].

3. Reinforcement learning for control

Neural networks have achieved considerable success in system identification and control of dynamic systems [13, 74]. Their universal approximation capabilities allow them to represent any continuous function [88], making them a popular choice for modeling system dynamics and learning control policies. Their ability to generalize and their low computational complexity during real-time deployment make them a powerful tool.

For **system identification**, neural networks can learn to predict the dynamic response of a system, even when uncertainties, external inputs, or nonlinearities are taken into account [110, 154]. Model training can be done in a supervised manner using simulation data, experimental data from real systems, or a combination of both. Often, neural network-based dynamic models provide higher accuracy than simple physics-based models, which may be incomplete or even incorrect for complex systems. One idea is to use neural network-based dynamic models as a plant for optimal control. For example, real-time neural model predictive control uses a neural network as a dynamic model within a typical model predictive control optimization loop [167].

Neural networks can also learn the optimal control policy directly. One approach, called imitation learning, aims to mimic the behavior of control experts or existing controllers [93]. Approximate model predictive control uses a neural network to approximate the outputs of a regular model predictive controller, significantly reducing the computational costs during deployment [83, 89]. A third option is reinforcement learning, where the neural network is trained by interacting with a simulation or the real physical system [193].

Neural network controllers can be efficiently deployed on embedded systems due to their low computational costs, making them suitable for scenarios with limited computing power or where high control frequencies are required [94]. For example, Degrave et al. [41] demonstrated closed-loop control of magnetic fields in a Tokamak nuclear fusion reactor with a control frequency of 1 kHz. In reinforcement learning, the use of neural networks presents challenges such as the loss of convergence properties and stability guarantees [193].

The following chapter presents the basics of reinforcement learning and discusses the challenges of learning neural network-based control policies for real-world systems. Domain randomization is introduced as an approach

to overcome some of these challenges and to make reinforcement learning applicable to real-world systems. Then, the reinforcement learning setup used in this work is described. Finally, related work of reinforcement learning for control is presented, with a particular focus on the control of real physical systems and space applications.

3.1. Fundamentals of reinforcement learning

Reinforcement learning belongs to the field of machine learning, but differs from supervised and unsupervised learning. Supervised learning uses labelled datasets to learn relationships or recognize patterns, often applied to regression or classification. Unsupervised learning uses unlabelled data to find patterns or structures. Reinforcement learning attempts to maximize some measure of performance or reward by interacting directly with a system. Unlike supervised learning, it cannot rely on examples of optimal behavior, but must explore and discover these behaviors through interaction [193].

The fundamentals of reinforcement learning have been established in pioneering work by Sutton and Barto [193], Bertsekas [20], Watkins and Dayan [212] or Werbos [218]. The goal is to learn a decision policy that maximizes some measure of performance over time [218]. It can take different forms, ranging from abstract goals such as winning a game of chess, to more concrete goals such as maximizing a cost function, similar to traditional control engineering. For example, if a drone is tasked with following a trajectory while minimizing energy consumption, the measure of performance, or reward, could be expressed as the sum of two terms: one representing the deviation between the current trajectory and the desired path, and the other representing energy consumption.

In reinforcement learning, an agent interacts with its environment by performing actions and receiving feedback in the form of the current state of the system and a reward signal, as shown in Figure 3.1. At each discrete time step t, the agent chooses an action a_t based on the current state s_t of the environment and its policy π. Upon receiving the chosen action, the environment undergoes a state transition according to its transition dynamics, and the agent receives a reward signal r_{t+1}. The reward is analogous to the cost function in classical control theory and provides feedback on how well the controller is achieving the desired control objective at each time step.

The interaction with the environment can be formalized using a Markov Decision Process (MDP), a mathematical framework commonly used in dynamic programming for decision making in systems with potentially stochastic dynamics [219]. An MDP is defined by the tuple $\{S, A, R, P, \rho_0\}$, where S represents

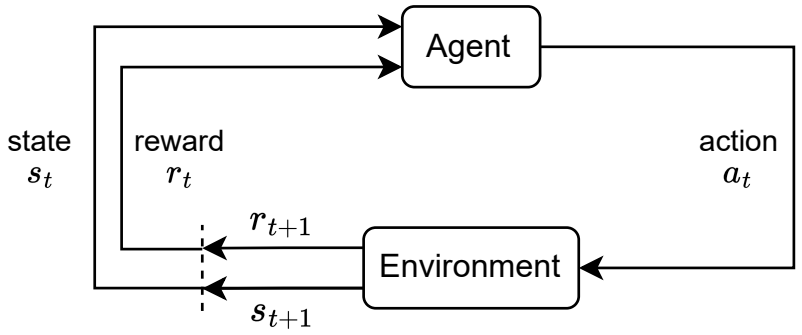


Figure 3.1.: The agent-environment interaction. In accordance with [193].

the set of possible states between which the system can transition, A denotes the set of actions that can be taken, R is the reward function for calculating rewards as $r_{t+1} = R(s_t, a_t, s_{t+1})$, P is the state transition probability function, encapsulating the system dynamics, and ρ_0 is the starting state distribution. It is crucial that the MDP satisfies the Markov property, which implies that the transition to the next state s_{t+1} depends only on the current state s_t and action a_t, without considering the history of past states of the system.

The policy π is the decision strategy for selecting actions based on observed states. π can either deterministic or stochastic. Stochastic policies are often preferred to efficiently explore the environment during training. A stochastic policy $\pi(a_t|s_t)$ defines the probability of selecting action a_t in state s_t.

Continually applying actions to the environment leads to trajectories of states, actions, and rewards. For a given trajectory, one can define the discounted cumulative reward or return by

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \tag{3.1}$$

The discount factor $\gamma \in [0, 1)$ balances the importance given to future and immediate rewards and guarantees that the return is finite.

Reinforcement learning problem

The goal of reinforcement learning is to construct a policy, called the optimal policy π_*, that maximizes the expected return J, defined by

$$J(\pi) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_{t+1} \sim P(\cdot|s_t, a_t), s_0 \sim \rho_0} \left\{ G_t | s_t = s_0 \right\}. \tag{3.2}$$

Equation 3.2 calculates the expected return, when starting in an initial state $s_0 \sim \rho_0$ and continually sampling actions according to the policy $a_t \sim \pi(\cdot|s_t)$.

The environment state changes according to the transition probability function P and the action taken $s_{t+1} \sim P(\cdot|s_t, a_t)$. $\mathbb{E}\{\cdot\}$ is the expected value of a function, which is necessary as π and P can be stochastic.

Finding π_* that maximizes Equation 3.2 can now be formulated as:

$$\pi_* = \arg\max_{\pi} J(\pi). \tag{3.3}$$

Reinforcement learning and optimal control in classical control engineering have many similarities, although the notation differs: Both methods aim to find policies that maximize some notion of performance or reward, albeit using different formalism and methods. Table 3.1 summarizes the key concepts.

Table 3.1.: Fundamental elements in reinforcement learning

Element	Description
Policy	The decision rule determining which action to take based on the current state of the system. The policy can be represented by a neural network.
Agent	The entity that interacts with the environment, collects samples, and learns the policy.
Environment	The representation of the underlying system. This could be a physic-based simulator.
Action	The control command produced by the policy and sent to the environment.
State	The current state of the system. Could be a set of sensor measurements.
Reward	A scalar value that the agent receives as feedback from the environment. It measures how well the system fulfills the user-defined reward function.

Value functions

Many reinforcement learning algorithms estimate the future returns that can be expected from a given state onward by following the current policy. The state-value $V^\pi(s)$ of a state s under a policy π is defined as the expected return when starting in state s and choosing all actions according to π. In other words, $V^\pi(s)$ contains how much return can be expected from state s when strictly following the policy. Formally, $V^\pi(s)$ is defined as:

$$V^\pi(s) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_{t+1} \sim P(\cdot|s_t, a_t)}\{G_t | s_t = s\}. \tag{3.4}$$

The action-value of an arbitrary action a in state s is the expected return if starting in state s, first taking action a, and then acting according to π:

$$Q^{\pi}(s,a) = \mathbb{E}_{s_{t+1}\sim P(\cdot|s_t,a_t)}\{G_t|s_t = s, a_t = a\}. \tag{3.5}$$

Value functions can be estimated from data by repeatedly sampling and averaging the returns of many episodes. As the number of visits to each state (or state-action pair) approaches infinity, the average return converges to the true state-value. Such methods are commonly referred to as Monte Carlo methods. Value functions help to evaluate the effect of actions taken on long-term future rewards. For many reinforcement learning algorithms, they form the basis of an update rule that works by using single samples of interactions rather than entire episodes.

Bellman equation

Value functions follow special self-consistency relations called Bellman equations, which form the basis of many reinforcement learning algorithms. The Bellman equation for the state-value function expresses the relationship between the value of the current state and its successor states. The value of a state s_t is given by the sum of the expected immediate reward r_{t+1} plus the expected discounted state-value of the successor states $V^{\pi}(s_{t+1})$:

$$V^{\pi}(s_t) = \mathbb{E}_{a_t\sim\pi(\cdot|s_t),s_{t+1}\sim P(\cdot|s_t,a_t)}\{r_{t+1} + \gamma V^{\pi}(s_{t+1})\}. \tag{3.6}$$

A similar consistency relation holds for the action-value function:

$$Q^{\pi}(s_t,a_t) = \mathbb{E}_{s_{t+1}\sim P(\cdot|s_t,a_t)}\{r_{t+1} + \gamma\mathbb{E}_{a_{t+1}\sim\pi(\cdot|s_{t+1})}\{Q^{\pi}(s_{t+1},a_{t+1})\}\}. \tag{3.7}$$

This equation expresses that the action-value for a state/action pair is given by the expected immediate reward plus the expected discounted action-value of the successor states if the next action a_{t+1} is chosen according to π.

Optimal value functions

A policy is defined as optimal if it yields the highest expected return for all states. In other words, π_* is the optimal policy if $V_*^{\pi}(s) >= V^{\pi}(s)$ for all $s \in S$. All optimal policies share the same optimal value function:

$$V_*(s) = \max_{\pi} V^{\pi}(s), \qquad Q_*(s,a) = \max_{\pi} Q^{\pi}(s,a). \tag{3.8}$$

The optimal action-value function $Q_*(s,a)$ is given as the expected immediate reward plus the expected discounted optimal value of the successor states:

$$Q_*(s,a) = \mathbb{E}_{s_{t+1}\sim P(\cdot|s_t,a_t)}\{r_{t+1} + \gamma V_*(s_{t+1})|s_t = s, a_t = a\} \tag{3.9}$$

If $Q_*(s,a)$ is known for all actions in a given state, then choosing the optimal action is trivial via:

$$a_*(s) = \arg\max_a Q_*(s,a) \tag{3.10}$$

Q-Learning

Q-learning [212] is a famous algorithm that uses the concept of optimal action-value functions and Bellman equations to derive an optimal policy for discrete state and action spaces. In Q-learning, the action values of each state-action pair are stored in a database called the Q-table. The idea is to first determine $Q_*(s,a)$ by interacting with the system. First, the Q-table is arbitrarily initialized. Using samples of observed state transitions and rewards, the table is iteratively updated using the following update rule:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha(r_{t+1} + \gamma \max_{a_{t+1}} Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)). \tag{3.11}$$

Here, $\alpha \in (0,1)$ is the learning rate that determines the importance of each sample. Equation 3.11 converges asymptotically to Q_* under mild assumptions on α and the use of an appropriate exploratory policy. Given Q_*, the optimal policy can be derived using Equation 3.10. Due to its reliance on tables, Q-Learning is limited to discrete action and state spaces of moderate size.

3.2. Deep reinforcement learning

Traditional reinforcement learning algorithms, such as Q-Learning, are not suitable for continuous state and action spaces, which are common in real-world applications. Discretising continuous spaces often results in impractical computational costs. Deep reinforcement learning offers a promising solution by replacing the discrete representations of policies and value functions with neural networks. Deep reinforcement learning algorithms have achieved impressive results in various fields, such as surpassing human performance in games like Go [174, 179, 180], demonstrating success in robotics [185], or controlling complex systems like a Tokamak nuclear fusion reactor [41]. Algorithms usually are classified as policy-based or value-based methods, but hybrid approaches are also possible.

Policy-based algorithms, such as Proximal Policy Optimization (PPO) [175], use neural networks to directly parameterize the control policy π_ϕ. Here ϕ are the tunable parameters, namely the weights and biases of the neural network. The idea is to optimize the policy directly through gradient ascent by interacting with the system. This optimization is done by updating the parameters ϕ in the direction of the policy gradient $\nabla_\phi J(\pi_\phi)$ of a suitable cost function $J(\pi_\phi)$. By estimating the policy gradient using samples, the policy is updated iteratively.

Value-based methods, such as Deep Q-Network (DQN) [126], aim to approximate $Q_*(s,a)$ with a neural network $Q_\theta(s,a) \approx Q_*(s,a)$, called the critic. The weights θ of the neural network are updated by gradient descent based on the Bellman equation, similar to Q-learning. While value-based methods are often more sample efficient, their learning process is often unstable due to the bias introduced by approximating $Q_*(s,a)$ [193].

Simple value-based methods like DQN only work for discrete action spaces. For continuous actions, the max operator in Equation 3.10 poses a challenge, as finding the optimal action itself becomes a non-trivial optimization problem, even if $Q_*(s,a)$ is known. A common solution is to introduce a second neural network $\mu_\phi(s)$, called the actor, which approximates the max operator. The optimal action is then given by

$$a_*(s) = \arg\max_a Q_*(s,a) \approx \mu_\phi(s), \tag{3.12}$$

such that

$$\max_a Q(s,a) \approx Q(s, \mu_\phi(s)). \tag{3.13}$$

Training the actor network itself is straightforward for continuous actions, since the critic network is differentiable with respect to actions. Therefore, gradient ascent can be performed on a batch of samples $(s_1, s_2, ...) \sim \mathbb{D}$ with respect to policy parameters ϕ, where the replay buffer $\mathbb{D}$ stores previously experienced samples:

$$\max_\phi \mathbb{E}_{s\sim\mathbb{D}} \left\{ Q_\theta(s, \mu_\phi(s)) \right\}. \tag{3.14}$$

Combining these ideas, training the critic network can be formulated by minimizing the mean squared Bellman error with stochastic gradient descent over a batch of samples $d = (s_t, a_t, r_{t+1}, s_{t+1}) \sim \mathbb{D}$:

$$\mathbb{L}(\theta, \mathbb{D}) = \mathbb{E}_{d\sim\mathbb{D}} \left\{ \left[Q_\theta(s_t, a_t) - (r_{t+1} + \gamma Q_\theta(s_{t+1}, \mu_\phi(s_{t+1}))) \right]^2 \right\}. \tag{3.15}$$

Soft Actor-Critic

Soft Actor-Critic (SAC) [71] is a popular algorithm for controlling real-world systems. It addresses two major challenges of many deep reinforcement learning algorithms that hinder their applicability to real-world systems: low sample efficiency and sensitivity to hyperparameters. It simultaneously trains two neural networks to learn the action value function and the policy. Compared to similar algorithms such as Deep Deterministic Policy Gradient (DDPG) [115], which use deterministic policies, SAC introduces a stochastic policy. The idea is to maximize not only the expected return, but also the entropy, or randomness, of the policy.

Algorithms that maximize entropy have demonstrated a higher degree of robustness against external disturbances and uncertainties in system dynamics, as shown by Eysenbach and Levine [56]. Moreover, the authors suggest that such algorithms may better generalize to new tasks compared to those using deterministic policies and can more efficiently explore the environment. SAC therefore learns a policy that not only maximizes performances, but also its entropy. In this way, the algorithm explores the environment effectively. Entropy regularization also avoids overfitting to simulator-specific properties.

The entropy of a policy $\mathcal{H}$ is calculated by

$$\mathcal{H}(\pi) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)} \left\{ -\log \pi(a_t|s_t) \right\}. \tag{3.16}$$

To include the entropy regularization, Equation 3.3 is extended:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_{t+1} \sim P(\cdot|s_t, a_t)} \left\{ \sum_{t=0}^{\infty} \gamma^t \left(r_{t+1} + \alpha \mathcal{H}(\pi) \right) \right\}. \tag{3.17}$$

Here, α is the entropy regularization coefficient, balancing the trade-off between maximizing return and entropy. This parameter explicitly controls the exploration-exploitation tradeoff: higher values of α encourage more exploration, while lower values favor exploitation. SAC automates the adjustment of α during training to achieve a desired target entropy, reducing the need for manual hyperparameter tuning.

SAC uses additional strategies to stabilize training and reduce dependence on hyperparameter tuning. These include the use of target networks, an off-policy formulation with a replay buffer, and the ability to train in a distributed manner using parallel simulation environments for data collection. SAC has demonstrated high performances with only modest amounts of hyperparameter tuning [71]. As a result, SAC has been studied for wide-ranging applications across various domains, including satellite attitude control [160], search and rescue task for humanoid robots [101], or autonomous driving [194].

3.3. Real-world challenges

Reinforcement learning research often takes place only in simulated environments where agents are trained and evaluated. However, deploying control policies trained in simulation to real-world systems poses additional challenges. Notably, publications by Dulac-Arnold et al. [50], Riedmiller [162], or Ibarz et al. [95] have investigated the typical challenges of deploying neural networks trained with reinforcement learning in real-world systems:

1. **Simulation-reality gap:** One of the major challenges is the so-called simulation-reality gap, which describes the unavoidable difference between the simulation model used for training and the real system. Simulations approximate the real system, but can be incomplete or even incorrect for complex systems. Modeling errors are inevitable, especially when using reduced-order models such as 0D or 1D physics, which are often used to simplify modeling and reduce computational complexity.
2. **Sensor and actuator noise:** Real world sensors and actuators often contain measurement and actuation noise. The trained policy must be robust enough to handle this noise when applied to the real system.
3. **System constraints:** Operational safety is a major concern for safety-critical applications. Improper control can lead to system degradation and irreparable damage. Constraints on physical variables must therefore be respected, even in the presence of disturbances or modeling errors.
4. **State information:** To fully describe the current state of the system and to fulfill the Markov property, the state vector must be comprehensive. For physical systems, this often means not only capturing current measurements, but also adding information about the rate of change.
5. **Multi-objective reward functions:** Combining multiple control objectives into a single reward function is a significant challenge. Reward engineering plays a critical role in reinforcement learning by adjusting the relative importance of each component. Real-world systems typically involve multiple parameters that must be controlled simultaneously, while also respecting mechanical and thermodynamic constraints across different components.
6. **System delays:** Most systems have inherent system, sensor or actuator delays. Causes include the time it takes to convert analogue measurements into digital information, or the low sampling time of some measurement principles. Actuators may also have a delay in responding to a command due to their inertia. However, the Markov framework assumes that the agent always acts on the most recent state. If not

included in the simulation, these delays can lead to discrepancies with the Markov framework, as the agent may act on outdated information. Non-zero delays can degrade the performance of the trained agent [95].

Various techniques have been investigated in the literature to address the challenges mentioned above. In this study, domain randomization and stacking of past observations is used, both of which are described below.

Domain randomization

Domain randomization is the idea of randomly changing model parameters ξ during training, such as pressure losses or efficiencies [147, 198]. By exposing the agent to model variations, it can learn a robust control policy that allows optimal responses even in the presence of modeling errors. Domain randomization is a central technique for training in simulation only, and then applying the neural network directly to the real system. Notable examples in literature include OpenAI's demonstration of a robotic hand that learns to solve a Rubik's cube entirely in simulation, and maintains its ability in the real world even when the physical properties of the hand are massively altered, such as by changing the contact forces with rubber gloves [140].

Muratore et al. [128] conducted research on unresolved issues surrounding sim-to-real techniques for robotics, with a particular focus on domain randomization. The authors address questions of how to randomize parameters, which parameters to change, when to change them, and how to handle physically implausible parameter combinations. They further formalized the idea of domain randomization by extending the objective function of Equation 3.2 to include an expectation over ξ. With the formulation in Equation 3.18, the agent aims to maximize the expected return not only for the nominal simulation, but for a distribution of ξ, thereby learning a robust policy:

$$J(\pi) = \mathbb{E}_{\xi} \sim p(\xi) \left\{ \mathbb{E}_{a_t \sim \pi(\cdot|s_t), s_{t+1} \sim P(\cdot|s_t, a_t), s_0 \sim \rho} \left\{ G_t | s_t = s_0 \right\} \right\}. \tag{3.18}$$

Stacking of past observations

In many real-world problems, the agent can only observe a partial or noisy representation of the environment's full state, e.g. due to limited available sensors. This reduced set of information is called the observation space and reflects what the agent perceives in each time step. For many problems, additional information is necessary to fully describe the current state of the system. Knowledge of past observations and actions provides valuable insights beyond the current time step. History allows the agent to learn system delays and infer details about the time-dependent behavior of the system, such as

the rate at which variables change. In addition, history enhances robustness to model parameters by allowing the agent to leverage the capability of an observer for online system identification [221]. Knowledge of past observations and actions also enables agents to learn in non-stationary systems where component parameters may change over time [50].

There are several approaches for integrating history. When using feed-forward networks, the inputs can be augmented with a history of previously observed states and actions. This stacking of observations effectively provides a memory-like history. Another option is to use recurrent neural networks, such as Long Short-Term Memory (LSTM) policies [147, 171]. While research by Peng et al. [147] suggests that recurrent network architectures may outperform the stacking approach, the original implementation of SAC uses a feed-forward neural network. Therefore, this study uses the method of stacking observations and a feed-forward network architecture.

Lets define $\tilde{o}(t) = \tilde{o}_t \in \mathbb{R}^n$ as the vector of n sensor measurements of the current time step t. When using observation stacking, the observation vector $o(t) = o_t$ that the agent receives is extended with a history of previous measurements. Mathematically, $o(t)$ is given as

$$o(t) = \bigoplus_{k=0}^{N_p} \tilde{o}(t - k\Delta t) = \begin{bmatrix} \tilde{o}(t) \\ \tilde{o}(t - \Delta t) \\ \dots \\ \tilde{o}(t - N_p \Delta t) \end{bmatrix}, \tag{3.19}$$

where N_p is the number of stacked observations. The operator $\oplus$ is the direct sum of the individual vectors, which concatenates them into a single extended vector. The resulting observation vector o_t has a dimension of $(N_p + 1) \cdot n$, where $N_p + 1$ represents the current and past stacked measurements.

3.4. Related work

Deep reinforcement learning has shown remarkable success in mastering complex board and computer games like Go [174, 179, 180], chess [174, 179], or StarCraft [207], surpassing the performance of human professionals. While these achievements highlight the potential of reinforcement learning for solving complex control problems, it's worth noting that the training and application of these controllers typically are performed within the same environment with known rules and dynamics.

Recent research has also focused on transferring policies trained in simulation to real-world scenarios using domain randomization [128]: In robotics,

reinforcement learning has been applied to control tasks for land-, air-, and water-based robots, ranging from navigation with obstacle avoidance to disaster management tasks [185]. For example, Peng et al. [147] demonstrated the effectiveness of domain randomization in a control task of a robot arm pushing an object to different locations on a table. The controller received input data of 52 positions and velocities of the arm's joints and the object, and outputs target joint angles for a low-level position controller. During training, physical parameters of the robot arm, the object, and the table, such as mass, damping, and friction, were randomized. Despite modeling deviations, the policy was successfully transferred to the real-world robot arm.

The control of magnetic fields within a Tokamak nuclear fusion reactor [41] is one of the most remarkable applications of reinforcement learning to a real-world system. The authors developed a controller for adjusting the shape and position of the plasma inside the reactor by simultaneously actuating 19 magnetic coils at an extreme control frequency of 1 kHz. Their study showed that the neural network-based control surpasses the performance of the current state-of-the-art control logic, which relies on a set of 19 proportional-integral-derivative (PID) controllers. Compared to this nested ensemble, reinforcement learning provides a more intuitive way of defining plasma fields, reducing the time required for test preparation and facilitating the exploration of alternative plasma shapes. The authors speculate that reinforcement learning can accelerate fusion science and contribute to future Tokamak developments.

In more detail, the authors use an actor-critic algorithm similar to SAC, called Maximum a Posteriori Policy Optimization [3]. The actor is a small feed-forward neural network with three layers and 256 neurons. The critic is a 256 unit LSTM network. The authors show that this asymmetric architecture of a small feed-forward actor and a recurrent LSTM critic, outperforms a symmetric architecture using only feed-forward networks. The controller is trained only in simulation on a physics simulator. The computational effort is significant: The simulation of a 2 second episode takes about 5 hours. To compensate for the slow simulation speed, 5000 simulations are run in parallel, resulting in training times of a few days. Domain randomization is used to increase the robustness of the controller. In a follow-up study [199], algorithmic improvements such as reward shaping and integrator feedback are investigated to reduce the required training and improve performance.

In the space domain, reinforcement learning has been studied to enhance the autonomous on-board capabilities of spacecrafts. Various studies aim to minimize reliance on human intervention and increase the spacecraft's ability to cope with uncertain and changing settings. New autonomous on-board systems may enable entirely new types of missions that are not currently feasible [197]. Examples of such studies include investigations of guidance,

navigation, and control for autonomous landing on celestial bodies [177], orbit transfers [85], space debris removal [160], missile guidance [62], and docking maneuvers with both controlled and uncontrolled spacecrafts [51, 57, 91].

3.5. Reinforcement learning setup of this work

The reinforcement learning setup used in this work has the overall goal of flexibility. The idea is that once a specific algorithm, set of hyperparameter and the training process have been established, the same setup can be applied to different rocket engine control tasks. This helps to develop new controllers for different engines, or to adapt existing ones to new control requirements. As a result, the SAC algorithm is chosen for its sample efficiency, minimal need for hyperparameter tuning, and robustness to noise and modeling errors due to its stochastic policy. The Ray RLlib framework [114] provides the implementation in TensorFlow [1] and allows parallel data collection by running multiple simulation environments simultaneously to reduce training time.

All **simulation models** are built in EcosimPro, the state-of-the-art modeling and simulation tool for space applications. Physics-based models of various components relevant for liquid propellant rocket engines are available in the European Space Propulsion System Simulation (ESPSS) libraries, which are commissioned by the European Space Agency (ESA). The libraries have been used to simulate liquid [37, 216, 97], solid [164] and electric [165] rocket engines and are widely used in the European space industry. EcosimPro simulations can be interfaced with Python via a custom interface class, allowing stepwise integration of the model. Furthermore, the simulation is encapsulated in an OpenAI Gym environment [27], which provides the framework for defining the state and action space. Training is performed with 12 parallel workers on a desktop workstation equipped with an Intel Xeon W-2265 processor, 128 GB RAM, and an NVIDIA RTX 3090 graphics processing unit (GPU).

A similar set of **hyperparameters** as in the original publication [71] is used, as listed in Table A.1. To fine-tune the hyperparmaters for rocket engine control tasks, Adler [5] and Berger [19] conducted a hyperparameter study of different discount factors, learning rates, target entropy parameters, and replay buffer sizes. In terms of neural network size, the same architecture as in the original publication of 2 layers and 256 neurons is used. This results in $256^2 + 2 \cdot 256 = 65792$ tunable parameters for each network (omitting the input and output layers, which depend on the dimension of the action and observation space). A study by Mysore et al. [131] showed that it is possible to reduce the network sizes by an average of 77 % for typical toy problems. This could be explored in a follow-up study to reduce numerical complexity.

Domain randomization and **observation stacking** is used to make the neural network robust to modeling errors. During training, model parameters that most influence the dynamics of each component are varied. Possible parameters are pressure loss coefficients, heat transfer rates, valve characteristics or turbine and pump efficiencies. Transient model parameters such as moments of inertia or the transfer function of valves are also randomized. The setup of domain randomization and the number of stacked observations depends on the test case, which provides further information on the randomized variables and the range of randomization.

Following an approach similar to the Rubik's Cube example of OpenAI [140], the amount of domain randomization is gradually increased during training. This strategy, known as **curriculum learning** [128], can decrease the necessary training time. Initially, the training starts with the nominal model parameters without domain randomization. When the controller achieves satisfactory performance, domain randomization is activated, where selected model parameters ξ are randomly modified at the start of each simulation using a truncated normal probability distribution with standard deviation σ, as described by Adler [5]. σ is gradually increased until it approximates a uniform distribution within the specified parameter bounds.

Figure 3.2 shows the truncated normal probability distribution as a function of σ for one million samples. The left figure considers a symmetrical case for domain randomization of the random variable ξ with upper and lower bounds ξ_{min} and ξ_{max} of ± 0.2 around the default value $\xi_{\text{nom.}} = 1.0$. The right figure shows an asymmetrical case with a lower boundary of 0.8 and an upper boundary of 1.4. Note how the distribution quickly widens as σ increases. At $\sigma = 2.0$, the distribution has approximated a uniform distribution.

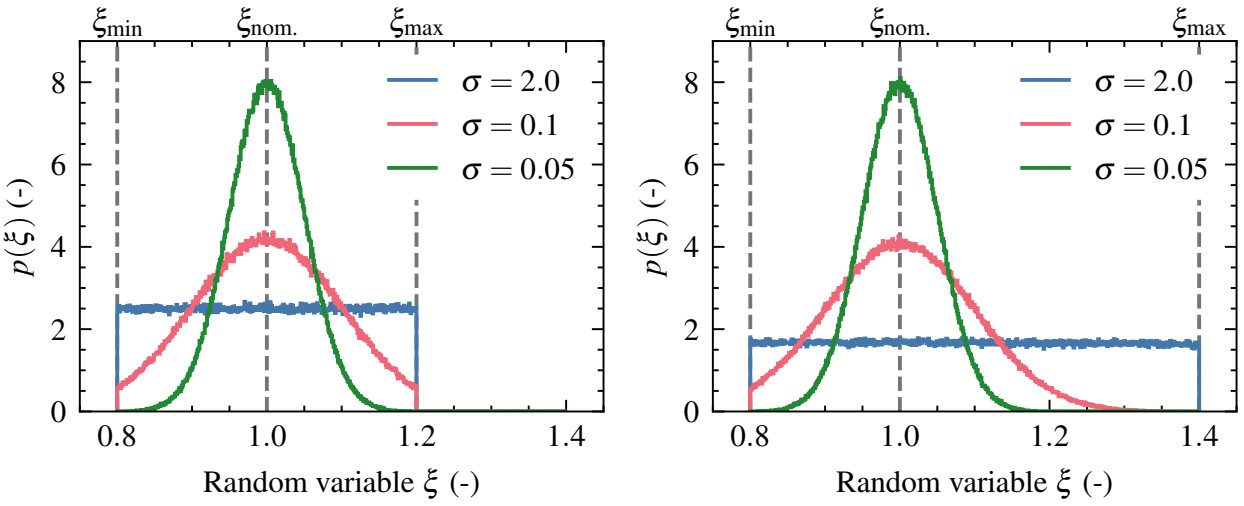


Figure 3.2.: Truncated normal distribution for domain randomization

4. LUMEN project

This research on neural network-based rocket engine control is being conducted within the Liquid Upper Stage Demonstrator Engine (LUMEN) project. LUMEN is a modular expander bleed engine developed by DLR Lampoldshausen [200]. The project began in 2016 with the definition of functional requirements [40]. By 2023, all components were designed, manufactured, and tested [200]. In 2024, LUMEN was successfully put into service, including the first real-world test of neural-network based rocket engine control.

The main objective of the project is to improve system-level understanding, which is essential for the assessment of future engine and launcher developments. But LUMEN is also ideal for studying control algorithms due to its heavy instrumentation, precise electrical control valves, and complex expander-bleed engine cycle. With up to six control valves, LUMEN allows simultaneous control of multiple variables, exceeding the complexity of classical thrust and mixture ratio control.

The development of a neural-network based engine control for LUMEN requires a transient simulation model and a detailed understanding of the engine and its operational challenges and constraints. The following chapter is dedicated to this task by performing the following steps:

1. Discussing the engine architecture (4.1) and giving an overview of the sub-component test campaigns (4.1.1).
2. Analyzing operational challenges and constraints that must be respected by the engine controller (4.1.2).
3. Modeling the system in EcosimPro (4.2).
4. Comparing the steady and transient simulation to experimental data from component tests (4.2).
5. Analyzing the input-output behavior with step functions (4.3).

The aim is to develop a transient simulation model of LUMEN that accurately represents both steady-state and transient phases. Each component of the model will be compared to experimental data. Following this, the controller's functional requirements can be defined based on the operational challenges.

4.1. Engine description

LUMEN is an expander-bleed rocket engine that burns liquid natural gas (LNG) and liquid oxygen (LOX). Figure 4.1 shows the engine flow diagram. The fuel is used to cool the main combustion chamber (MCC) wall by passing through channels manufactured directly within the chamber wall in a counter-flow arrangement. After picking up energy in the cooling channels, some of the heated methane is then diverted to power the turbines. Afterwards, the heated methane is vented without being burned in the combustion chamber, therefore LUMEN is classified as an open cycle. The other portion of the heated methane is remixed into the LNG flow via a fuel mixer to actively control the injector (INJ) temperature. Engines with a similar architecture are the LE-5A and LE-9 [123].

The two independent turbopumps are arranged in parallel, which simplifies their design and the control of the engine. The turbopump bearings are lubricated by oil, providing long service life and thermal management [169]. The turbines are supersonic partial admission impulse turbines, designed in-house at DLR [75, 76].

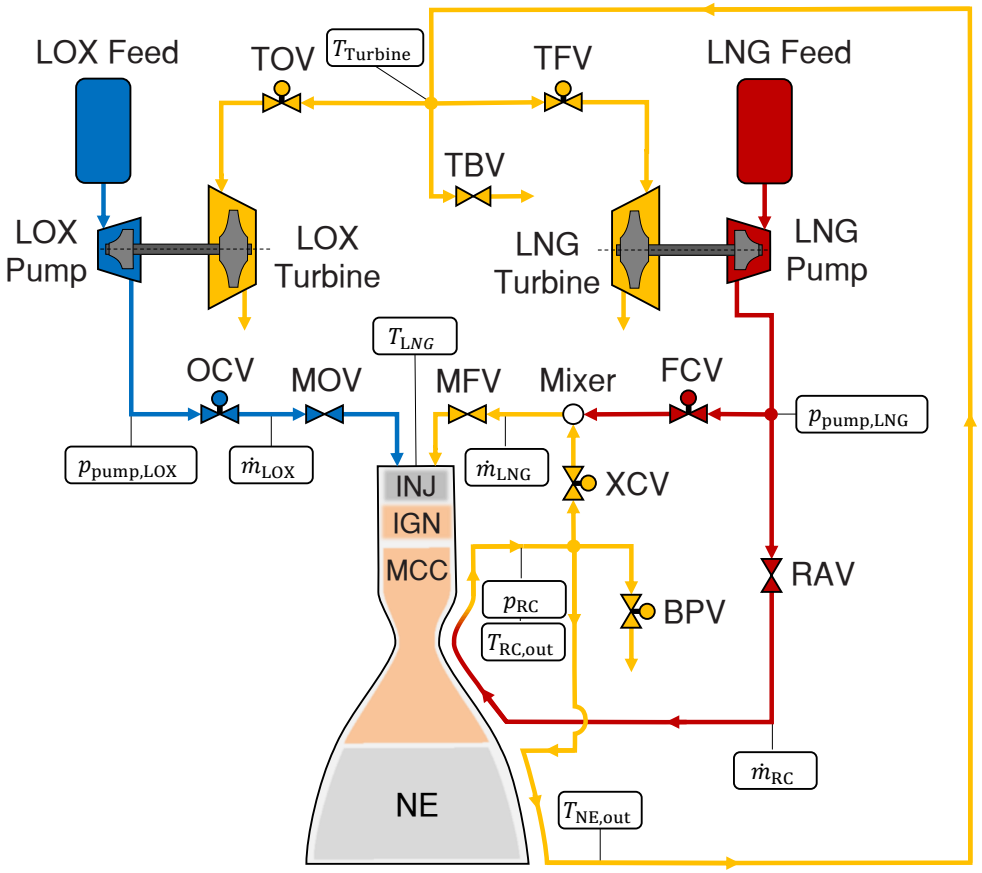


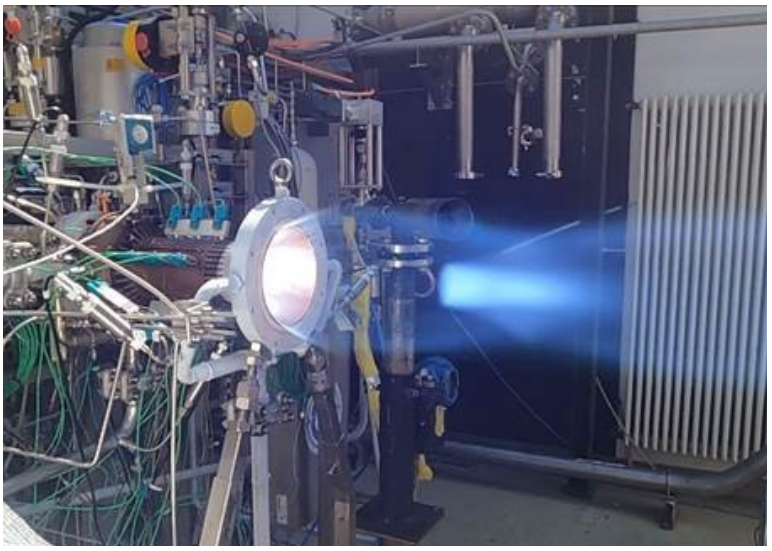
Figure 4.1.: LUMEN flow diagram. Modified with permission from [205]

Two different combustion chambers have been designed, one by conventional manufacturing (liner 1) and the other by selective laser melting (SLM) (liner 2). Liner 1 has an inner CuCrZr liner with axially milled cooling channels, which are closed by copper electro-plating, and an outer nickel shell [73]. Liner 2 is additively manufactured using SLM as a single, integral part [190, 200]. Figure 4.3a shows liner 2 at the test bench. The injector head has 42 shear-coax elements [26, 80]. The nozzle extension is regeneratively cooled and further heats up the methane which goes to the turbines.

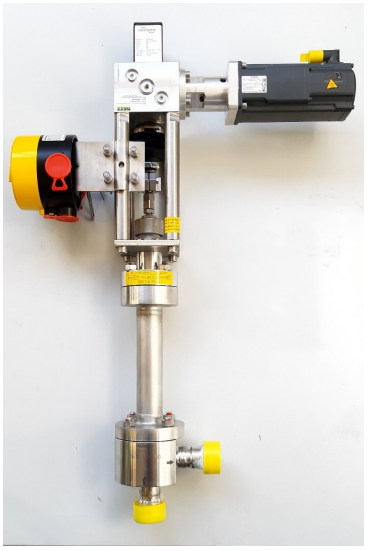
Figure 4.1 also contains the sensor positions that are used during this work for control. Static pressure is measured with pressure transducers. Thermocouples are used for temperature measurements. The injector mass flow rates $\dot{m}_{\mathrm{LOX}}$ and $\dot{m}_{\mathrm{LNG}}$ are measured with Coriolis flow meters and used to determine the mixture ratio ROF. For the cooling channel mass flow rate $\dot{m}_{\mathrm{RC}}$, a turbine flow meter is used. All in all, LUMEN is equipment with 40 thermocouples and 62 static and dynamic pressure sensors. [66]

Figure 4.2 shows LUMEN on the test bench during a hot-fire test. The turbopumps (with the fuel turbopump on the right side of the image) are separated from the main combustion chamber and connected by long pipes to increase flexibility. Unlike flight engines, LUMEN does not prioritize maximum efficiency or minimum weight but focuses on high flexibility [40, 200]. The choice of large distances between components increases the time delays that need to be considered when actively controlling the engine.

Figure 4.2.: LUMEN at test bench P8.3 during testing



(a) SLM combustor during testing



(b) Control valve [201]

Figure 4.3.: LUMEN components

Continuous flow control valves

LUMEN has six continuous, electrically actuated flow control valves and four additional on/off valves for sequencing: RAV is the regenerative cooling valve used for chill-down. MOV and MFV are the main oxidizer and fuel valves used for ignition and shutdown. TBV is an emergency safety valve for rapid reduction of turbine power. The control valves have high positioning accuracy, no overshoot, no pressure-dependent positioning and no hysteresis [201]. Their main function can be described qualitatively as follows:

- The turbine control valves (TFV, TOV) regulate the turbine inlet pressure and thus the turbine power. Opening both valves symmetrically increases the combustion chamber pressure, while opening TFV and TOV asymmetrically adjusts the mixture ratio [44].
- The oxidizer control valve (OCV) regulates the oxidizer mass flow with minimal delay due to its proximity to the injector head [44].
- The fuel control valve (FCV) redirects a portion of the LNG mass flow after the pump to the injector head, bypassing the regenerative cooling system. With FCV the cooling channel mass flow rate can be adjusted, similar to the Japanese LE-5B engine [44, 60].
- The mixer control valve (XCV) enables supercritical conditions in the cooling channels while operating the combustion chamber subcritically.
- The bypass control valve (BPV) allows methane to be vented after the chamber cooling, thus regulating the fuel injection temperature [44].

With 6 valves, important parameters can be controlled independently, such as combustion chamber pressure, mixture ratio, pressure and mass flow in the cooling channels, turbopump speeds, and fuel injection temperature [44].

4.1.1. Test campaign overview

Multiple component test campaigns have been conducted to qualify the design of each component and generate a database for modeling and validation:

The thrust chamber test campaign (TCA-23) was conducted to characterize the combustion chamber, regenerative cooling, nozzle extension, injector head, and control valves under fully representative conditions [191]. Figure 4.4 shows the achieved steady-state conditions in relation to the operational envelope of LUMEN (see Section 6.3.2). Different fuel injection temperatures between 210 K to 290 K were tested, and the minimum cooling channel mass flow rate for acceptable combustion chamber cooling was determined.

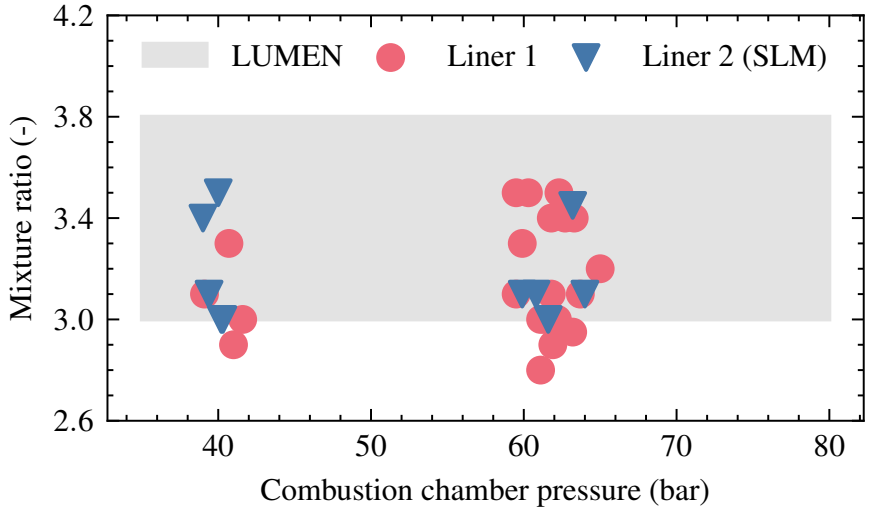


Figure 4.4.: LUMEN operational envelope and test data

The turbopump test campaigns (OTP-22 and FTP-23) were conducted to characterize the turbopumps. Gaseous nitrogen at ambient temperature was used to drive the turbines. The oxidizer turbopump (OTP) reached a maximum speed of 22 000 rpm, an outlet pressure of up to 70 bar, and mass flows of up to 5.7 $\mathrm{kg\,s^{-1}}$. The fuel turbopump (FTP) was tested up to 43 000 rpm, an outlet pressure of 110 bar, and mass flows of up to 3.5 $\mathrm{kg\,s^{-1}}$ [200].

In the demonstrator test campaign (DEMO-24), LUMEN was tested and put into service. After focusing on the ignition sequence, the operating envelope was characterized. Finally, the neural network controller developed in this thesis was successfully tested.

4.1.2. Operational envelope and constraints

The nominal operating point is at a combustion chamber pressure of 60 bar and a mixture ratio of 3.4. However, LUMEN can operate across a wide throttling range, between 35 bar to 80 bar, and mixture ratios between 3.0 and 3.8. The significant throttling range of 58 % to 133 %, as shown in Table 4.1, increases the challenges for the engine controller, which must handle changing system dynamics during throttling.

Table 4.1.: Operational envelope of LUMEN

	Minimum	Nominal	Maximum
Chamber pressure (bar)	35	60	80
Mixture ratio (-)	3.0	3.4	3.8
Throttling range (%)	58	100	133

All components are designed with large safety margins to ensure a long service life. However, several mechanical and thermodynamic constraints must be met during operation to prevent damage. The objective of the following section is to provide an overview of the operational challenges of all major components and the associated constraints. Most of this section has already been published in "Design and Control Challenges for the LUMEN LOX/LNG Expander-Bleed Rocket Engine" [44].

Regenerative cooling circuit

The regenerative cooling system is one of the most important components of an expander-bleed engine. Designing an efficient cooling system involves balancing sufficient cooling of the combustion chamber wall with a low pressure drop in the cooling channels. During operation, the combustion chamber pressure p_{cc} determines the minimum cooling channel mass flow $\dot{m}_{\mathrm{RC}} \geq f(p_{\mathrm{cc}})$ to keep the wall below its maximum temperature limit.

Using LNG as a coolant introduces additional challenges due to its high critical pressure of $p_{\mathrm{crit}} = 46\,\mathrm{bar}$. Operating the system below the critical point is difficult because of phenomena such as boiling, two-phase flow, and heat transfer deterioration [45, 133]. To avoid these challenges, LUMEN requires to maintains supercritical pressure in the cooling circuit.

In summary, the regenerative cooling system constraints are:

- Sufficient coolant mass flow rate $\dot{m}_{\mathrm{RC}} \geq f(p_{\mathrm{cc}})$
- Supercritical coolant pressure $p_{\mathrm{RC}} > 46\,\mathrm{bar}$

Injector head

Coaxial propellant injection is a well-established technique for cryogenic propellants [92]. It involves injecting the outer, faster gaseous fuel and the inner, high density oxidizer at different velocities, resulting in strong shear forces that create a highly turbulent shear flow. This turbulent flow enables flame anchoring at the injector faceplate, which is critical for combustion stability and efficiency.

Flame anchoring is a major concern for LOX/LNG [14, 26, 122, 146]. It can generally occur in three different ways: (1) a stable combustion zone at the injector exit plane, (2) a stable but lifted combustion zone downstream of the injector exit plane, and (3) an unstable and lifted combustion zone downstream of the injector exit plane [26]. The worst-case scenario for flame anchoring is flame blowout, which can occur during ignition, operation point changes, or unfavorable injection parameters [26]. The injection, atomization, vaporization, and mixing of propellants can be described by nondimensional parameters. The momentum flux ratio J is particularly important for the injection and anchoring processes:

$$J = \frac{\rho_{\mathrm{CH4}} \cdot v_{\mathrm{CH4}}^2}{\rho_{\mathrm{LOX}} \cdot v_{\mathrm{LOX}}^2} \tag{4.1}$$

Anchoring thresholds were determined experimentally for LUMEN, since no generalized thresholds are available, especially for LOX/LNG [26]. The tests confirmed the hypothesis that higher J numbers improve the anchoring of the combustion zone at the injector faceplate. A threshold of 7.5 to 10 was identified for successful anchoring.

In summary, the injection constraints are:

- Stable flame anchoring with $J > 10$
- Fuel injection temperature $T_{\mathrm{inj,CH4}}$ between 210 K to 300 K to minimize thermal stresses on the combustion chamber and injector.

Turbopumps

The operation of turbopumps is subject to numerous constraints that affect their performance and life expectancy. These constraints include rotordynamics, high cycle fatigue of turbine blades due to structural vibrations, turbine flutter, cavitation in the pump, and bearing temperature. Not all of these need to be addressed during operation, as turbopumps are designed to avoid them. However, some variables must be kept within predefined ranges during operation.

The rotational speed plays a crucial role in the operation of turbopumps. Resonance vibrations due to rotordynamics at the eigenfrequencies of the turbopump can lead to fatigue failure and excessive bearing loads. Critical rotor speeds have been determined numerically and should be avoided [202]. If a load point change crosses a critical speed the rotational speed should increase as fast as possible to minimize the time in the resonant regime.

The turbine inlet temperature effects the thermal stresses on the turbine blades. High temperatures cause a reduction of material properties and increase the damage rate.

The turbopumps constraints are as follows:

- Rotational speed OTP $n_{\mathrm{OTP}} < 28\,000\,\mathrm{rpm}$
- Rotational speed FTP $n_{\mathrm{FTP}} < 50\,000\,\mathrm{rpm}$
- No prolonged operation at critical speeds
- Turbine inlet temperature $T_{\mathrm{turbine}} < 700\,\mathrm{K}$

Valves

The electrically actuated flow control valves can change direction rapidly. Preventing valve oscillation is critical because of the potential risks involved. Uncontrolled oscillations can place excessive mechanical stress on the valves, leading to fatigue and possible failure. The resulting pressure oscillations can propagate through the system, potentially damaging other components such as pumps and turbines. Oscillations in the supply system can also cause chugging. Preventing unnecessary valve oscillations is therefore critical, and must be considered in the control design by penalizing valve usage.

4.2. Simulation model: Modeling and validation

The simulation model of LUMEN is built in EcosimPro 6.4.0 using the European Space Propulsion System Simulation (ESPSS) libraries version 3.6.0. EcosimPro is a modeling and simulation tool for continuous and discrete systems. The modeling is based on differential-algebraic equations. The ESPSS libraries [55], commissioned by ESA, are widely used in the European space industry. These libraries are suited for the simulation of liquid [37, 97, 216], solid [164], and electric [165] rocket engines. The simulation model is built by using predefined components from the ESPSS libraries, like pipes, pumps, or the combustion chamber. Each component has a set of physical parameters that can be adjusted to change its properties.

Overview

The modeling is structured as follows: First, all components are modeled based on the ESPSS libraries. Next, the steady-state and transient behavior of each component is compared to experimental data from various component test campaigns. All components are then combined into the complete LUMEN model, which is finally analyzed by applying step functions to the control valves and evaluating the system response in critical quantities such as the combustion chamber pressure and the mixture ratio.

During the project, component design and modeling were done in parallel. As a result, the modeling had to be refined after each component test campaign, which included some design changes. In total, three different simulation models with increasing levels of refinement were created, as shown in Table 4.2:

1. The first model (V1) is based on the initial component designs. It was used for a feasibility study of neural network-based control [48].
2. Based on the final component designs, a second model (V2) was created and is used in test case 1 (Chapter 5).
3. The final model (V3) was improved based on experimental data from the component tests. This model is used in test case 2 (Chapter 6) to train the neural network that was then tested on the test bench.

Having three models makes the comparison and validation more difficult, but it was unavoidable due to the tight project schedule and the ambitious goal of testing the neural network controller during the first integrated test campaign of LUMEN. Overall, all models are similar, but slightly differ in the parameters of the components.

Table 4.2.: Overview of different simulation models of LUMEN

Model	Based on	Usage	References
V1	initial component design	[48]	[12, 48, 52]
V2	final component design	test case 1	[5, 43, 44, 159]
V3	component test data	test case 2	–

Figure 4.5 shows the of LUMEN simulation model. The model includes the thrust chamber assembly (TCA), the fuel and oxidizer turbopumps (FTP, OTP), the fuel mixer, the bypass line with BPV, and the remaining valves and pipes. The TCA consists of the combustion chamber and two separate cooling circuits for the combustion chamber and the nozzle extension. The heat flow is split via thermal multiplexer elements and scaled according to the current operation point.

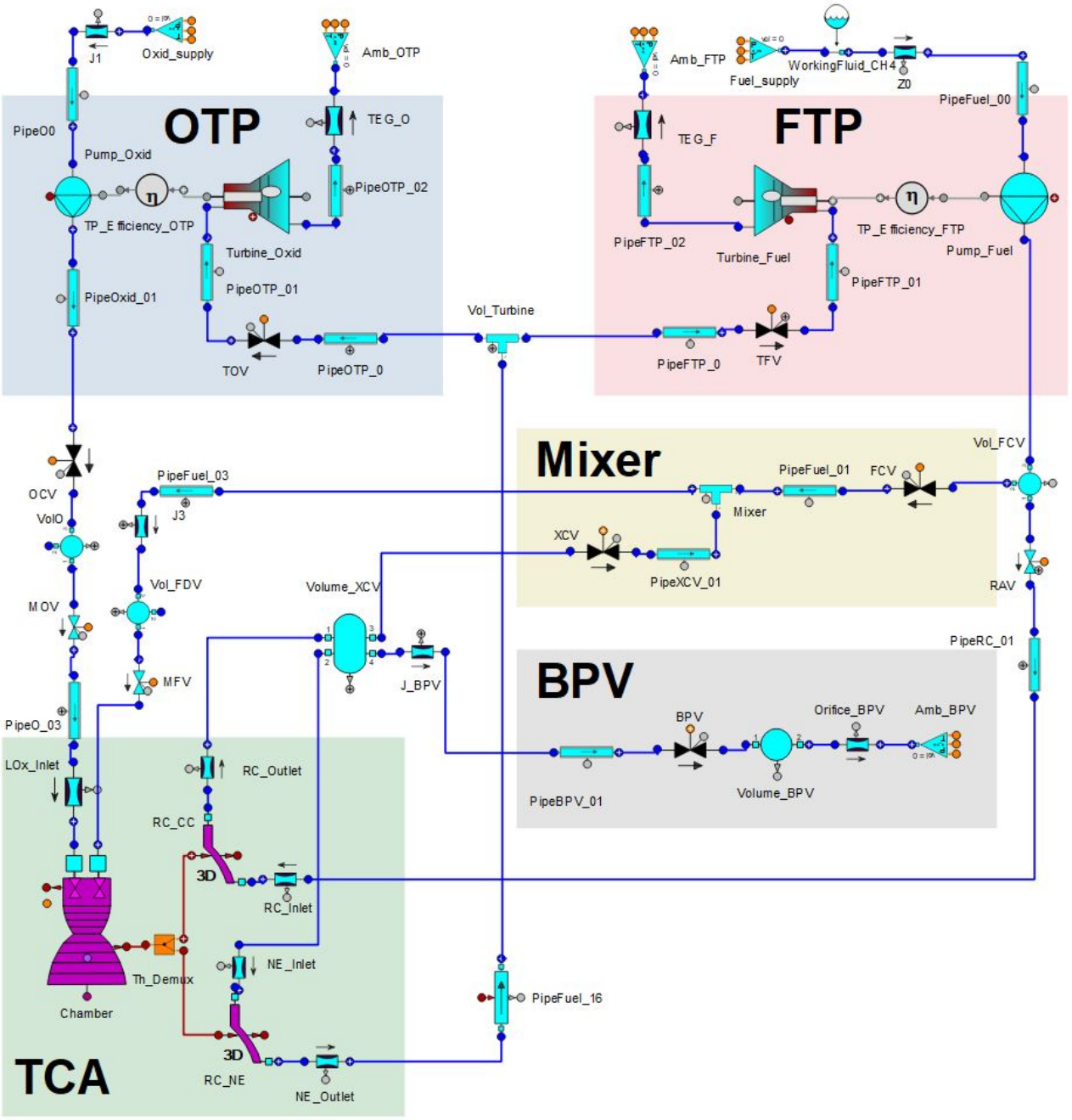


Figure 4.5.: Simplified LUMEN EcosimPro model

4.2.1. Thrust chamber assembly

The thrust chamber assembly (TCA) is modeled using the *CombustChamber-Nozzle* component from the ESPSS library. It consists of the injector head, the combustion chamber itself, and the nozzle extension. Key model parameters include the combustion chamber geometry, injection areas and pressure loss coefficients, combustion efficiency, and the heat transfer modeling, which is crucial for expander-cycle engines.

The standard Bartz equation is inaccurate in predicting the integral heat flux from the combustion chamber to the cooling system. To counteract this, empirical factors are introduced that scale the heat flux calculated from the Bartz equation (denoted as $\hat{\dot{q}}$) as a function of the engine operating point:

$$\text{Combustion Chamber:} \quad \dot{q}_{\mathrm{cc}} = k_1 k_2(T_{\mathrm{LNG}})\hat{\dot{q}}_{\mathrm{cc}} \tag{4.2}$$

$$\text{Nozzle Extension:} \quad \dot{q}_{\mathrm{ne}} = k_3(p_{\mathrm{cc}})\hat{\dot{q}}_{\mathrm{ne}} \tag{4.3}$$

The factor k_1 is constant. $k_2 = f(T_{\mathrm{LNG}})$ accounts for increasing heat fluxes associated with higher fuel injection temperatures T_{LNG}. The heat flux in the nozzle extension is scaled by a third empirical factor $k_3 = f(p_{\mathrm{cc}})$, which accounts for a reduced heat flux during flow separation while throttling. k_1, k_2 and k_3 are tuned based on test data of the TCA-23 campaign. Figure 4.6 shows the integral heat flux $\dot{Q}$ as a function of the combustion chamber pressure p_{cc}. It roughly follows the theoretical correlation of $\dot{Q} \propto p_{\mathrm{cc}}^{0.8}$.

The cooling channels are modeled using the *CoolingJacket* component, which assumes a one-dimensional channel flow with three-dimensional walls. The pressure drop Δp_{RC} is tuned by adjusting the wall roughness. Figure 4.6 shows Δp_{RC} as a function of the coolant mass flow $\dot{m}_{\mathrm{RC}}$.

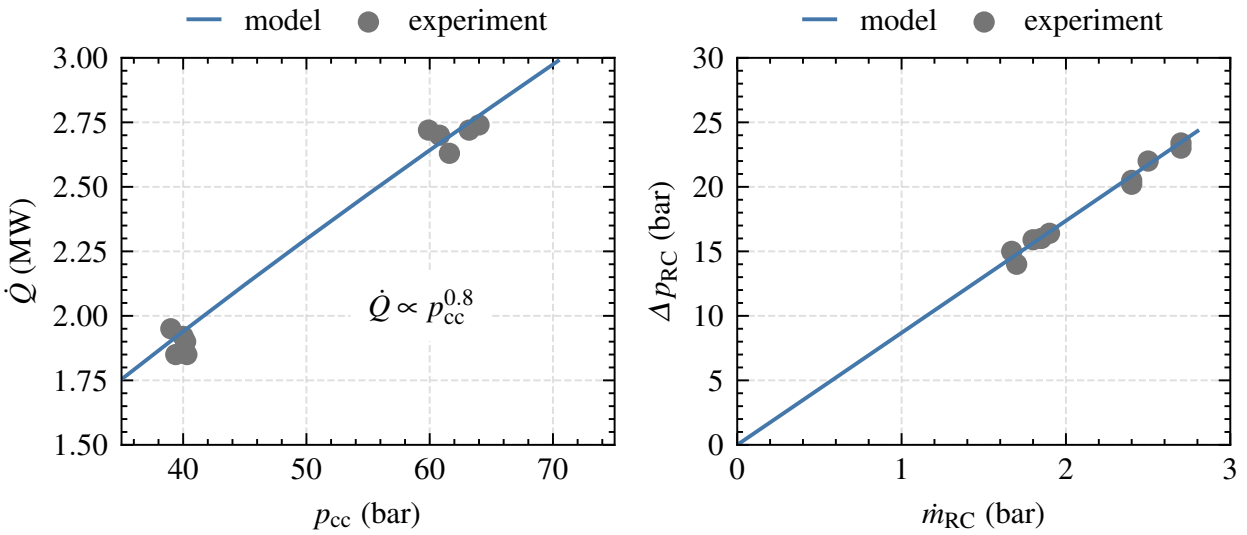


Figure 4.6.: Heat flux and pressure loss modeling

Comparison with experimental data

The TCA model is compared to test data from the TCA-23 campaign to validate the steady-state and transient behavior. The thermodynamic conditions at the test bench interface and all mass flow rates are set as boundary condition

in EcosimPro. The resulting combustion chamber pressure p_{cc}, fuel injection temperature T_{LNG}, and the temperature at the outlet of the regenerative cooling $T_{RC,out}$ and nozzle extension $T_{NE,out}$ are compared to test data.

Figure 4.7 shows a typical hot-run test sequence. First, the combustion chamber is ignited and ramped up to $p_{cc} = 60\,\text{bar}$. In this test, the influence of different fuel injection temperatures T_{LNG} is studied for a constant p_{cc} and ROF. First, T_{LNG} is set to 260 K and 290 K until steady state conditions are reached. Then T_{LNG} is gradually decreased to 210 K.

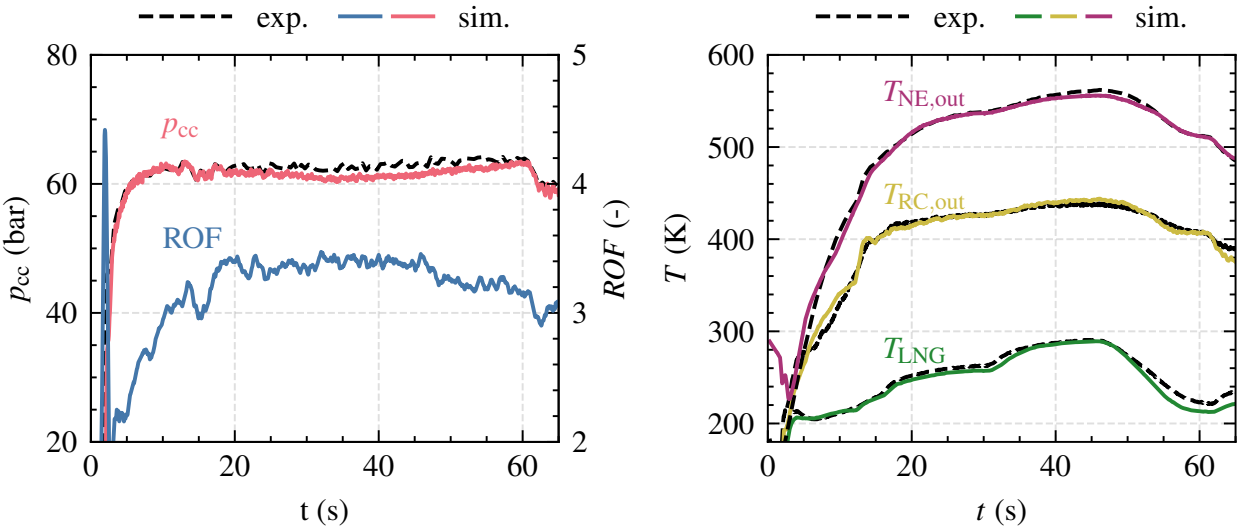


Figure 4.7.: Validation of thrust chamber model with experimental data. Mass flows are inputs for the simulation, therefore $ROF_{sim.} = ROF_{exp.}$

Several observations can be made by comparing simulation and experiment:

- p_{cc} is modeled with reasonable accuracy. This indicates that the combustion efficiency and chamber geometry is modeled accurately.
- T_{LNG}, $T_{RC,out}$ and the nozzle extension outlet temperature $T_{NE,out}$ are well predicted, showing that the heat transfer processes are modeled accurately, even for different T_{LNG} levels.
- Transient phases, such as ramping up the pressure and operating point changes, are also predicted with reasonable accuracy.

Table 4.3 summarizes the modeling accuracy over five different hot runs, totaling over 250 s of test duration. The steady error is calculated at several discrete operating points. The transient error is calculated across all data points after ignition ($t > 10s$). The parameters that are compared are: p_{LNG}, p_{LOX}: pressure in the fuel/oxidizer dome, T_{LNG} fuel injection temperature, dp_{RC} cooling channel pressure loss, $T_{RC,out}$ outlet temperature of the cooling channels, ΔT_{NE} coolant temperature increase over the nozzle extension.

Table 4.3.: Quantitative comparison between simulation and experiment

Component	Variable	Mean	Steady error (%)		Transient error (%)	
			Mean	Max	Mean	Max
Chamber	p_{cc}	51 bar	0.5	0.9	1.4	4.0
Injector	p_{LOX}	55 bar	0.4	1.1	1.1	3.0
	p_{LNG}	61 bar	0.9	2.2	1.4	3.7
	T_{LNG}	238 K	0.7	3.0	1.8	3.4
Cooling	dp_{RC}	19 bar	2.8	5.6	2.4	6.6
	$T_{RC,out}$	407 K	1.7	3.0	1.8	5.2
Nozzle	ΔT_{NE}	100 K	5.0	7.4	5.5	17.5

Overall, the modeling accuracy is high, with mean and maximum errors around 5 %. As expected, the modeling error increases during transients compared to steady-state points. The largest error is observed for the temperature rise in the nozzle extension, with a maximum error of 17.5 % during transients.

4.2.2. Turbopumps

The *Turbo_Machinery* library offers two modeling approaches for pumps and turbines. One possibility is to use *generic components* based on nominal design parameters. The second approach involves *custom components*, where performance maps are specified directly including the off-design behavior. This method offers more freedom in modeling but requires the availability of performance maps over the entire operating envelope.

The initial models were provided by Hahn [77] and are based on the expected nominal performance from pre-design tools and experimental data from water tests [203]. These models are then refined with data from the cryogenic test campaigns. One challenge encountered is that the shaft torque is not directly measured. Consequently, pump and turbine efficiencies can only be estimated indirectly through temperature and pressure measurements and by verifying the pump-turbine power balance. In detail, the following steps are performed:

1. Estimate the pump head curve based on experimental data.
2. Compare the produced turbine torque with the consumed pump torque for steady-state data points to check the power balance.

Pump modeling

Pumps are modeled using the custom pump component, which relies on user-defined tables for the head coefficient ψ^+ and reduced torque C^+ as a function of the mass flow coefficient ϕ^+. These coefficients are defined as follows:

$$\psi^+ = \frac{\Delta p}{\rho_{\text{in}}\omega^2} \qquad \phi^+ = \frac{\dot{m}}{\rho_{\text{in}}\omega} \qquad C^+ = \frac{T_{\text{pump}}}{\rho_{\text{in}}\omega^2} \tag{4.4}$$

Δp is the pressure rise of the pump, ρ_{in} the density of the fluid at pump inlet, ω the rotational speed, and $\dot{m}$ is the pump mass flow rate. C^+ defines the necessary shaft torque T_{pump} for a given ω and ρ_{in}.

The pump head curve $\psi^+ = f(\phi^+)$ is fitted to experimental data, as shown in Figure 4.8. With this curve, the pressure rise of the pump matches the experimental data with a maximum error of below 4 % (1.5 bar) for the fuel pump and below 5 % (2 bar) for the oxidizer pump.

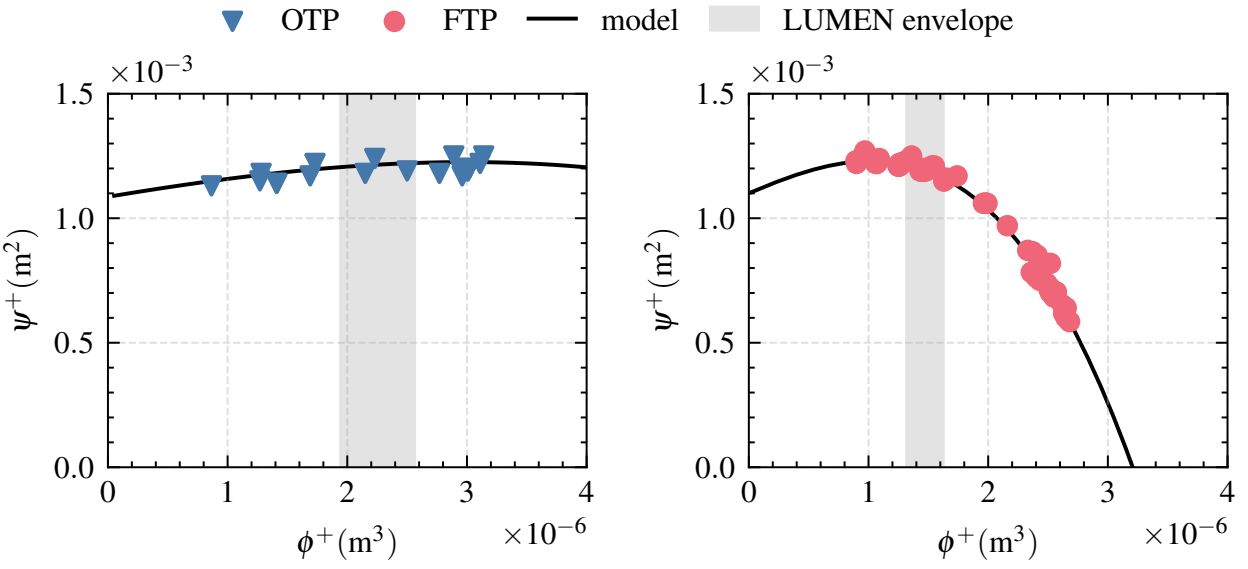


Figure 4.8.: Pump head curve modeling based on experimental data

Turbine modeling

Turbines are modeled with the custom turbine component, which relies on user-defined data tables for dimensionless performance maps. By using dimensionless numbers, these performance maps are valid across different fluids. The maps specify the specific torque T_s as a two-dimensional function of the dimensionless speed N and the pressure ratio Π:

$$T_s = \frac{T_{\text{turbine}}}{\dot{m} \cdot r \cdot c} = f(\Pi, N) \qquad N = \frac{\omega \cdot r}{c} \tag{4.5}$$

T_{turbine} is the shaft torque, c is the speed of sound at the turbine outlet, and r is the blade tip radius. $\dot{m}$ is the turbine mass flow, which can be calculated from the stator nozzle area and the inlet pressure for supersonic turbines. N is the ratio of the blade speed $\omega \cdot r$ with respect to c.

Turbopump power balance

The turbine and pump torque must be in equilibrium ($T_{\text{turbine}} = T_{\text{pump}}$) at steady state. Figure 4.9 compares the simulated values of T_{turbine} and T_{pump} for experimental data points. Positive errors indicate that the turbine produces too much torque for a given speed and vice versa. Overall, the error is less than $\pm 4\,\%$, indicating that the power balance is satisfied over the tested envelope.

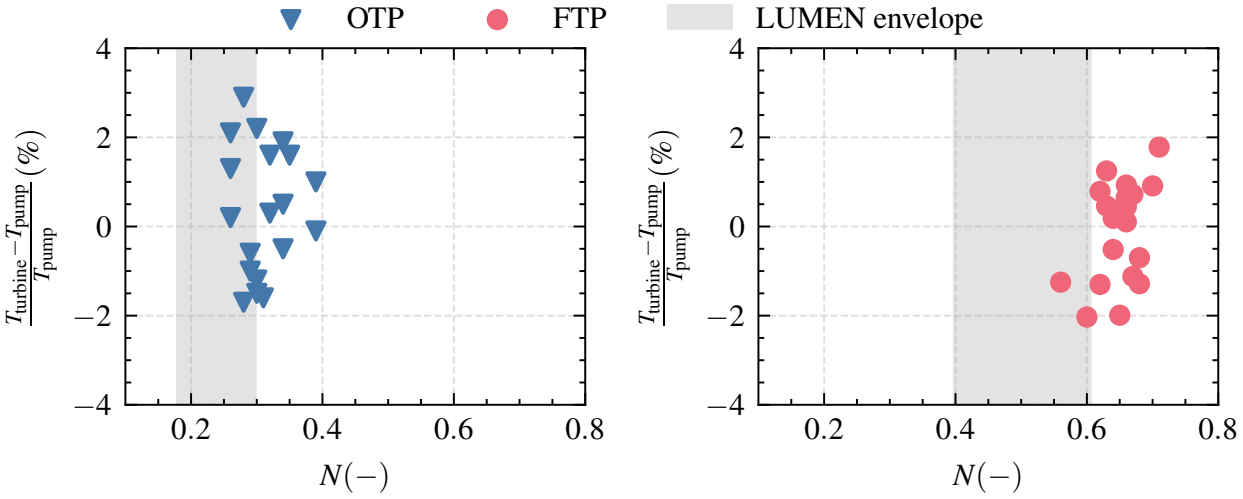


Figure 4.9.: Turbopump power balance

Uncertainties remain about how the turbines will perform with hot methane instead of ambient nitrogen used in the turbopump tests. Hot methane has a much higher speed of sound ($\approx 500\,\mathrm{m\,s^{-1}}$) compared to nitrogen ($\approx 350\,\mathrm{m\,s^{-1}}$). Since $N \propto c^{-1}$, the specific torque $T_s = f(\Pi, N)$ changes. Two new parameters η_{FTP} and η_{OTP} are therefore implemented to linearly scale the turbine torque. η_{FTP} and η_{OTP} can be used during domain randomization to change the turbopump efficiency.

4.2.3. Flow control valves

Valves are modeled with a new custom component based on the default valve of the *FluidFlow1D* library. The new model reuses the pressure drop formulation but introduces a new transfer function, which describes the time-dependency of the valve position with respect to the commanded signal.

Valve pressure drop formulation

The pressure drop across the valve depends on the volumetric flow rate, the thermodynamic conditions of the fluid, and the pressure drop coefficient of the valve K_v. K_v can be thought of as the flow capacity of the valve: the larger K_v, the more flow can pass through for a given pressure drop. The relationship between K_v and valve position depends on the physical shape of the internal valve geometry.

For linear valves, K_v changes proportionally with valve position, while equal-percentage valves open progressively more at larger positions, causing K_v to change exponentially with position. In EcosimPro, one option is to provide K_v as a function of valve opening. This curve is supplied by the valve manufacturer and fine-tuned using experimental data, ensuring accurate modeling of the valve's flow characteristics.

Table 4.4 compares simulated mass flow rates $m_\mathrm{sim.}$ to the experimental measurement $m_\mathrm{exp.}$ for different valve positions. The upstream pressure p_0 and temperature T_0, and the downstream pressure p_1 are taken from the experiment and set as boundary conditions.

Table 4.4.: Valve pressure drop validation

Valve	Position [%]	p_0 [bar]	p_1 [bar]	T_0 [K]	$m_\mathrm{exp.}$ [kg s^{-1}]	$m_\mathrm{sim.}$ [kg s^{-1}]	Error [%]
FCV	26	79.2	55.8	127	0.46	0.47	2
	45	114.4	87.8	124	1.05	1.06	1
	100	23.2	20.1	119	2.51	2.55	2
TFV	64	79.8	39.3	413	1.42	1.40	-1
	80	50.8	29.5	415	1.08	1.07	-1
	98	63.3	42.1	499	1.39	1.39	0
BPV	30	57.3	16.3	426	0.05	0.05	-6
	59	73.3	50.8	389	0.15	0.16	4
	81	82.0	77.4	389	0.23	0.24	4
	100	82.6	81.5	406	0.23	0.23	0
XCV	75	92.3	80.9	405	0.87	0.90	3
	75	72.2	58.9	389	0.75	0.75	0
OCV	57	99.5	48.1	101	3.60	3.65	1
	75	96.2	80.6	101	5.40	5.33	-1
	95	86.2	81.8	101	5.85	5.80	-1

The simulated mass flow matches the experimental measurements with small errors over a wide range of valve positions. This indicates that the K_v curve is well approximated. The temperature and pressure levels are representative for the future use in LUMEN: TFV, BPV, and XCV are exposed to hot methane with temperatures of up to 499 K, while FCV and OCV operate in cryogenic conditions. The higher relative error for BPV can be attributed to measurement uncertainties due to the low absolute mass flow.

Valve transfer function

The standard valve of the ESPSS library assumes a first-order transfer function between valve position and command signal. While this is a good approximation for hydraulic or pneumatic valves, it cannot accurately describe the dynamics of LUMEN's electromechanical valves. To address this, a new semi-empirical model was developed by Reitemeyer [159].

The new model has an inertia term that limits the maximum acceleration of the valve. Additionally, a control logic decelerates the valve before it reaches its target position. The model uses empirical parameters, such as maximum acceleration, maximum velocity, and a delay time, which are determined based on experimental data.

Figure 4.10 shows the step response of the valve position x_{valve}. After receiving the command, a small delay time, t_d is present until the valve starts moving. $v_{\max}$ is the maximum velocity. t_s is the total switching time. The default model line shows the first-order time response of a standard valve in the ESPSS library. The third line represent experimental data from OCV.

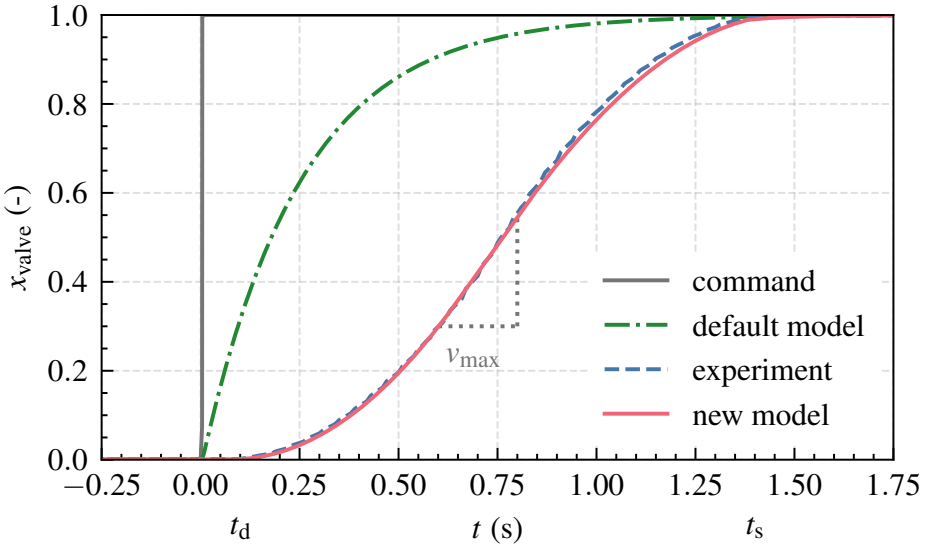


Figure 4.10.: Valve step response

Sensor delays

Real-world measurements are not instantaneously available due to sensor delays, which result from the measurement principles themselves, the measurement chain, and a moving average often applied to the raw data. These delays can pose significant problems for feedback control and must be included in the simulation model for reinforcement learning training.

Table 4.5 lists the sensors that will be used for closed-loop control of LUMEN at the test bench in Chapter 6. The table includes the sensor type, the sampling rate T_s of the data acquisition, and, if applicable, the moving average window T_a applied to the raw data. Depending on the measurement principles, different time constants (i.e. the response time of the sensor itself to step changes) also need be considered. Pressure transducers generally have low time constants, whereas thermocouples or turbine flowmeters react more slowly to changes due to their measurement principles. Unfortunately, the sensor manufacturers do not provide exact time constants for the installed sensors. Therefore, sensor delays are implemented in the EcosimPro model as a moving average with window lengths ΔT_s and tuned such that the model output matches the experimental measurements.

Table 4.5.: Sensor delays: ΔT_s is the total sensor delay, T_s the sampling rate, and T_a the moving window applied to the raw data

Sensor	Type	T_s[Hz]	T_a[s]	ΔT_s[s]
p_cc	pressure transducer	1000	0.1	0.1
T_LNG	thermocouple typ T	100	–	0.2
$\dot{m}_\mathrm{RC}$	turbine flowmeter	100	–	0.1
$\dot{m}_\mathrm{LOX}$	coriolis flowmeter	100	0.1	0.2
$\dot{m}_\mathrm{LNG}$	coriolis flowmeter	100	0.1	0.2

4.3. Simulation model: Analysis

Model analysis is the second step in designing a closed-loop controller after the model has been created and, if possible, validated on experimental data. Various techniques such as time domain analysis, state-space analysis, or frequency domain analysis, can be used to analyze the input-output behavior of the system. The goals of the analysis is to determine the stability or sensitivity [38, 104]. Further aspects are controllability and observability.

An important characteristic of a system is how the system responds to changes in its input. In general, the response of a dynamic system consists of a transient and a steady state component [139]. The transient response describes how the system transitions from the initial state to the steady state after being excited. The steady state is reached after all derivatives have decreased to zero.

The transient and steady-state response can be analyzed by applying typical test signals as input to the system. Such signals are usually step, ramp, impulse, or sinus functions [139]. The response of the system tells the control engineer important information about the system, such as the time required to reach a new steady state (settling time), the error while following an input signal, as well as the static gains between input and output [139].

The **static gain** K refers to the ratio of the change in an output Δy to a change in an input Δu after reaching steady state [59]. It is a measure of the sensitivity of the output of the system to changes in the input. For nonlinear systems, static gains may not be constant, but may change depending on the operating point. In this case, static gains can be approximated around an operating point [210].

Steady response of the LUMEN model

The simulation model is excited with individual step functions for each of the six valves. First, the engine is initialized with a combustion chamber pressure of 40 bar and a mixture ratio of 3.4. Then, in each simulation, a single valve is opened by 10 % ($u_{\text{step}} = u_{\text{steady}} + 0.1$), similar to the work by Raposo [155]. The step response is simulated until steady state conditions are reached and the static gain $\tilde{K}$ is computed. $\tilde{K}$ is defined as the static gain specifically for a 10 % step in u compared to the initial value before the step y_0:

$$\tilde{K}(y)|_{y=y_0} = \lim_{t\to\infty} y_{\text{step}}(t) - y_0 \text{ for } 10\,\% \text{ step in } u \tag{4.6}$$

Table 4.6 shows the static gains for various parameters, including p_{cc}, ROF, the pump mass flow ($\dot{m}_{\text{LNG}}$, $\dot{m}_{\text{LOX}}$), the cooling channel mass flow ($\dot{m}_{\text{RC}}$), the cooling channel outlet temperature ($T_{\text{RC,out}}$), and the fuel injection temperature (T_{LNG}). The table visualizes $\tilde{K}_u(y)$ in a matrix-like format, displaying the effect of each valve (column) on each output variable (rows).

TFV: Opening TFV increases the mass flow to the fuel turbine, boosting its power output. This higher turbine power accelerates the LNG turbopump, resulting in increased pump- and cooling channel mass flow $\dot{m}_{\text{LNG}}$, $\dot{m}_{\text{RC}}$. Consequently, p_{cc} increases, and ROF decreases. Due to the higher $\dot{m}_{\text{RC}}$, both $T_{\text{RC,out}}$ and T_{LNG} decrease.

Table 4.6.: Static gain $\tilde{K}_u(y)$ for 10 % step function

		$u \rightarrow$	TFV	TOV	FCV	BPV	OCV	XCV
$y \downarrow$	$y_0 \downarrow$	$u_0 \rightarrow$	0.30	0.21	0.24	0.73	0.80	0.80
p_{cc}	40	bar	3.7	2.7	-0.7	-3.6	1.7	-0.9
ROF	3.4	–	-0.6	0.9	-0.5	0.0	0.2	-0.1
$\dot{m}_{LNG}$	2.0	$kg\,s^{-1}$	0.6	0.0	0.0	-0.1	0.0	0.0
$\dot{m}_{LOX}$	3.8	$kg\,s^{-1}$	0.1	0.6	-0.3	-0.3	0.2	-0.1
$\dot{m}_{RC}$	1.6	$kg\,s^{-1}$	0.6	0.0	-0.1	0.0	0.0	0.0
$T_{RC,out}$	473	K	-109.8	12.3	-12.5	-45.0	12.8	1.1
T_{LNG}	280	K	-47.8	0.6	-53.3	-37.8	9.1	18.8

TOV: Opening TOV increases the mass flow to the oxidizer turbine, boosting its power output. The oxidizer turbopump spins up, which increases $\dot{m}_{LOX}$. As a result, opening TOV leads to an increase in ROF and p_{cc}. The influence of TOV on the fuel side is minimal.

FCV: Opening FCV increases the LNG mass flow to the fuel mixer, which directly reduces $\dot{m}_{RC}$ and ROF. The influence of FCV on T_{LNG} is significant, leading to substantially lower fuel injection temperatures T_{LNG}.

BPV: Opening BPV increases the mass flow that is vented after the cooling channels. The most prominent effect is a decrease in T_{LNG}. As energy is removed from the turbines, p_{cc} also decreases.

OCV: Opening OCV increases p_{cc} and ROF by reducing the pressure loss in the oxidizer feed line. The influence on the fuel side is minimal.

XCV: Opening XCV decreases the pressure level within the cooling channels and at the turbine inlet. Consequently, the turbine mass flow decreases, reducing the turbine power output, which in turn lowers p_{cc}. However, at an initial position of 0.8, opening XCV has only a marginal effect.

Time domain analysis of the LUMEN model

The next step is to analyze the time response when exciting the system with the same step responses as before. Figure 4.11 show the time response to the step function at $t = 0s$. The following observations can be made:

- The system is highly coupled. Opening a single valve affects several variables at once. For example, TFV changes the value of each output significantly.

- The response of p_{cc} to the step function of TFV is worth noting. There is a large overshoot of $\Delta p_{cc_{max}} = 6.5\,\text{bar}$, which is 75 % larger than the static gain $\tilde{K}_u(p_{cc}) = \Delta p_{cc_{static}} = 3.7\,\text{bar}$. $\dot{m}_{RC}$ shows a similar trend.
- Mostly the system directly responds in the direction of its final steady value. Only a small non-minimum phase behavior is observed for XCV.
- There are processes with different time scales. *ROF* and $\dot{m}_{RC}$ respond quickly to the step responses, reaching their steady-state values after a few seconds. However, T_{LNG} transitions much slower, taking over 20 s to reach steady state. This is due to the large thermal masses of LUMEN because of the long piping, heavy flow control valves, non-weight optimized combustion chamber and nozzle extension.

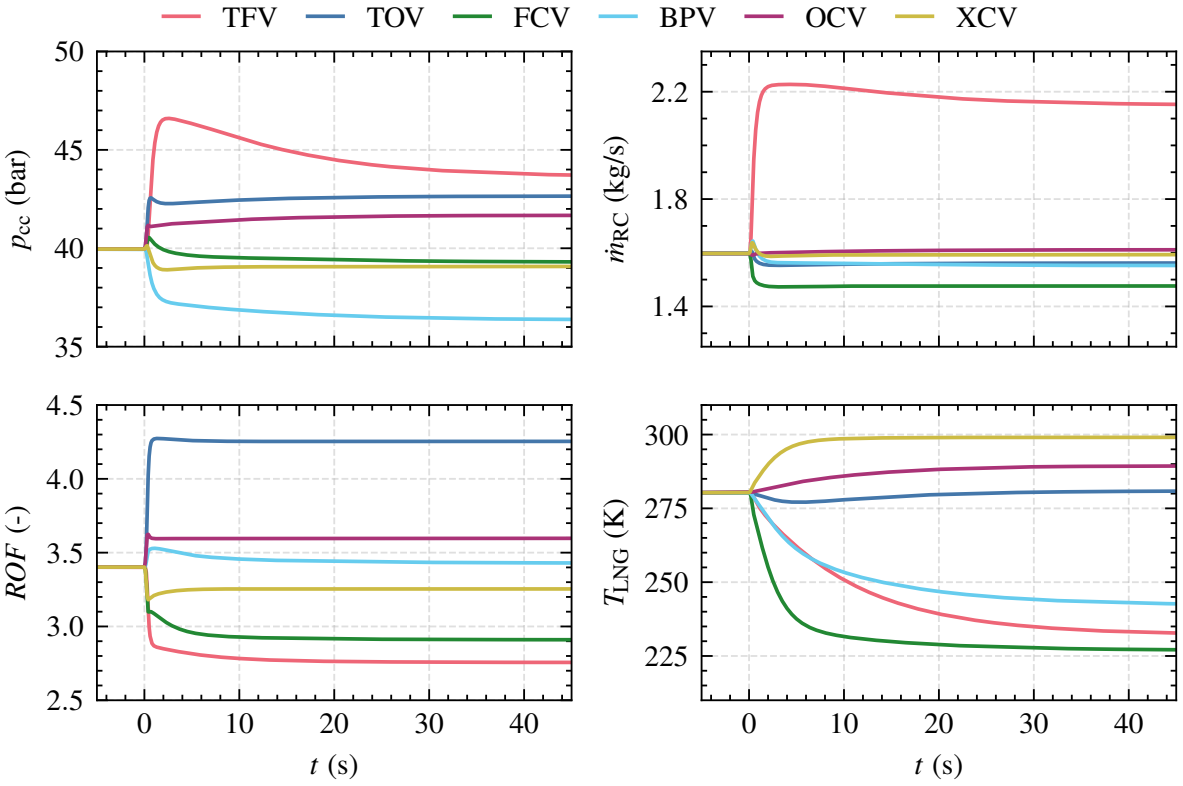


Figure 4.11.: System response to 10 % step function. In each simulation, the system is excited with a step function in a single control valve.

Settling time analysis

"The settling time t_s is the time required for the response curve to reach and stay within a range of certain percentage (usually 5 % or 2 %) of the final value." [195]. Table 4.7 contains the settling time for each input and output combination within a 2 % tolerance band for the 10 % step function.

Inputs on the oxidizer side (TOV and OCV) have mostly small settling times of less than 1 s. This is because th oxidizer side is not strongly coupled to the heat exchange processes in the cooling circuit. Inputs on the fuel side have larger settling times, up to 23.8 s. From a perspective of the output variables, *ROF* reacts quickly with a mean t_s of 2.5 s. Changes in T_{LNG} are much slower, with a mean t_s of 12.1 s. This is likely a result of the coupling between the heat transfer processes, the large thermal mass, and the turbopump power that relies on the enthalpy pickup in the cooling channels.

Table 4.7.: Settling time t_s in seconds for step function of 10 %

y \ u	TOV	TFV	FCV	BPV	OCV	XCV	Mean
$\dot{m}_{LNG}$	0.5	15.5	–	1.4	–	0.6	4.5
$\dot{m}_{LOX}$	0.5	19.5	3.4	7.4	0.3	0.4	5.3
$\dot{m}_{RC}$	0.5	14.9	0.5	1.2	–	0.6	3.5
p_{cc}	0.4	19.3	1.5	4.7	0.3	0.5	4.5
ROF	0.6	5.4	4.4	3.6	0.2	0.6	2.5
T_{LNG}	–	23.8	10.0	18.9	4.7	3.1	12.1
Mean	0.5	16.4	4.0	6.2	1.4	1.0	

4.4. Conclusion for control

This chapter has covered the LUMEN architecture, the modeling and validation of the EcosimPro model, and a system model analysis. At each step, relevant implications for the development of a closed-loop controller have been gained and are briefly summarized below:

Engine architecture and constraints: With up to six control valves, LUMEN is a flexible test platform for closed-loop control and offer more possibilities for regulation beyond standard thrust and mixture ratio control. The expander-bleed engine architecture presents operational challenges and constraints that must be addressed by a closed-loop controller. One of the challenges is that it can be beneficial to operate close to the limits of some constraints. For example, the highest efficiency in terms of specific impulse (I_{sp}) is achieved with the lowest possible cooling channel mass flow [44].

Modeling parameters: During modeling, several parameters of each component have been identified that influence the steady-state and transient dynamics. The controller must be robust to some degree of uncertainty in

these parameters. This approach is similar to the study by Perez-Roca [148], who randomized relevant engine parameters to evaluate the robustness of the controller. These uncertain model parameters are used in the following chapters for domain randomization. Table 4.8 summarizes the most important modeling parameters. In addition to the parameters of the main components, the thermodynamic conditions in terms of pressure and temperature at the interface are included.

Table 4.8.: Overview of LUMEN modeling parameters

Component	Parameter	Symbol
Valves	Pressure loss coefficient	K_v
	Maximum acceleration	a_max
	Maximum velocity	v_max
	Delay time	T_d
Thrust chamber	Heat transfer modeling	k_1, k_2, k_3
	Wall roughness	$r_\mathrm{RC}, r_\mathrm{NE}$
	Injector pressure loss coefficients	ζ_oxy, ζ_red
	Combustion efficiency	η_{c^*}
	Nozzle throat area	A_t
Turbopumps	Efficiency factor	η_FTP, η_OTP
	Moment of inertia	I_FTP, I_OTP
Pipes	Wall roughness	r_i
Interfaces	Interface pressure	p_LOX, p_LNG
	Interface temperature	T_LOX, T_LNG
Sensors	Sensor delay	ΔT_s

Modeling errors

The model is built in EcosimPro using the ESPSS libraries. By relying on well-validated physics-based models that have proven their accuracy in numerous applications, a high level of precision is achieved. Only the transient dynamics of the flow control valves needed to be modeled with a newly developed custom model. For validation, the model characteristics have been compared with experimental test data from component test campaigns. Overall, the steady-state errors are around 5 %. The transient characteristics are also modeled accurately but, as expected, with slightly higher deviations.

However, modeling errors are inevitable when using reduced order models, such as assuming 0D or 1D flow, as most EcosimPro components do. In addition, some components rely on empirical factors that further increase

uncertainties when the system is operated outside the range of experimental data. This is particularly challenging for liquid rocket engines because testing is extremely expensive and data is limited.

Component testing is also not always fully representative. For LUMEN, the turbines were tested with gaseous nitrogen instead of methane during the turbopump test campaigns. This introduces additional modeling uncertainties. Moreover, the coupling between the turbopumps, the combustion chamber, and the test bench itself can introduce unanticipated effects that are not present in component testing. Finally, the flow control valves may respond differently when actuated by a dedicated controller at high control frequency.

The following is a non-exhaustive list of potential sources for modeling error that may be present in the final model and should be considered during reinforcement learning training:

- Changing interface pressure during operating point changes due to the pressure regulation of the test bench itself.
- Operating the turbines with hot methane instead of nitrogen.
- Changing pressure losses, efficiencies, and heat transfer coefficients due to wear after each test run.
- Uncertain valve dynamics when actuated with high control frequency.
- Additional sensor delays due to the measurement principle itself, the data acquisition or moving average applied to smooth the signal

Model analysis

The model was excited with a step function of 10 % in each input. The system's response revealed important properties that needs to be considered for the controller design: The system is coupled, meaning that each flow control valve influences multiple system variables. This coupling makes designing an appropriate controller more challenging, as stated in Section 2.1. Some overshoots are also present. Additionally, different physical effects with varying time scales are present. Changing mass flows occurs quickly. In contrast, heat transfer processes are much slower due to the thermal mass of the system and the coupling between combustion and turbopumps.

After training the neural network, it will be important to analyze if it can handle these challenges. The controller will need to operate multiple valves simultaneously to change a single output while keeping other quantities constant. It will also have to manage different time scales and potential overshoots.

5. Test case 1: Fuel-efficient rocket engine control in simulation

The first test case covers the fuel-efficient control of combustion chamber pressure (p_{cc}) and mixture ratio (ROF). The primary control objective is to closely follow a predefined profile of p_{cc} and ROF. This is a typical scenario for a launch from Earth into orbit, where the required thrust profile (equivalent to p_{cc}) is known in advance. The secondary objective is to minimize fuel consumption by operating the engine at its peak efficiency. Third, the controller must respect various thermodynamic and mechanical constraints.

The goal of this chapter is to demonstrate that reinforcement learning can be used to train a neural network for a representative control task. It is shown how to define the reinforcement learning setup in terms of the reward function, the action, and the observation space. The training process is discussed and the neural network is evaluated in terms of tracking accuracy, fuel efficiency, and robustness to model parameter uncertainties and sensor noise. A study of the control frequency and the observation space is performed.

Another important aspect is the comparison of the neural network with a model predictive control (MPC) approach, which was developed in cooperation with the Institute for Automatic Control of the RWTH Aachen University [205]. MPC is the current standard for controlling dynamic systems in engineering applications and is therefore a suitable way of benchmarking the performance of the neural network.

5.1. Control objectives

Control objectives define the functional requirements for the controller. In this test case, the controller must steer the engine to 5 different, predefined setpoints without overshoot and maintain these setpoints with minimal steady-state error. Transitions between setpoints should be time-optimal. The controller should also prevent unnecessary oscillations and respect various thermodynamic and mechanical constraints.

The secondary objective is maximizing the fuel-efficiency for a given combination of p_{cc} and ROF. For flight engines, the combination of p_{cc} and ROF

usually defines the operating point unambiguously. However, since LUMEN has more flexibility by using six control valves, different operating points are possible for the same combination of p_{cc} and ROF. For example, increasing the speed of the LOX turbopump creates an additional pressure rise that can be counteracted by closing the OCV valve, resulting in the same p_{cc}. However, this approach wastes energy and deviates from the most fuel-efficient operation.

In summary the functional requirements are as follows:

1. Performance: demonstrate the time-optimal tracking of a predefined reference trajectory for p_{cc} and ROF.
2. Economy: operate the engine at peak efficiency. For this test case, minimizing the turbine mass flow rate $\dot{m}_{\text{turbines}}$ is used as a measure of the engines efficiency.
3. Safety: demonstrate constraint handling and adhere to operational limits of various components.
4. Stability: demonstrate that oscillations are prevented or damped.

Reference trajectory

Figure 5.1 shows the reference trajectory for p_{cc} and ROF. At $t = 10\,\text{s}$ closed-loop control is activated with the first setpoint of $p_{cc} = 60\,\text{bar}$ and $ROF = 3.0$. Then, ROF shall be increased to 3.4 and 3.8, each held for 10 s. At $t = 40\,\text{s}$, p_{cc} shall be throttled down to 50 bar, and ROF shall be reduced to 3.4. At $t = 50\,\text{s}$ the engine shall be throttled up, increasing p_{cc} to 80 bar, while keeping ROF constant at 3.4. All operating point changes shall be made as quickly as possible without intermediate ramps.

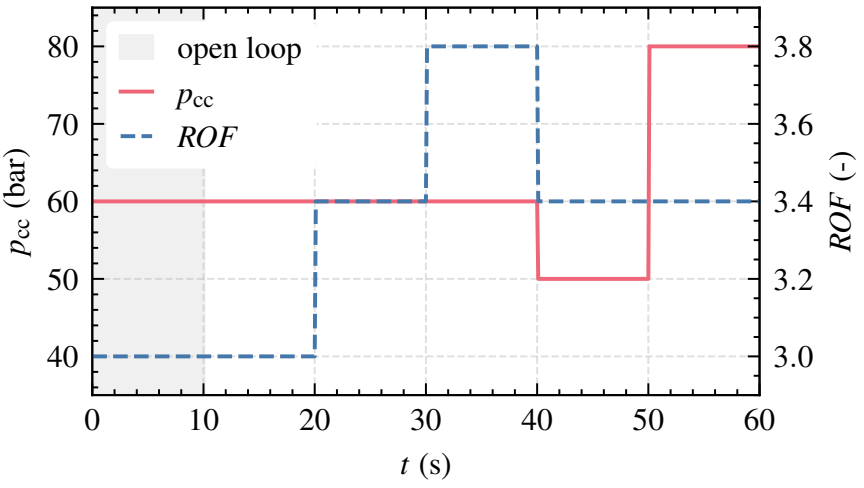


Figure 5.1.: Reference trajectory for test case 1

Constraints

Table 5.1 contains the operational constraints from Section 4.1.2 that are considered in this test case. For each variable, a minimum and/or maximum value is defined that the controller must respect. As shown in the simulation model analysis in Section 4.3, the constrained variables have different time constants: The mixture ratio is the most time-sensitive constraint, which can quickly deviate beyond its critical value, potentially causing fatal damage to the combustion chamber. On the other hand, the turbine inlet temperature, the momentum flux ratio, and the speeds of the turbopumps have larger time constants. To handle these constraints, the controller must anticipate changes over an extended prediction horizon.

Table 5.1.: Constraints for test case 1

Variable		Unit	Min.	Max.
Mixture ratio	ROF	–	2.5	4
Momentum flux ratio	J	–	10	–
Turbine inlet temp.	T_{turbine}	K	–	700
Rotational speed OTP	n_{OTP}	1/min	–	28 000
Rotational speed FTP	n_{FTP}	1/min	–	50 000
Coolant mass flow rate	$\dot{m}_{\text{RC}}$	kg/s	$f(p_{\text{cc}})$	–
Coolant outlet pressure	p_{RC}	bar	46	–

In summary, the vector of the constrained variables is defined as:

$$C = [ROF, J, T_{\text{turbine}}, n_{\text{OTP}}, n_{\text{FTP}}, \dot{m}_{\text{RC}}, p_{\text{RC}}]^T \tag{5.1}$$

5.2. Controller synthesis

The next step is controller synthesis. It includes the definition of the reinforcement learning framework, the sampling time, the input-output vector, and the training of the control policy. First, all control objectives are formulated within a Markov Decision Process by defining an appropriate action space, observation space, and reward function. The action space defines the output vector of the control policy, while the observation space contains all necessary information about the current system state. The reward function translates the multidimensional control objectives into an appropriate scalar reward signal. The control interval is initially set to $\Delta t = 0.1\,\text{s}$, but alternative values are explored later.

Action space

The action space A defines the set of possible actions or outputs of the controller. At each control interval t, the agent samples an action $a_t \in A$. For this test case, the action space is given by continuous commands u to 5 control valves (TOV, TFV, FCV, XCV, and OCV). BPV is not used. The bounds of the action space are determined based on initial system studies.

$$A = \begin{bmatrix} \mathrm{u_{TFV}} \\ \mathrm{u_{TOV}} \\ \mathrm{u_{FCV}} \\ \mathrm{u_{XCV}} \\ \mathrm{u_{OCV}} \end{bmatrix} \in \begin{bmatrix} [0.1, 0.6] \\ [0.1, 0.4] \\ [0.3, 0.8] \\ [0.7, 1.0] \\ [0.7, 1.0] \end{bmatrix} \tag{5.2}$$

Note that stochastic policies like Soft Actor-Critic (SAC) typically operate internally with an action range of $[-1, 1]$. This normalization of the action range improves training stability and helps to explore the action space more efficiently. Therefore, a linear scaling function is applied to convert the action space from the physically meaningful valve positions to the $[-1, 1]$ range.

Observation space

The observation space contains the controlled variables $Y \in \mathbb{R}^2$, a subset of the constrained variables $\bar{C} \in \mathbb{R}^5$, the turbine mass flow $\dot{m}_{\mathrm{turbines}}$, and the positions $X \in \mathbb{R}^5$ and command signals $U \in \mathbb{R}^5$ to the control valves.

$$Y = \begin{bmatrix} p_{\mathrm{cc}} \\ ROF \end{bmatrix}, \; \bar{C} = \begin{bmatrix} \dot{m}_{\mathrm{RC}} \\ \dot{m}_{\mathrm{RC,min}} \\ p_{\mathrm{RC}} \\ \mathrm{J} \\ T_{\mathrm{turbine}} \end{bmatrix}, \; Y_{\mathrm{valves}} = \begin{bmatrix} \mathrm{x_{TFV}} \\ \mathrm{x_{TOV}} \\ \mathrm{x_{FCV}} \\ \mathrm{x_{XCV}} \\ \mathrm{x_{OCV}} \end{bmatrix}, \; U = \begin{bmatrix} \mathrm{u_{TFV}} \\ \mathrm{u_{TOV}} \\ \mathrm{u_{FCV}} \\ \mathrm{u_{XCV}} \\ \mathrm{u_{OCV}} \end{bmatrix} \tag{5.3}$$

The observation space also contains reference values Y_{ref} for the controlled variables for the current and next N_f time steps to improve the transient response. $\oplus$ is the direct sum of the individual vectors, which concatenates them into a single extended vector of dimension $2 \cdot (N_f + 1)$. The influence of different values of N_f is studied in Section 5.3.3.

$$Y_{\mathrm{ref}} = \begin{bmatrix} p_{\mathrm{cc,ref}}(t) \\ p_{\mathrm{cc,ref}}(t + \Delta t) \\ \dots \\ p_{\mathrm{cc,ref}}(t + N_f \Delta t) \end{bmatrix} \oplus \begin{bmatrix} ROF_{\mathrm{ref}}(t) \\ ROF_{\mathrm{ref}}(t + \Delta t) \\ \dots \\ ROF_{\mathrm{ref}}(t + N_f \Delta t) \end{bmatrix} \tag{5.4}$$

In summary, the observation vector of each time step is defined as

$$\tilde{o}_t = Y \oplus \bar{C} \oplus [\dot{m}_{\text{turbines}}] \oplus Y_{\text{valves}} \oplus U \oplus Y_{\text{ref}} \in \mathbb{R}^{18+2\cdot(N_f+1)} \tag{5.5}$$

An important feature is observation stacking, as defined in Equation 3.19. Instead of proving the agent only with the observation vector of the current time step, a history of previous time steps is appended to form the full observation vector. This allows the agent to infer information about the time-dependent behavior of the system, such as how fast a variable is changing. Observation stacking also improves robustness to model perturbations.

The full observation vector o_t has a dimension of $(N_p + 1) \cdot (18 + 2 \cdot (N_f + 1))$, where N_p is the number of past stacked measurements. For a typical setting of $N_f = 4$ and $N_p = 3$, the observation vector has a length of 84. All elements are normalized by their default values to improve training [162].

Reward function

The reward function R provides feedback to the learning algorithm during training. It calculates a scalar reward at each control interval to evaluate the current state of the system. The shape of the reward function plays a crucial role in successfully training controllers with reinforcement learning [199]. A reward function with shallow slopes over a wide range aids exploring and guides the agent towards the optimal policy. Conversely, a tighter and more sharper reward shape encourages accuracy but may hinder exploration.

For this test case, a reward function with three parts, each representing a specific control objectives is defined as:

$$R(t) = \sum_{y \in [p_{\text{cc}}, ROF]} r_{\text{tracking,y}}(t) + \sum_{c \in C} r_{\text{penalty,c}}(t) + r_{\text{economic}}(t) \tag{5.6}$$

The first term evaluates tracking performance. Tracking rewards for each controlled variable

$$r_{\text{tracking,y}}(t) = \left(e^{-\delta \frac{|y(t) - y_{\text{ref}}(t)|}{y_{\text{ref}}(t)}} - 1 \right) \text{ for } y \in [p_{\text{cc}}, ROF] \tag{5.7}$$

are calculated using an exponential function of the absolute percentage tracking error, similar to Tracey et al. [199]. When the error is zero ($y(t) = y_{\text{ref}}(t)$), the tracking reward is maximum with a value of 0. As the tracking error increases, the tracking reward decreases. For significant errors, the tracking reward converges to -1. This formulation bounds the maximum possible cumulative

tracking reward that an agent can earn over an episode. This is useful for monitoring the training process as the best and worst possible performance of the agent over one episode is known.

Figure 5.2 shows the tracking reward for different values of δ, where δ scales the slope of the exponential function. In general, the function has a steeper slope near zero, which guides the learning algorithm towards an optimal control policy with minimal tracking error. For large errors, the slope of the reward function is smaller, but the agent is still positively incentivised to reduce large errors. This helps to provide early feedback in the training process.

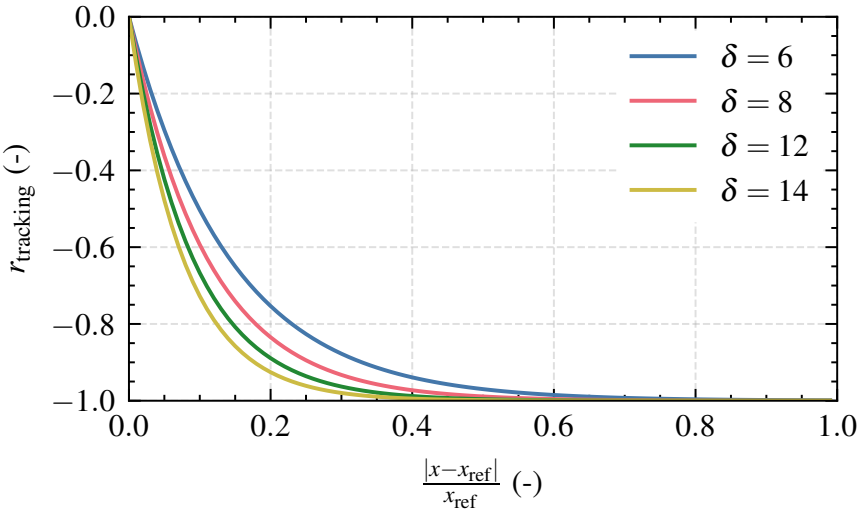


Figure 5.2.: Exponential reward function for tracking

The second term in Equation 5.6 penalizes constraint violations. When constraints are violated, the agent receives a constant penalty

$$r_{\text{penalty,c}}(t) = \begin{cases} -\beta(c), & \text{if } c(t) < c_{\min}(t) \text{ for } c \in C \\ -\beta(c), & \text{if } c(t) > c_{\max}(t) \text{ for } c \in C \\ 0, & \text{otherwise} \end{cases} \tag{5.8}$$

for each constraint violation, regardless of the extend of the violation. The penalty size $\beta(c)$ can be set individually for each constrained variable, thus determining the severity of violating a particular constraint. In addition, an extra penalty is applied if the EcosimPro simulation crashes due to unreasonable inputs of the agent.

The third term is the economic term. For an open cycle architecture such as LUMEN, maximizing the efficiency of the engine is equivalent to minimizing the turbine mass flow for a given combustion chamber pressure and mixture ratio [44]. Therefore, the economic term is defined by the negative sum of the

mass flow rates of both turbines, weighted by a factor γ:

$$r_{\text{economic}}(t) = -\gamma \dot{m}_{\text{turbines}}(t) = -\gamma(\dot{m}_{\text{OTP,turbine}}(t) + \dot{m}_{\text{FTP,turbine}}(t)) \quad (5.9)$$

The challenge in reinforcement learning is to combine all control objectives into a scalar reward function. For example, minimizing $\dot{m}_{\text{turbines}}$ could be achieved by entirely closing TFV and TOV, thus reducing $\dot{m}_{\text{turbines}}$ to zero. However, this would distinguish the engine and therefore lead to large tracking errors and penalty violations. Similar considerations apply to penalty weights; if chosen too small, the agent might briefly violate constraints to speed up setpoint changes. If chosen too large, the agent might maintain a safety margin from the constraint limits, compromising tracking performance or efficiency. By running various training runs with different values for β, γ, and δ, a working set of weights is found and summarized in Table A.2.

Reinforcement learning setup

The general reinforcement learning setup has been described in Section 3.5. The SAC algorithm is used and training is performed on a workstation with an Intel Xeon W-2265 processor and an NVIDIA GeForce RTX 3090 GPU. With this setup, data generation can be done in parallel with 10 simulation models, resulting in a training time of about one million training steps per day. The V2 simulation model of LUMEN is used for this test case (see Table 4.2). It is based on the final component design and uses the same schematics as the latest V3 model. However, it is not fine-tuned with results from component test campaigns and therefore uses slightly different component parameters, such as turbine efficiencies, or valve pressure loss coefficients. It also does not incorporate sensor delays.

The training is structured as follows: Each episode of 50 s, i.e. a simulation of the reference trajectory, consists of 500 simulation steps. It is only reset prematurely if the simulation crashes. During training, the agent explores the environment, collects data, and refines the control policy by performing gradient ascent updates of the weights of the actor and critic networks to maximize the expected (discounted) cumulative reward. The entire training is further structured into distinct iterations of 40 000 steps. At the end of each iteration, the current policy is evaluated and the neural network weights are stored in a checkpoint. After training, the checkpoint with the best performance can be loaded for deployment and further evaluation. In this test case, domain randomization is only considered in Section 5.4.2 and is not applied elsewhere.

Training results

Figure 5.3 shows the training progress in terms of the undiscounted cumulative episode reward, which is evaluated every 40 000 training steps with the currently best policy. Initially, the agent randomly selects actions to explore the environment, resulting in a very low episode reward. As the agent gains knowledge of the system dynamics, its performance improves, leading to an increase in performance. After about 1.5 million training steps, or 3000 training episodes, the training process converges.

The right plot in Figure 5.3 shows the integrated turbine mass flow m_{turbines} over an episode. m_{turbines} should be minimized to satisfy the economic control objective. The agent quickly reduces m_{turbines} and converges to approximately 55 kg after 1.3 million training steps.

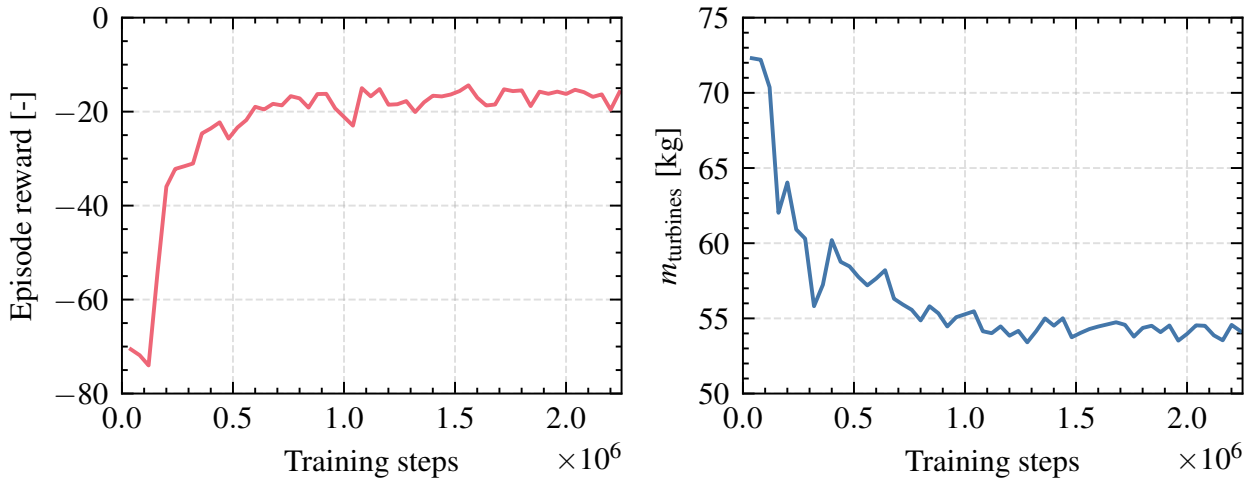


Figure 5.3.: Training progress

Study of different reward shapes

Tracey et al. [199] investigated different reward shapes for the complex task of controlling magnetic fields nuclear fusion reactors. The authors suggest initiating training with a broader reward function and then performing a second training phase with a sharper reward. For this test case, the reward shape can be modified with δ from Equation 5.7. Three different configurations are compared in terms of control performance and necessary training time:

1. Only train with a wide reward function with $\delta = 8$.
2. Only train with a sharp, exciting reward function with $\delta = 12$.
3. Start training with the wider reward function and then perform a second phase of training with a sharper reward that encourages accuracy.

Table 5.2.: Training results for different reward shapes. For evaluation, the return is calculated with $\delta = 12$ for all runs.

Run	Return	Δp_{cc} (%)	ΔROF (%)	m_{bleed} (kg)
1	-15.0	0.34	0.18	54.3
2	-14.8	0.35	0.13	54.2
3	-15.1	0.35	0.18	54.4

The training results of all three configurations are summarized in Table 5.2. All training runs converged after approximately the same number of training steps with no visible influence of the reward shape. All configurations yield comparable results in terms of return, mean tracking error in p_{cc} and ROF, and fuel consumption m_{turbines}. The training algorithm has no problems exploring the environment and improving its policy even for the sharp reward function with $\delta = 12$. This is probably due to the continuous nature of the underlying control problem, i.e. the monotone reward signal always points the agent towards the optimum. Since it yields marginal higher overall accuracy, $\delta = 12$ is selected for further training.

5.3. Controller analysis

The next step is to analyze if the controller trained with the default set of hyperparameters defined in Table A.2 meets the given functional requirements. First, the tracking performance is examined to determine if the controller is able to successfully follow the given reference trajectory of p_{cc} and ROF. Next, the constraint handling and fuel efficiency are investigated. Finally, parameter studies are performed for the control frequency and the number of future references.

5.3.1. Tracking performance

Figure 5.4 shows the tracking performance in p_{cc} and ROF and the corresponding valve positions. The controller follows the reference trajectory with negligible overshoot and high accuracy with an error of only 0.35 % in p_{cc} and 0.18 % in ROF. All setpoint changes are performed with short settling times and no oscillations. The controller is able to change p_{cc} or ROF independently without causing errors in the other quantity. This is remarkable considering the coupling between p_{cc} and ROF. Examples are ROF changes at $t = 20\,\mathrm{s}$ and $t = 30\,\mathrm{s}$, which are performed without deviations in p_{cc}. Similarly, the

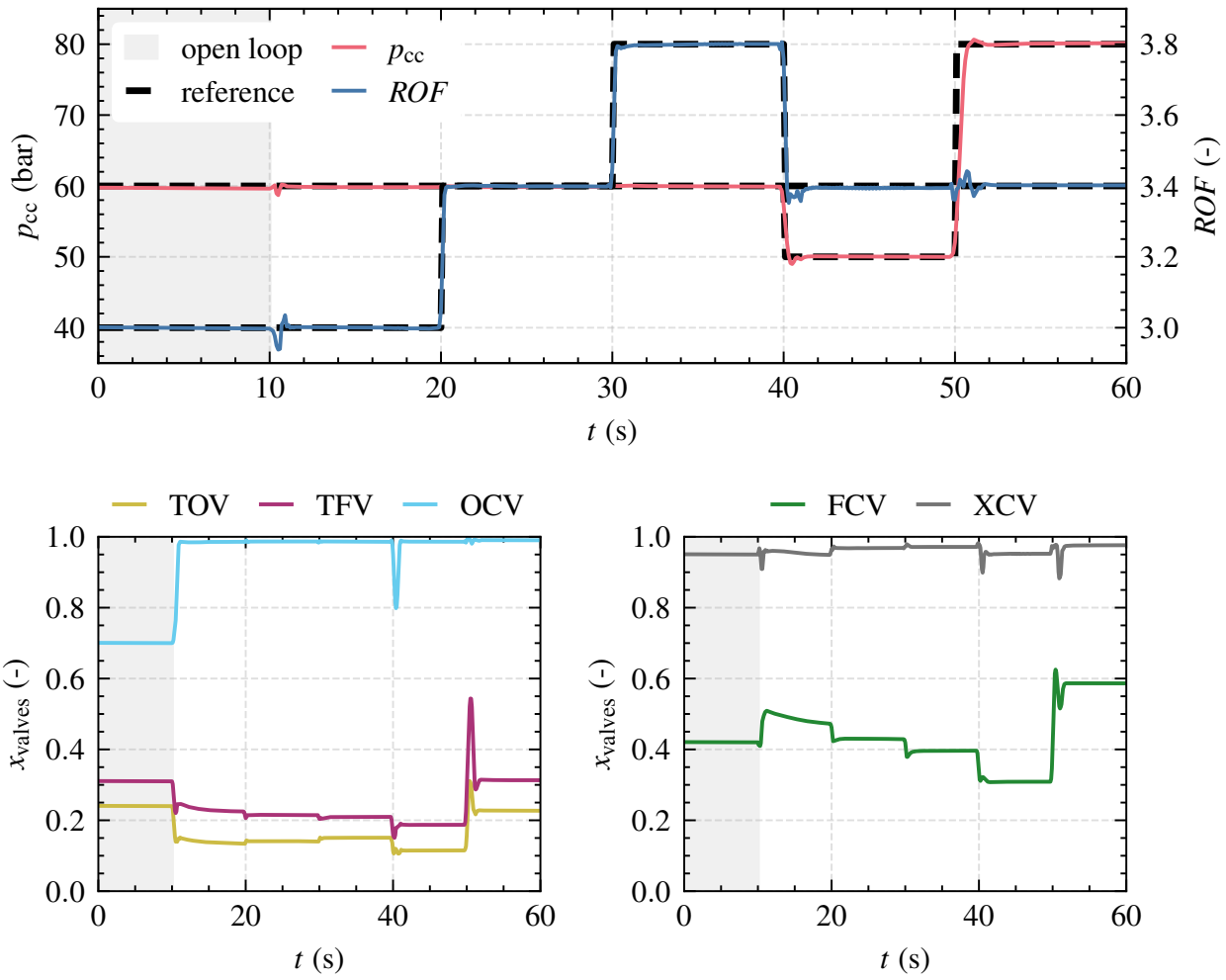


Figure 5.4.: Nominal tracking of the neural network controller

agent successfully changes p_{cc} at $t = 50\,\text{s}$ with only minimal deviations in ROF. Changing both quantities at $t = 40\,\text{s}$ is also performed without problems.

An interesting observation is the remarkable ability of the controller in changing p_{cc}. Even the largest change from 50 bar to 80 bar at $t = 50\,\text{s}$ is executed in less than $t = 2\,\text{s}$. This result is surprising given the large time constants of LUMEN, as found out in the model analysis in Section 4.3 with a mean settling time of 4.5 s for changes in p_{cc}.

The neural network is able to achieve this fast control by applying aggressive control inputs. The most notable example is the setpoint change at $t = 50\,\text{s}$, where p_{cc} is increased by 30 bar. The controller opens the turbine valves TOV and TFV for approximately 1.1 s beyond the final steady state positions to increase the turbine power and thus p_{cc} as quickly as possible. A similar effect is observed during engine throttling at $t = 40\,\text{s}$: OCV is only closed for 1 s to create an additional pressure drop on the oxidizer side to quickly lower p_{cc}.

5.3.2. Fuel efficiency

Next, the secondary functional requirement of fuel efficiency is analyzed. Figure 5.5 shows p_{cc}, the cooling channel mass flow $\dot{m}_{RC}$, the minimum cooling channel mass flow $\dot{m}_{RC,min} = f(p_{cc})$ and the turbine mass flow $\dot{m}_{turbine}$. In a previous study, it was shown that the highest steady-state fuel efficiency is achieved when LUMEN is operated at the minimum cooling channel mass flow $\dot{m}_{RC} = \dot{m}_{RC,min}$. In addition, XCV and OCV should be fully open to avoid unnecessary pressure losses [44].

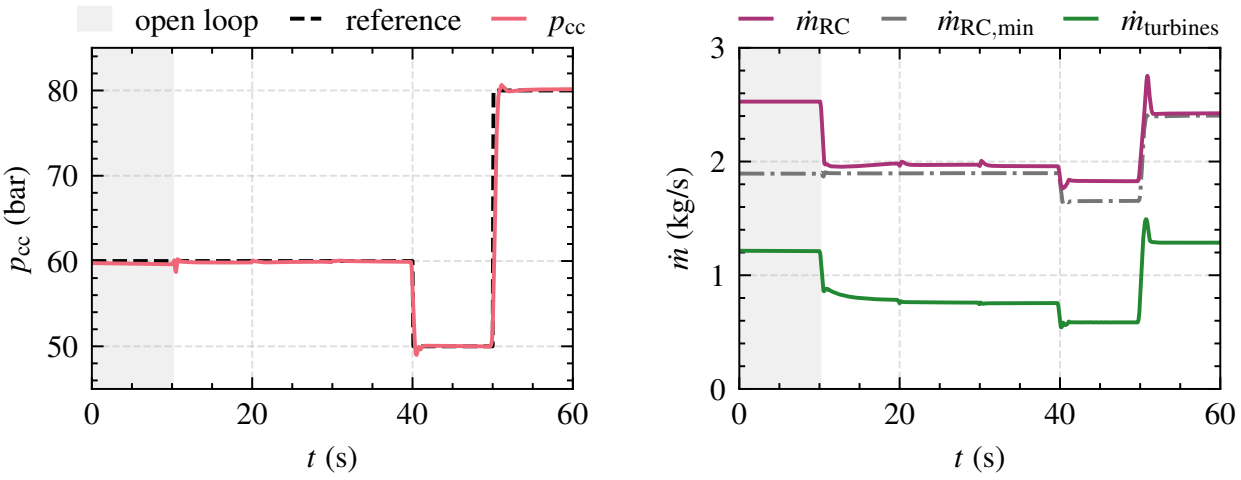


Figure 5.5.: Fuel efficiency analysis

The following observations can be made:

1. At $t = 10\,\mathrm{s}$ the controller is activated and directly reduces $\dot{m}_{turbine}$ by reducing $\dot{m}_{RC}$, while keeping p_{cc} and ROF constant.

2. The controller keeps $\dot{m}_{RC}$ very close to $\dot{m}_{RC,min}$, but slightly above its minimum value, sacrificing marginal fuel efficiency. A possible explanation is that a slightly higher $\dot{m}_{RC}$ allows the controller to speed up large setpoint changes. In other words, the agent may sacrifice marginal fuel efficiency to improve tracking accuracy, although it is difficult to verify this claim. However, the comparison with a model-predictive controller in the following sections will help to gain a deeper insight.

3. During the entire episode, the controller keeps $\dot{m}_{RC}$ always above $\dot{m}_{RC,min}$. There are no constraint violations in $\dot{m}_{RC}$.

4. The controller uses OCV and XCV only during transients and keeps them nearly fully open during steady state.

5.3.3. Parameter studies

The next step is to analyze the influence of different numbers of future reference values and different control frequencies.

Number of future references

The idea of adding a number of future reference values N_f to the observation space is to improve the transient response to setpoint changes. With this additional information, the agent can anticipate an upcoming setpoint change and react in advance if necessary. This is critical for systems with large time constants, where reacting after the setpoint has changed is too late.

Figure 5.6 compares two different controllers for a scheduled setpoint change at $t = 30.0\,\mathrm{s}$. The first one does not use future references with $N_f = 0$. The second uses the default of $N_f = 4$. The results are as expected: Without knowledge of future references, the first controller can only react after the setpoint has been changed and at $t = 30.2\,\mathrm{s}$ ROF starts to increase. In contrast, the future-aware controller with $N_f = 4$ anticipates the upcoming setpoint change and starts adjusting ROF already at $t = 29.9\,\mathrm{s}$.

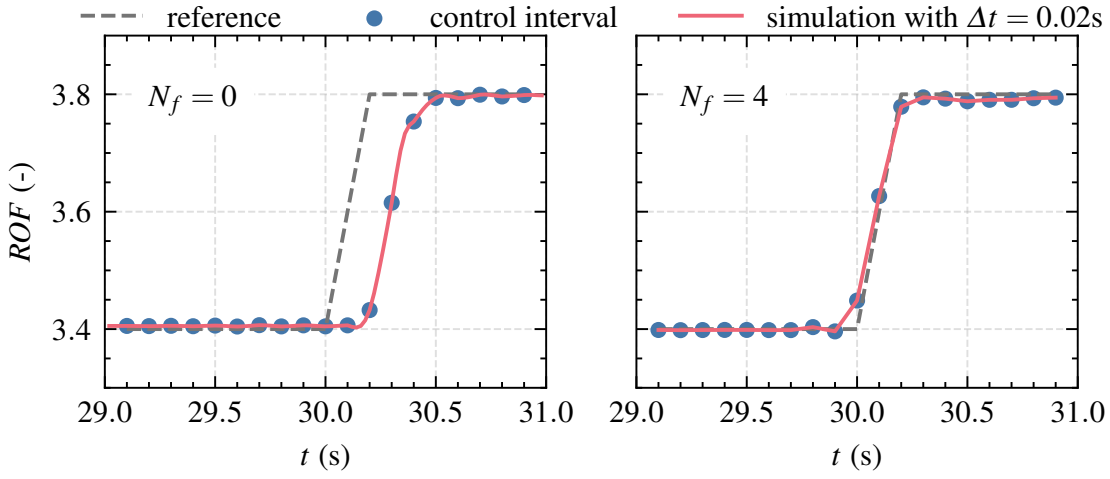


Figure 5.6.: Neural network controller for different number of future references

Control frequency

The sampling interval Δt, or control frequency, is an important parameter in control. At each sampling instance t, a new control signal is calculated and applied to the actuators. This signal is then held constant until the next sampling instance. For classical digital controllers, sampling interval that are

too large can degrade stability and damping properties [58]. The sampling interval is also critical for optimal control algorithms. Low sampling intervals allow faster responses to disturbances and higher accuracy in following a reference. For MPC, small sampling intervals require powerful processors for online optimization. Reinforcement learning, on the other hand, does not suffer from this drawback, since optimization takes place during training and studies have demonstrated a sampling interval as low as 1 ms [41].

Figure 5.7 compares three neural networks that are trained with different sampling times between 0.05 s to 0.2 s for a single step in ROF from 3.4 to 3.8. All controllers are evaluated by running the underlying simulation with an even smaller sampling time of $\Delta t_{\text{sim.}} = 0.02\,\text{s}$. The largest sample time $\Delta t_1 = 0.2\,\text{s}$ is too large for the system dynamics, failing to capture the overshoot in ROF between two sample intervals. A sample time of $\Delta t_2 = 0.1\,\text{s}$ seems sufficient, and $\Delta t_3 = 0.05\,\text{s}$ provides only marginal benefits in tracking performance.

The disadvantage of small sampling times with reinforcement learning is a potential increase necessary time steps for training. Here, between Δt_2 and Δt_3, the number of training steps until convergence increase from 1.5 million to 2.5 million. Because the necessary computation time per time step decreases for Δt, the total training time increases by (only) 10 %.

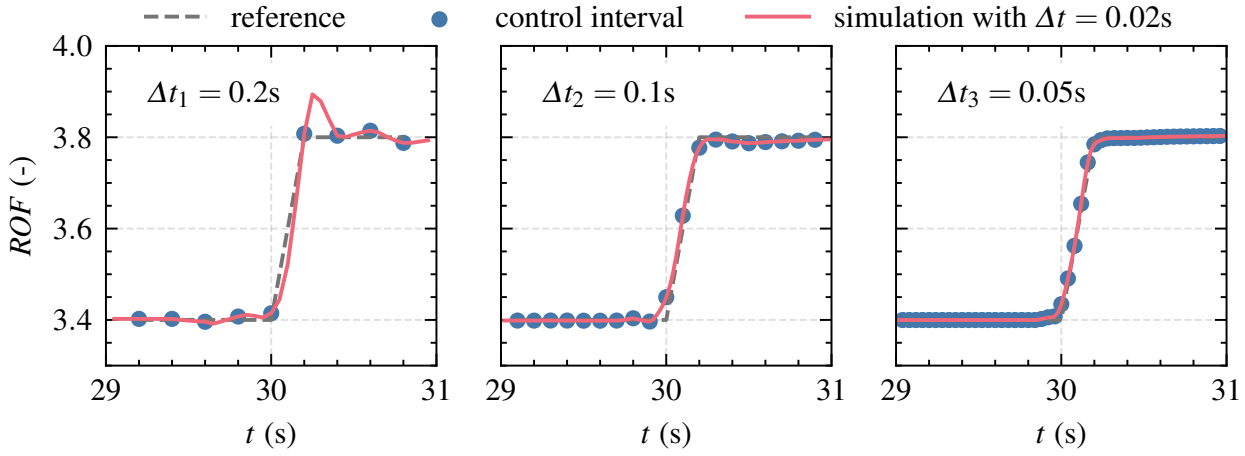


Figure 5.7.: Neural network controller for different control frequencies

5.4. Robustness studies

The high cost of real physical interactions incentives training in simulation only. However, a simulation rarely captures all physical aspects of the real system. Thus, it is necessary to study the agent's ability to handle real-world effects such as sensor noise or unavoidable modeling errors.

5.4.1. Sensor noise

Real measurements are always subject to sensor noise. To investigate the robustness of the neural network, Gaussian noise with a standard deviation of ±5 % is added to all measurements. Figure 5.8 shows that the controller provides good tracking performance even in this case. The noise causes only very small valve jittering, which could be eliminated by a low pass filter.

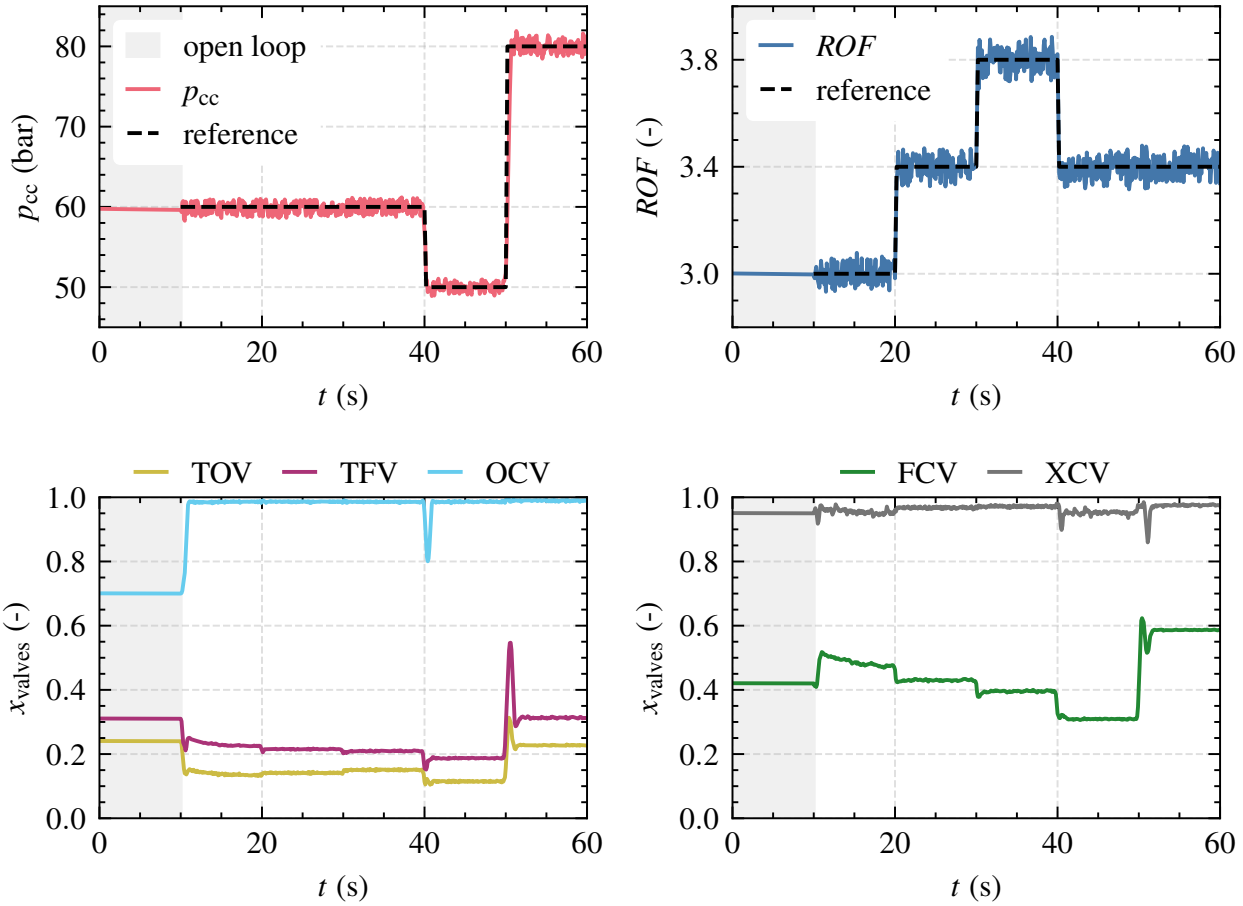


Figure 5.8.: Tracking performance with sensor noise

5.4.2. Modeling errors

The controller needs to be able to deal with unavoidable modeling errors. The idea is to expose the agent to model deviations during training via domain randomization. This allows the agent to learn a robust policy against model deviations. For this test case, domain randomization of a single variable, the fuel turbopump efficiency η_{FTP}, is studied. First, training starts as before, without domain randomization. When the agent reaches a predefined return, domain randomization is activated. Then, at the start of each episode, η_{FTP} is randomized with a uniform distribution between $\hat{\eta}_{\mathrm{FT}} \in [0.92, 1.0]\eta_{\mathrm{FT}}$.

Figure 5.9 compares an open-loop sequence, the baseline neural network that is trained without domain randomization, and the controller that is trained with domain randomization for a 5 % decrease in η_{FTP}:

In open loop, since there is no feedback, the error in p_{cc} and *ROF* is significant with a mean tracking error of 4.8 %. The constrained variable $\dot{m}_{\mathrm{RC}}$ is below its allowable lower bound for most of the episode.

The baseline controller reacts to model parameter variations and is able to slightly reduce the tracking error to 1.7 %. However, without domain randomization, it's performance is far from perfect. $\dot{m}_{\mathrm{RC}}$ also drops below its allowed lower limit for the last operating point.

The controller trained with domain randomization is able to adapt to the unknown parameter variation. It achieves a significantly better tracking with a mean error of 0.3 %. Domain randomization seems therefore suitable for making the controller robust to modeling errors and parameter changes without losing its control accuracy.

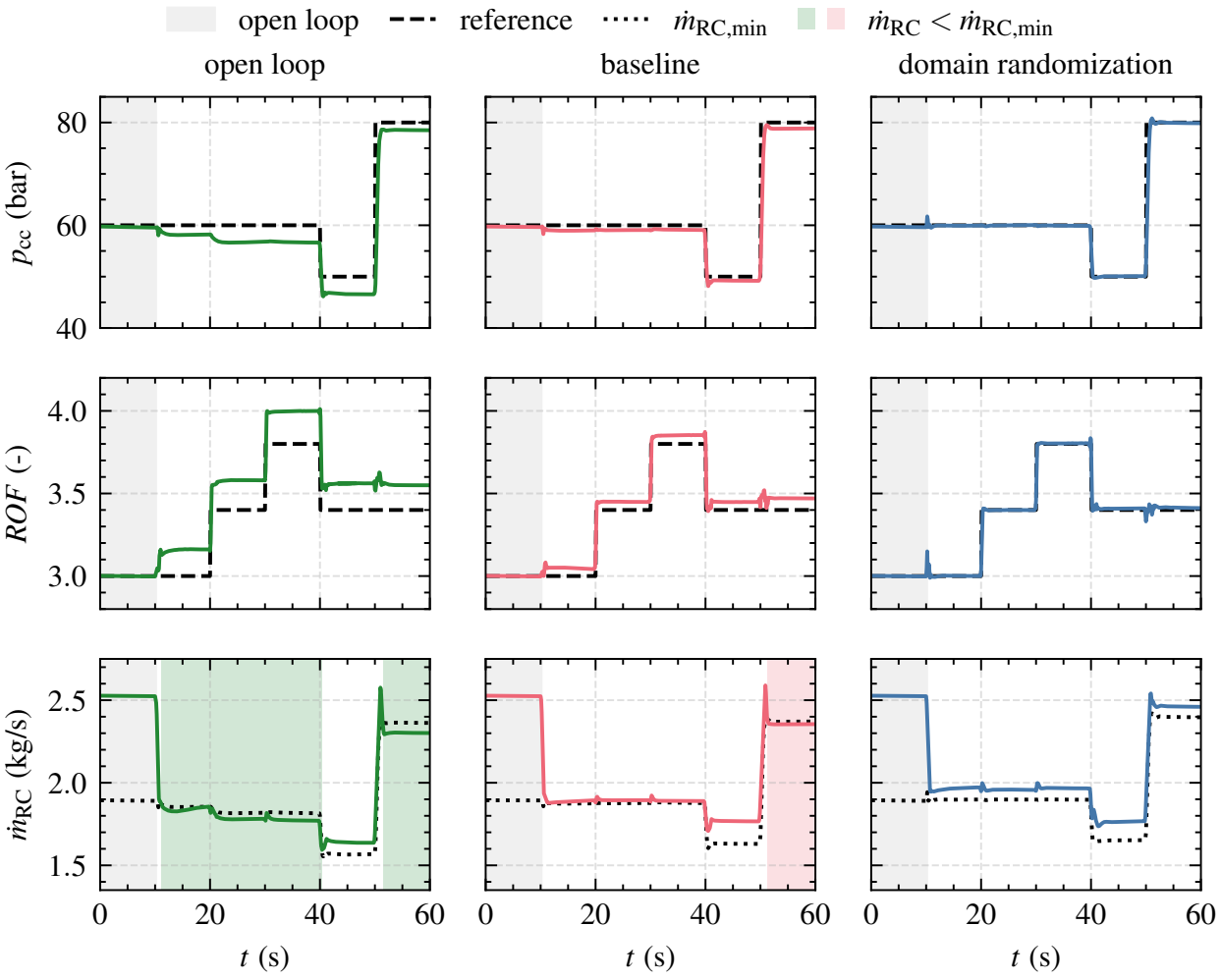


Figure 5.9.: Tracking performance for a 5 % lower fuel turbopump efficiency.

Table 5.3 compares the open-loop sequence, the baseline agent, and the controller trained with domain randomization qualitatively in terms of the mean percentage absolute error (MAPE) and the necessary training time. While domain randomization improves the robustness to parameter changes, it also doubles the training time. This extra training time is needed because the neural network must learn to estimate uncertain parameters indirectly given its observation vector while also learning the optimal control strategy for all parameter variations.

Table 5.3.: Control performance for a modeling error in η_{FT} of 5 %

	$MAPE$ in %				
	$\hat{\eta}_{FT} = 1$		$\hat{\eta}_{FT} = 0.95$		
	p_{cc}	ROF	p_{cc}	ROF	Train steps
Open-loop	0.3	0.1	4.7	4.9	–
Baseline	0.3	0.1	1.7	1.6	$1.5e6$
Domain randomization	0.4	0.3	0.3	0.2	$2.9e6$

5.5. Comparison to model predictive control

MPC is often the state-of-the art choice for controlling complex physical systems because it can naturally handle constraints and achieve high control performance by using a system model. In partnership with the Institute of Automatic Control at RWTH University Aachen, an MPC controller was designed for this test case. The development began with the work by Valchev [205] and was continued internally at the Institute. The system model and optimization loops are implemented in Matlab using the IPOT solver [208] and the CasADi interface [11]. The implementation is summarized below:

The MPC control loop works in two steps: First, a target selector calculates the optimal steady-state point based on the reference values, constraints, and the goal of fuel efficiency. Secondly, dynamic control inputs are optimized based on a dynamic system model, the target selector output, constraints and the prediction horizon. The system model is formulated as a data-based Wiener model with linear dynamics and a nonlinear steady-state output function in the form of a neural network [99]. Compared to the traditional approach of using a state-space model, where physics-based modeling is extremely time-consuming, the data-based Wiener model can be learned directly from simulation data from the V2 LUMEN EcosimPro model. Since every system

model is only a rough approximation of the real-world behavior, extensions are necessary: Offset-free tracking is achieved via disturbance estimation [145] with an extendend Kalman filter [183]. Constraints are formulated as soft constraints to guarantee a solution to the optimization [121]. MPC parameters, such as the prediction horizon or constraint penalties, are manually tuned to achieve the best tradeoff between performance and constraint handling.

Tracking performance

Figure 5.10 compares the tracking performance between MPC and the neural network trained with reinforcement learning (RL). Both controllers successfully reach all operating points with low steady-state error. Overall, the neural network achieves a tracking error of 0.3 % in p_{cc} and 0.1 % in ROF. The MPC controller has slightly larger errors of 1.4 % and 0.6 %.

RL provides faster tracking and smaller overshoots. In contrast, the MPC controller is more conservative with longer settling times. A notable example is at 50 s when p_{cc} is throttled up from 50 bar to 80 bar. The MPC controller requires 3.0 s until p_{cc} stabilizes at the new load point within a 2 % band. The neural network accomplishes the set-point change in 0.7 s. Other examples are at 20 s and 30 s: while changing ROF, deviations in p_{cc} occur for the MPC controller, whereas RL avoids any deviations.

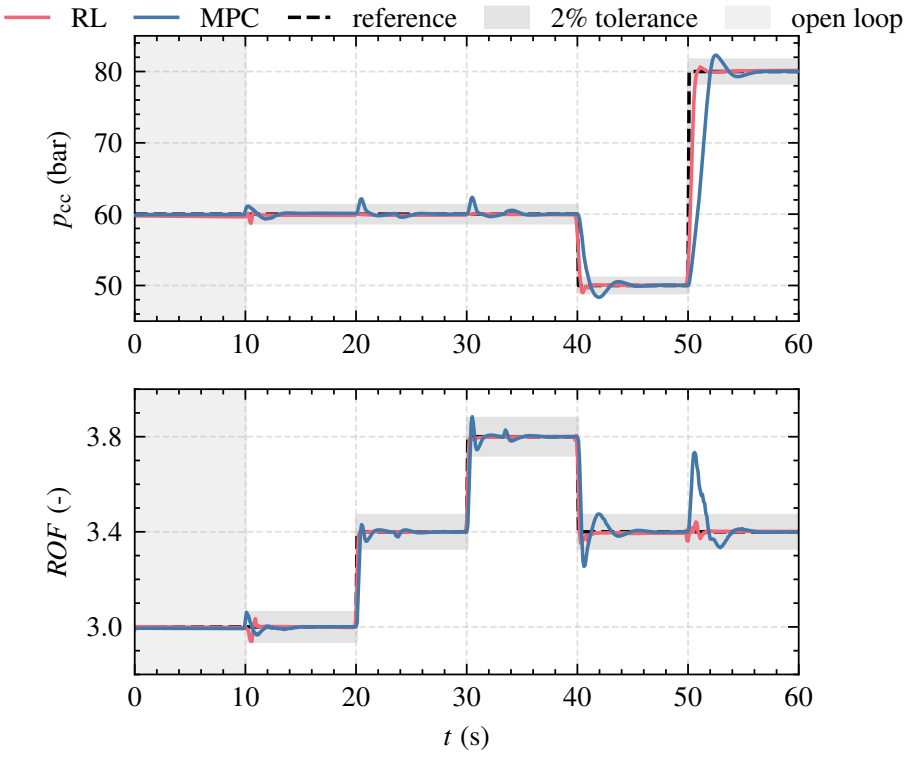


Figure 5.10.: Comparison between MPC and RL: Tracking performance

The aggressiveness of the MPC controller can possibly be increased by adjusting its hyperparameters. However, this comes at the cost of an increased risk of constraint violations or overshoots. The reason for this is the limited accuracy of the MPC system model due to the necessary simplifications imposed by the resource-intensive online optimization process.

Flow control valve usage

Figure 5.11 compares the valve positions between RL and MPC. Overall, both controllers use similar valve positions in steady state. This indicates that both controllers have found similar, near-optimal steady-state solutions. Both controllers use smooth valve commands without oscillations. The figure also shows the more aggressive control by RL during setpoint changes. The most notable example is when the combustion chamber pressure is increased at $t = 50\,\mathrm{s}$, where RL opens the turbine valves TOV and TFV more aggressively. Another example is at $t = 40\,\mathrm{s}$, where RL briefly closes OCV to throttle down the engine - a strategy not used by the MPC controller.

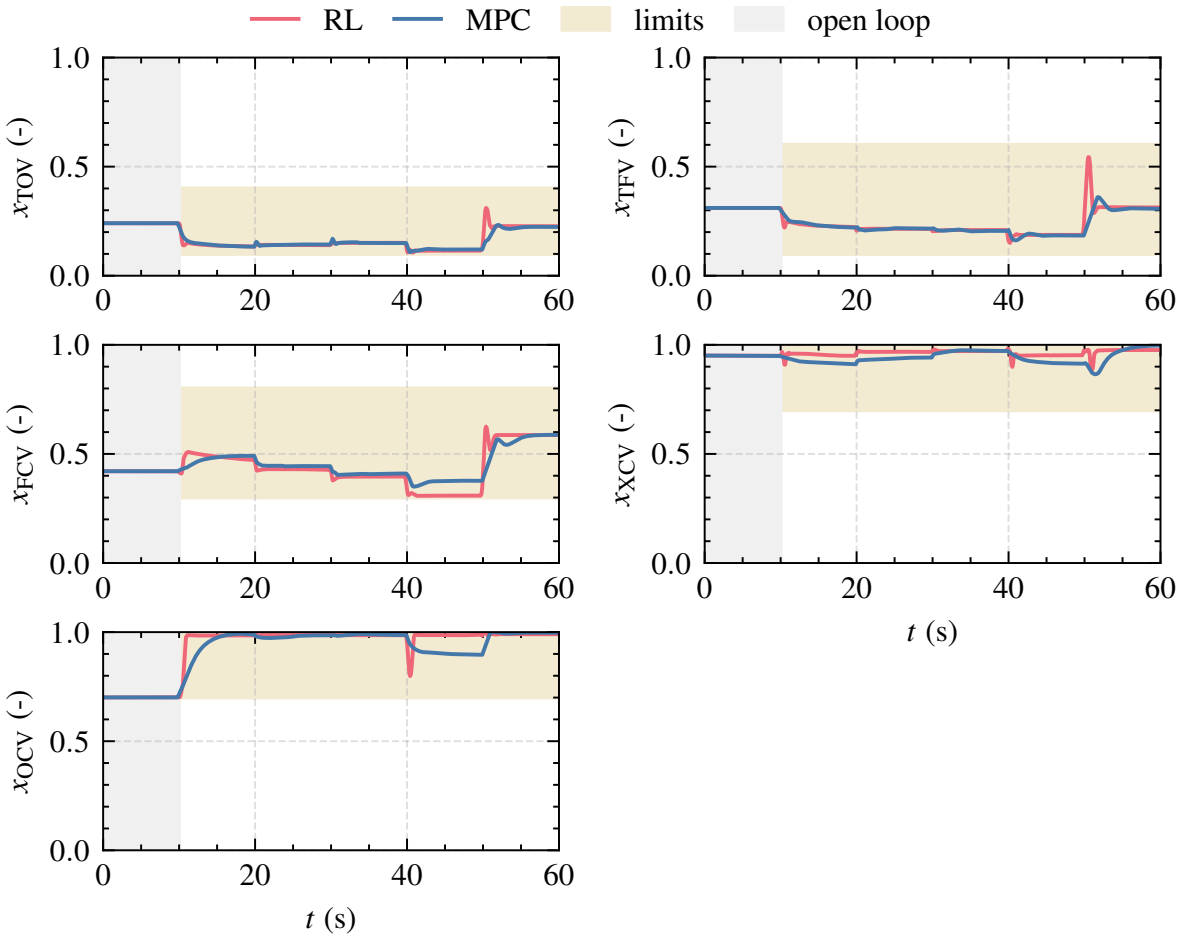


Figure 5.11.: Comparison between MPC and RL: Valve positions

Constraint handling capabilities

Figure 5.12 compares the constraint handling capabilities of the two controllers for the most critical variables: mixture ratio ROF, cooling channel mass flow $\dot{m}_{\mathrm{RC}}$, injection momentum flux ratio J, and turbine inlet temperature T_{turbine}. Both controllers operate within the constraint limits during steady state. During transients, the MPC controller suffers from few constraint violations: $\dot{m}_{\mathrm{RC}}$ drops 22 time steps below its lower limit between 20 s and 35 s, but the magnitude and time of the violation is small with a maximum deviation of $0.022\,\mathrm{kg\,s^{-1}}$, or about 1.2 %. The neural network, on the other hand, is able to satisfy all constraints even during transients.

The challenge of this test case lies in the opposing control goals of constraint handling and fuel efficiency: On the one hand, $\dot{m}_{\mathrm{RC}}$ needs to be minimized to maximize fuel efficiency. On the other hand, operating close to the lower limit $\dot{m}_{\mathrm{RC,min}}$ means that small modeling errors can lead to minor constraint violations. These deviations are quickly corrected by the MPC controller and could probably be avoided by tuning its design parameter. For practical applications, one should consider including safety margins into the constraint limits if those are critical.

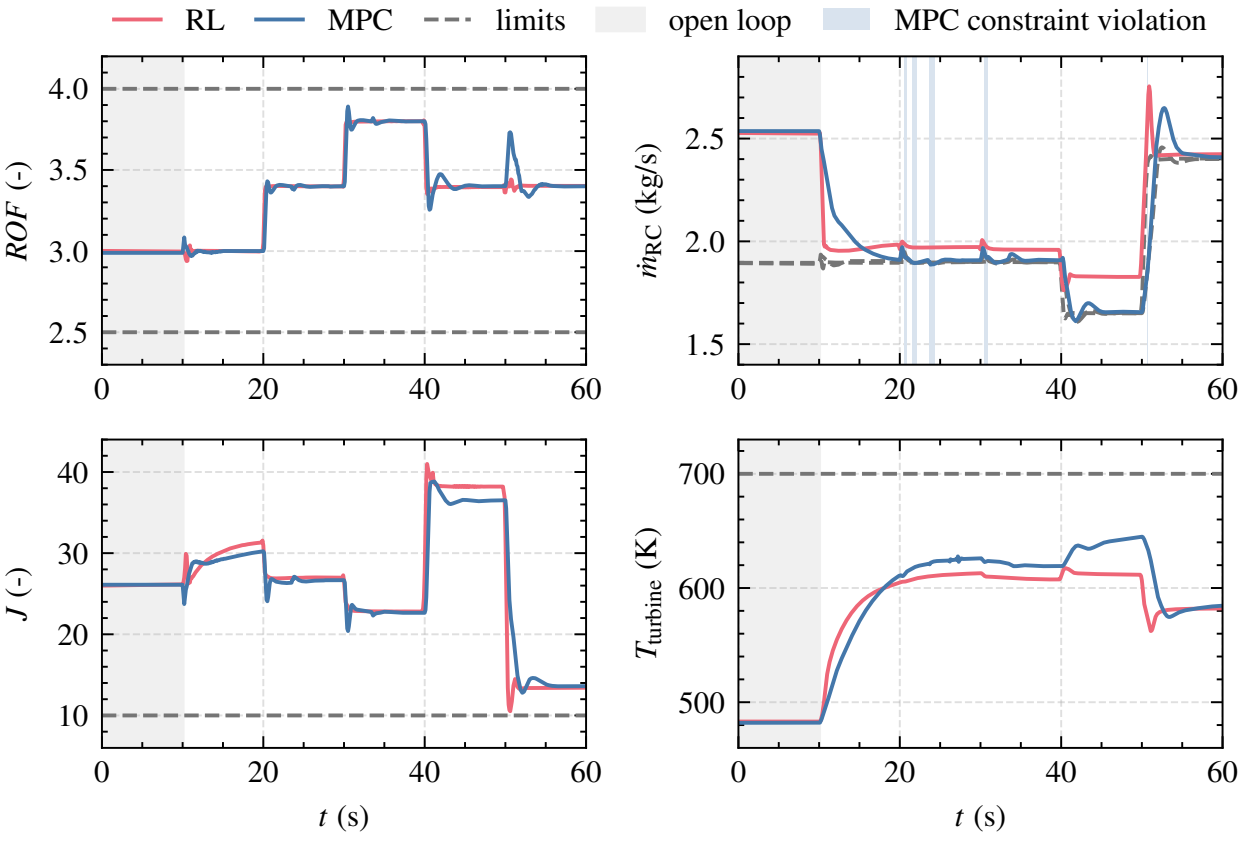


Figure 5.12.: Comparison between MPC and RL: Constraint handling

Fuel efficiency

Figure 5.13 compares the turbine mass flow $\dot{m}_{\text{turbines}}$, which is an indicator for the fuel efficiency. The goal is to minimize $\dot{m}_{\text{turbines}}$. During steady state, the MPC controller is slightly more fuel efficient with marginal lower $\dot{m}_{\text{turbines}}$. The biggest difference is at the operation point at 40 s with an offset of $\Delta\dot{m}_{\text{turbines}} = 0.026\,\text{kg}\,\text{s}^{-1}$ or 4.4 %. During transients, $\dot{m}_{\text{turbines}}$ is difficult to compare due to the different settling times of both controllers. As a global metric, the integrated turbine mass flow over the episode can be compared. The MPC controller uses a total of $m_{\text{turbines}} = 53.5\,\text{kg}$ of methane for the turbines. The neural network is slightly less fuel-efficient, using $m_{\text{turbines}} = 54.4\,\text{kg}$. In terms of the total mass flow of oxidizer and fuel, this is a small propellant saving of 0.38 %.

Differences in fuel-efficiency were already observed when discussing the constraint handling capabilities. The MPC controller optimizes fuel consumption by operating the engine very close to the lower limit of the cooling channel mass flow. The neural network seems to slightly sacrifice fuel efficiency either to speed up setpoint changes or to reduce the risk of constraint violations.

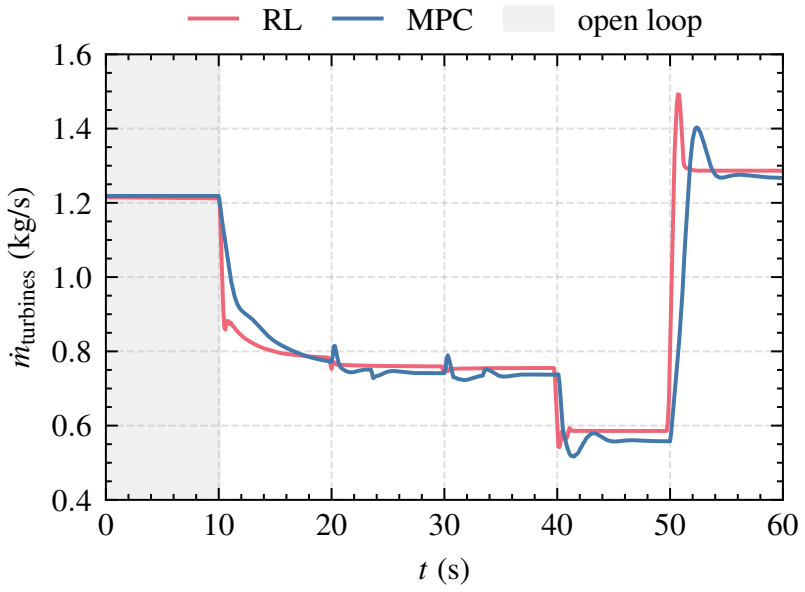


Figure 5.13.: Comparison between MPC and RL: Fuel efficiency

Quantitative comparison

Both controllers are finally compared quantitatively in terms of tracking performance, settling time, necessary control effort, and propellant usage.

The discrete integrated absolute error (IAE)

$$IAE = \sum_{k=0}^{k=\infty} |e_k| \Delta t \tag{5.10}$$

is used to evaluate tracking performance, weighting all errors $e_k = y - y_{\text{reference}}$ equally, regardless of their magnitude. The settling time is the duration required for all controlled variable to reach and stay within a 2 % band around the new steady-state point.

The total control effort Δu that is used by each controller is calculated as the summed difference of valve commands between two time steps. High values indicate an agressive control policy. Δu is defined as

$$\Delta u = \sum_{t=10}^{60} |u(t) - u(t-1)|. \tag{5.11}$$

Table 5.4 compares both controllers in terms of tracking performance, settling time, necessary control effort, and propellant usage over the entire trajectory.

Table 5.4.: Comparison between MPC and RL: Control performance

	Tracking IAE		Settling time		Control	Economy
	p_{cc}	ROF	$t_{\max}$ [s]	t_{mean} [s]	Δu	m_{bleed}[kg]
MPC	54.3	1.1	3.0	1.5	2.4	53.5
RL	12.2	0.3	0.7	0.3	3.9	54.4

Summary

The comparison of the neural network trained with reinforcement learning and a state-of-the-art MPC controller indicate that both approaches are suitable for fuel-efficient rocket engine control. Both controllers show sufficient tracking performance and achieve comparable steady-state setpoints. They differ in their aggressiveness during operating point changes, their constraint handling abilities, and how they deal with model parameter deviations.

Tracking performance: The neural network demonstrates superior tracking performance with faster settling times and smaller overshoots. The advantage of the neural network is that it is trained with the full nonlinear simulation model, allowing it to learn and leverage all system dynamics. The offline

nature of the training process further allows for extensive optimization. In contrast, the MPC's system model, constrained by online optimization, is limited in complexity. Consequently, the MPC controller is more conservative to mitigate constraint violations. It seems possible to increase the tracking speed of the MPC controller by adjusting its hyperparameters. However, this comes at the direct cost of an increased risk of constraint violations or overshoots in the case of deviations between the limited MPC's system model and the real world behavior.

Robustness to modeling errors: Models are only an approximation of the real system. Modeling errors will result in a persistent steady-state offset if not properly handled by the controller. The basic idea of offset-free control is to continuously correct for any steady-state errors between the predicted model behavior and the actual system by incorporating an estimator to ensure that the system reaches the desired setpoint without persistent deviation. An example of offset-free control is the integral component of PID controllers. For MPC, different formulations have been studied [145]. The simplest form is an output correction term defined as the difference between the measured and the predicted output. A more advanced approach is estimating the modeling errors through disturbance estimation with an extended Kalman filter. As a result, the MPC controller provides offset-free control even when modeling errors are not explicitly quantified during controller design. If the model deviations can be quantified, robustness theorems have been developed that guarantee robustness under certain assumptions [176].

In contrast, standard reinforcement learning lacks inherent robustness to modeling errors. Domain randomization can be applied, but robustness guarantees are still lacking for standard reinforcement learning. Robust reinforcement learning is currently an active area of research [127].

Optimal control and deployment: The similarities between MPC and reinforcement learning are not surprising: both algorithms are optimal control techniques for dynamic systems that use a cost function to determine the optimal next control action and aim to minimize expected future costs over a prediction horizon. An important difference lies in the online and offline nature of the optimization. MPC performs optimization online, resulting in high computational demand during deployment. This limitation affects the complexity of the system model that MPC can effectively use. In contrast, reinforcement learning performs optimization offline, which allows the use of more complex and nonlinear system models and makes deployment numerically cheaper. The relationship between model predictive control and reinforcement learning has been extensively studied [54, 65, 209]. In some cases, these two approaches can even be combined, as shown by Pan et al. [144].

6. Test case 2: Real-world demonstration

In test case 2, a closed-loop controller for continuous throttling of LUMEN is developed and then tested at the real-world LUMEN engine at the P8.3 test facility. The control objective is motivated by the following scenario: a high-level vehicle controller continuously calculates thrust requirements in real time based on the vehicle's position, attitude, and velocity to perform a landing maneuver. These requirements are then communicated to the low-level engine controller, which adjusts the engine thrust within operational constraints.

The chapter is organized as follows: First, the control objectives are formulated and the reinforcement learning setup is defined in terms of observation space, action space, and reward function, followed by the training process. Then, the performance of the controller is analyzed in simulation, focusing on its tracking accuracy and robustness to model parameter changes and sensor noise. Finally, the controller is experimentally tested.

6.1. Control objectives

The control objectives differ slightly from those of the first test case. In test case 1, the controller follows a predetermined trajectory of combustion chamber pressure (p_{cc}) and mixture ratio (ROF). With the known trajectory, the control policy is optimized for this specific scenario, resulting in an optimal yet non-generalized controller. In this second test case, the controller is tasked with managing arbitrary changes in p_{cc} across the operational envelope of the engine, between 40 bar to 60 bar.

In addition to p_{cc}, three other quantities must be regulated: ROF should be kept constant at a value of 3.0. The fuel injection temperature (T_{LNG}) should be kept at a constant value of 270 K for stable flame anchoring. The cooling channel mass flow ($\dot{m}_{\mathrm{RC}} = f(p_{\mathrm{cc}})$) should be adjusted based on the current measurement of p_{cc} to ensure sufficient cooling of the combustor walls. The controller should output commands to the four flow control valves TOV, TFV, FCV, and BPV. The control sampling time is $T = 0.05\,\mathrm{s}$.

Figure 6.1 shows example reference trajectories of p_{cc} used for training. Each sequence contains ramps with short steady-state plateaus. The step sizes and slopes are randomized to ensure that different profiles are created. The target

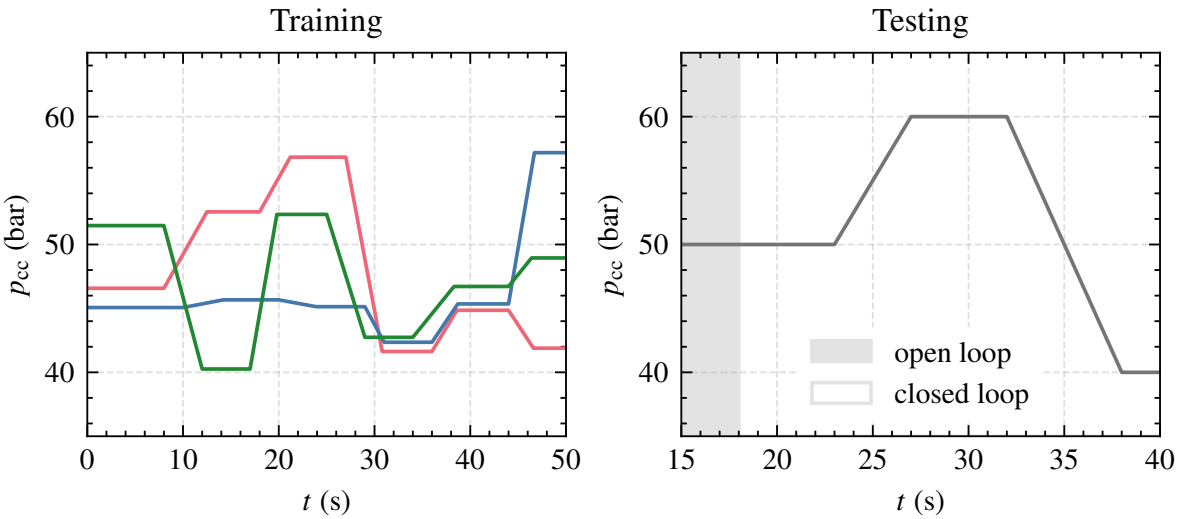


Figure 6.1.: Example reference trajectories of combustion chamber pressure

pressure levels and ramp times are uniformly sampled between 40 bar to 60 bar and 2 s to 5 s. Figure 6.1 also shows the test trajectory. It is simpler, with only two ramps. The test trajectory is not used during training and is reserved for performance evaluation and experimental testing.

Table 6.1.: Summary of the control objectives

Objective	Criteria	Value
Controlled variables	Chamber pressure	$p_{cc} \in [40, 60]$ bar
	Mixture ratio	$ROF = 3.0$
	Injection temperature	$T_{LNG} = 270\,K$
	Coolant mass flow	$\dot{m}_{RC} = f(p_{cc})$
Thrust changes	Ramps	$t_{Ramp} \in [2, 5]$ s
Actuators	Valve commands	TOV, TFV, FCV, BPV
Performance	Sampling time	$\Delta t_s = 0.05\,s$
	Steady-state error	$e_\infty < 2\,\%$
Robustness	Gaussian noise	$\sigma_{noise} = 1.5\,\%$
	Modeling errors	according to Table A.3

Table 6.1 summarizes the control objectives. The controller is required to achieve continuous throttling of p_{cc} with ramps of $t_{Ramp} \in [2, 5]$ seconds while keeping ROF and T_{LNG} constant. At the same time, $\dot{m}_{RC}$ must be adjusted based on p_{cc}. Control performance is evaluated based on the steady state error and the transient response. Deviations should be less than 2 %, and

transients should be performed with low overshoots and no oscillations. The controller further needs to be robust against Gaussian sensor noise with a standard deviation of $\sigma_{\text{noise}} = 1.5\,\%$ and against modeling errors.

6.2. Controller synthesis

A suitable learning setup in terms of observation space, action space, and reward function was established in test case 1. With the flexibility of reinforcement learning, this setup is adapted to this test case with only minor modifications to align with the new control objectives. The LUMEN V3 EcosimPro model, which is based on experimental data from the subcomponent test campaigns, is used as specified in Table 4.2. It includes sensor delays as defined in Table 4.5 and allows the addition of sensor noise.

Training and reference trajectory

The reinforcement learning framework and hyperparameters for the SAC algorithm are reused from test case 1. The generation of each training episode is adapted to the task of continuous throttling: Training the agent requires exposing it to a large number of different profiles. Therefore, at the beginning of each episode, a randomly generated reference trajectory with multiple operating point changes is created, as illustrated in Figure 6.1. In addition, each episode starts with slightly randomized valve positions of $\pm 5\,\%$.

Action space

The action space A is given by continuous commands u to the four control valves TOV, TFV, FCV, and BPV. The upper and lower positions are tailored to the operational envelope.

$$A = \begin{bmatrix} \mathrm{u_{TFV}} \\ \mathrm{u_{TOV}} \\ \mathrm{u_{FCV}} \\ \mathrm{u_{BPV}} \end{bmatrix} \in \begin{bmatrix} [0.2, 0.7] \\ [0.1, 0.4] \\ [0.2, 0.4] \\ [0.6, 1.0] \end{bmatrix} \tag{6.1}$$

Observation space

The observation space contains measurements of the controlled variables $Y \in \mathbb{R}^4$, the current reference values $Y_{\text{ref.}} \in \mathbb{R}^4$, the current valve positions $Y_{\text{valve}} \in \mathbb{R}^4$, and the last command signals $U \in \mathbb{R}^4$ to the control valves as

defined in Equation 6.2. The observation space is extended with additional measurements $Y_{\text{add.}} \in \mathbb{R}^3$, which are not necessarily required but can provide additional information during domain randomization. For example, by adding the cooling channel outlet temperature $T_{\text{RC,out}}$, the agent can indirectly estimate the thermal heat flux from the combustion chamber into the coolant. $p_{\text{pump,LOX}}$ and $p_{\text{pump,LNG}}$ are the pump outlet pressures of the oxidizer and fuel pumps, which can help estimate turbopump efficiency.

$$Y = \begin{bmatrix} p_{\text{cc}} \\ ROF \\ T_{\text{LNG}} \\ \dot{m}_{\text{RC}} \end{bmatrix}, \quad Y_{\text{ref.}} = \begin{bmatrix} p_{\text{cc,ref}} \\ ROF_{\text{ref}} \\ T_{\text{LNG,ref}} \\ \dot{m}_{\text{RC,ref}} \end{bmatrix}, \quad Y_{\text{add.}} = \begin{bmatrix} p_{\text{pump,LOX}} \\ p_{\text{pump,LNG}} \\ T_{\text{RC,out}} \end{bmatrix},$$
$$Y_{\text{valves}} = \begin{bmatrix} \text{x}_{\text{TFV}} \\ \text{x}_{\text{TOV}} \\ \text{x}_{\text{FCV}} \\ \text{x}_{\text{OCV}} \end{bmatrix}, \quad U = \begin{bmatrix} \text{u}_{\text{TFV}} \\ \text{u}_{\text{TOV}} \\ \text{u}_{\text{FCV}} \\ \text{u}_{\text{OCV}} \end{bmatrix} \tag{6.2}$$

In summary, the observation vector of each time step is defined as

$$\tilde{o}_t = Y \oplus Y_{\text{ref.}} \oplus Y_{\text{add.}} \oplus Y_{\text{valves}} \oplus U \in \mathbb{R}^{19} \tag{6.3}$$

Similar to test case 1, the observation vector is formed by stacking three observations ($N_p = 3$), resulting in a total of 57 elements. Future reference values are not included, as the reference trajectory is not known in advance. Instead, the controller is designed to react to changing references immediately.

Reward function

The reward function $R(t)$ consists of two terms, each reflecting a specific control objective:

$$R(t) = \sum_{y \in Y} r_{\text{tracking,y}}(t) + \alpha_u \sum_{u \in U} r_{\text{actuators,u}}(t) \tag{6.4}$$

The first term evaluates the tracking performance, similar to test case 1. The tracking reward is computed using an exponential function of the absolute percentage error, as defined in Equation 5.7. The second term penalizes actuator usage to prevent oscillating valve commands. For each valve the actuator reward at time step t

$$r_{\text{actuators,u}}(t) = |u(t - \Delta t) - u(t)| \text{ for } u \in U \tag{6.5}$$

is calculated as the absolute difference between two consecutive valve commands. $\alpha_u = 5$ is a scaling factor to weight the penalty of actuator usage.

Curriculum learning and domain randomization

The training is divided into several distinct phases, each with an increasing level of complexity. This curriculum learning approach can decrease the necessary training time as described in Section 3.5:

1. Baseline training with nominal model parameters
2. Domain randomization of model parameters according to Table A.3
3. Addition of Gaussian sensor noise

The first training phase is performed with the set of nominal model parameters. In this phase, the agent starts the training process by randomly selecting actions. With increasing experience, the agent learns the dynamics of the system and how to use the flow control valves to achieve the pressure level changes, while also controlling the three other quantities.

The second training phase is initiated as soon as the agent achieves an average return greater than -5, indicating satisfactory performance on the nominal model parameters. Domain randomization is enabled, and model parameters from Table A.3 are randomly modified at the beginning of each episode using the truncated normal probability distribution from Figure 3.2. The standard deviation of this distribution σ_{DR} is gradually increased until it approximates a uniform distribution within the specified parameter bounds.

The model parameters that are randomized are chosen based on the component modeling in Chapter 4.2. The amount of randomization is determined by two factors: the uncertainty of the specific parameter based on the domain knowledge of the component experts, and its impact on the system according to the EcosimPro model. It is a trade-off between achieving high robustness and avoiding excessive changes in system dynamics beyond physically reasonable values, which can lead to longer training times and potentially poor control performance. Therefore, the selected parameters are randomized between $\pm 1\,\%$ and $\pm 10\,\%$ around the nominal value.

Large time delays in the system response or in the measurement chain itself can result in poor and unstable control performance. Therefore, sensor delays are also randomized up to 1.5 times the nominal value. For example, p_{cc} is averaged at the test bench with a moving average of 0.1 s. For domain randomization, it is randomized between 0.05 s to 0.15 s.

In the second third phase, sensor noise is added to all pressure and mass flow measurements. The noise is sampled from a Gaussian distribution. In each episode, a standard deviation $\sigma_{\mathrm{noise}} \in [0, 1.5\,\%]$ is sampled and the same noise level is applied to all noisy sensors. Temperature measurements are assumed to be noise free.

6.2.1. Training results

During training, the policy is evaluated after every 20 episodes of 1000 steps each. The evaluation is performed by randomly generating 10 episodes with different combustion chamber pressure profiles. Evaluating the policy over multiple episodes is critical in stochastic environments because the maximum achievable return depends on the specific setting. For example, the agent may have already learned how to operate LUMEN at high pressure levels, but does not yet know how to throttle down the engine.

Figure 5.3 shows the training progress in terms of the undiscounted cumulative reward or return. In the early stages, the agent uses random actions to explore the environment. As the agent acquires knowledge of the system dynamics, its performance steadily improves. After about 1.9 million training steps, the return exceeds the predefined threshold of -5 and domain randomization is activated with an initial standard deviation of $\sigma_{\mathrm{DR}} = 0.2$.

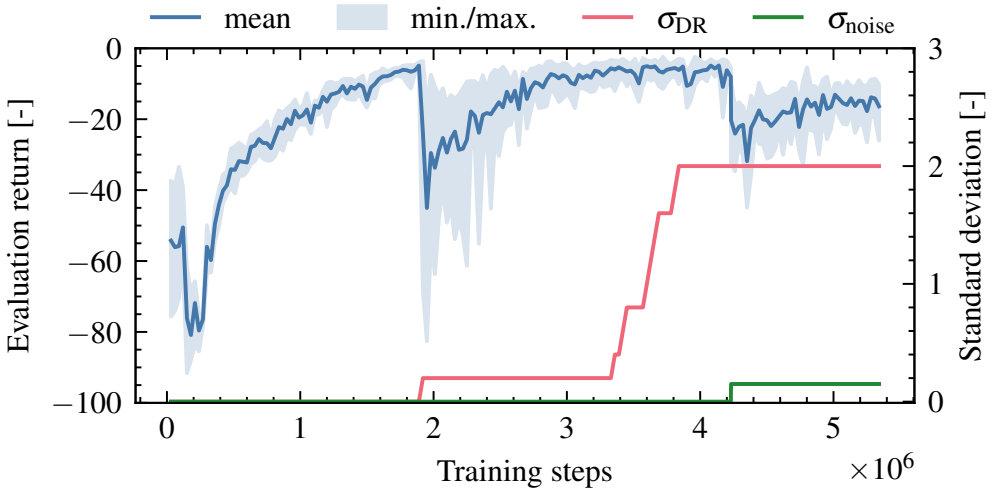


Figure 6.2.: Training progress with domain randomization and sensor noise

Due to the now unknown and changing model parameters, the return drops significantly. However, over the next 1.2 million training steps, the agent learns how to be robust to model parameter variations. Once the return has recovered, $\sigma_{\mathrm{DR}} = 0.2$ is further increased. After about 4.1 million training steps, sensor noise is activated with $\sigma_{\mathrm{DR}} = 0.15$.

Overall, the training results are promising because the mean return converges to a high value, indicating good performance of the trained agent and robustness to model parameter variations. Additionally, the agent is able to reduce the spread between the worst and best performance during domain randomization. This indicates that the agent is able to achieve high control performance for many combinations of model parameter variations.

6.3. Controller analysis

The controller is evaluated in simulation to ensure that it meets the control objectives defined in Table 6.1. First, the tracking performance is examined using the test trajectory with the default set of model parameters. In addition, it is analyzed whether the controller can throttle the engine over the entire operating envelope between 40 bar to 60 bar. Finally, a Monte Carlo study is performed to evaluate the robustness of the controller to modeling errors and sensor noise.

6.3.1. Control metrics

A new set of control metrics is introduced to evaluate the tracking performance across all four controlled variables. The **tracking error** $e_{\mathrm{Y}}(t)$ for each controlled variable Y at time step t is defined as the absolute percentage error:

$$e_{\mathrm{y}}(t) = \frac{|y(t) - y_{\mathrm{ref}}(t)|}{y_{\mathrm{ref}}(t)}. \tag{6.6}$$

The **mean tracking error** $e_{\varnothing}(t)$ at time step t is given by averaging $e_{\mathrm{Y}}(t)$ over all four controlled variables:

$$e_{\varnothing}(t) = \frac{1}{4}\left[e_{\mathrm{p_{cc}}}(t) + e_{\mathrm{ROF}}(t) + e_{\mathrm{T_{LNG}}}(t) + e_{\dot{\mathrm{m}}_{\mathrm{RC}}}(t)\right]. \tag{6.7}$$

The **average tracking error** is calculated by averaging over the episode:

$$\bar{e}(Y) = \frac{1}{T}\sum_{t=t_0}^{T} e_Y(t) \quad \text{and} \quad \bar{e}_{\varnothing} = \frac{1}{T}\sum_{t=t_0}^{T} e_{\varnothing}(t), \tag{6.8}$$

The **maximum tracking error** is defined as the largest error during the episode:

$$\bar{e}_{\max} = \max_{t>t_0+\Delta t}\left[e_{\mathrm{p_{cc}}}(t), e_{\mathrm{ROF}}(t), e_{\mathrm{T_{LNG}}}(t), e_{\dot{\mathrm{m}}_{\mathrm{RC}}}(t)\right]. \tag{6.9}$$

Here, t_0 is the time at which control is activated. Δt should be set greater than zero to give the controller time to adjust the initial error to zero. In this test case, $\Delta t = 2\,\mathrm{s}$ is used, i.e. if control is activated at $t_0 = 2\,\mathrm{s}$, $\bar{e}_{\max}$ is calculated from $t = 4\,\mathrm{s}$ to the end of the episode.

6.3.2. Tracking performance

Figure 6.3 shows the nominal tracking performance in terms of the controlled variables and valve positions for the test trajectory. The controller is activated at $t = 18\,\mathrm{s}$ and closely follows the reference with a mean tracking error of $\bar{e}_{\varnothing} = 0.3\,\%$, which is well below the required performance target of $2\,\%$. The errors in all controlled variables are similar, with $\bar{e}_{\mathrm{p_{cc}}} = 0.5\,\%$, $\bar{e}_{\mathrm{ROF}} = 0.4\,\%$, $\bar{e}_{\mathrm{T_{LNG}}} = 0.1\,\%$, and $\bar{e}_{\dot{m}_{\mathrm{RC}}} = 0.5\,\%$. The maximum error is $\bar{e}_{\mathrm{max}} = 1.4\,\%$.

The valve positions show smooth control with no persistent oscillations. When control is activated at $t = 18\,\mathrm{s}$, the controller quickly reduces the existing offset by moving BPV, FCV and TOV beyond their steady-state values for a short time. This causes a small oscillation in p_{cc}, which is quickly dampened.

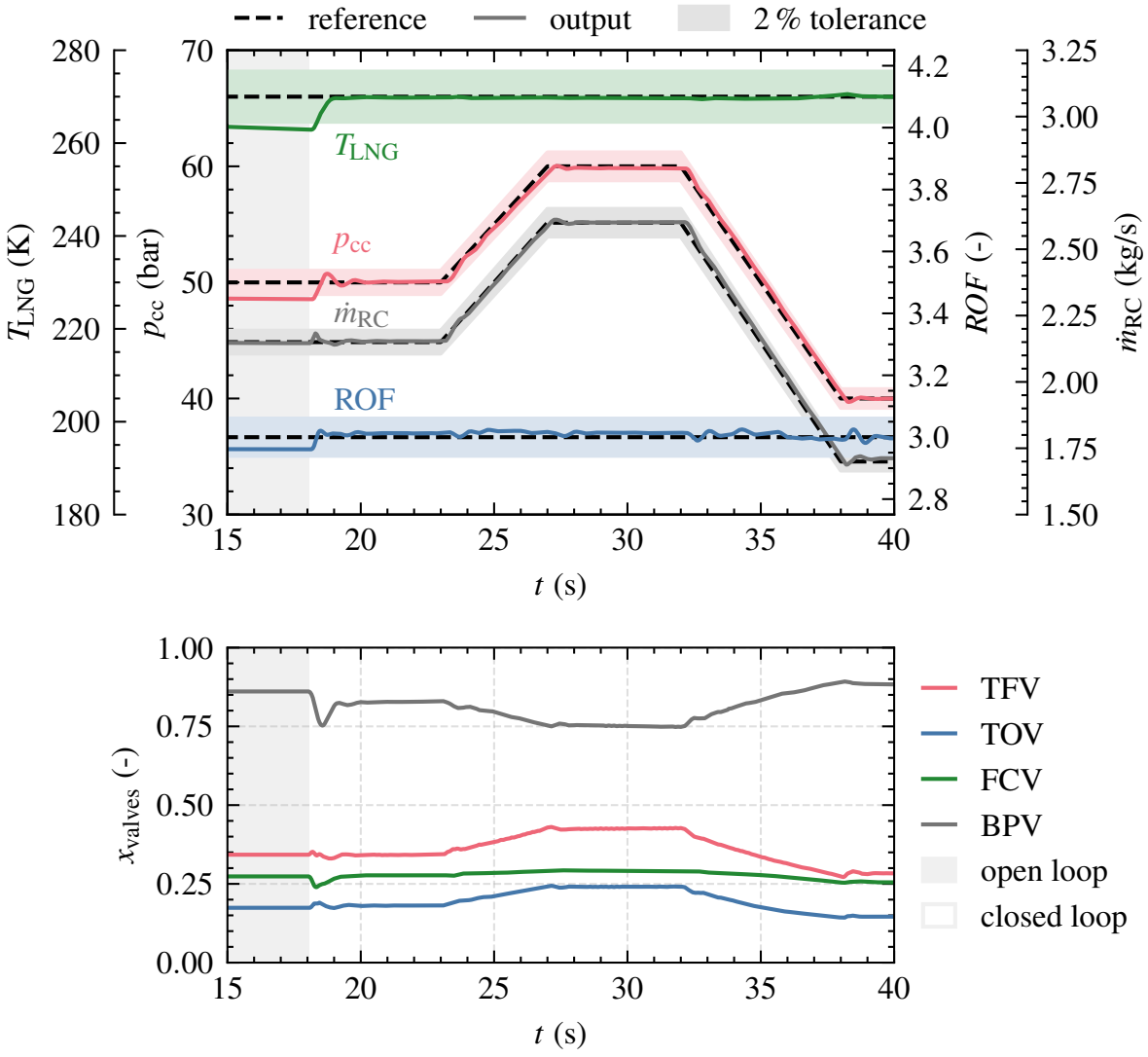


Figure 6.3.: Tracking results for the default set of model parameters

Operational envelope

The next step is to test the controller's ability to throttle the engine over the entire operating range. Multiple simulations are conducted with p_{cc} targets between 35 bar to 65 bar in 0.5 bar increments. Each simulation is run for 15 s to reach steady-state conditions, and then the steady-state offset is calculated by averaging the error over 1 s. Figure 6.4 shows the mean and maximum offset ($\bar{e}_{\varnothing}$, $\bar{e}_{\max}$) as a function of p_{cc}.

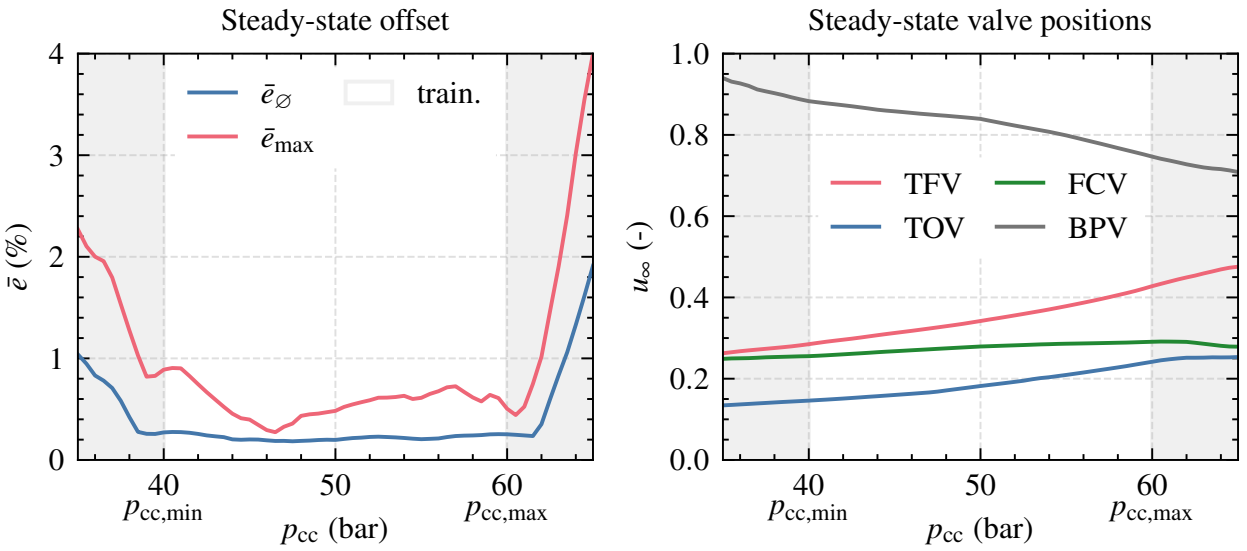


Figure 6.4.: Controller analysis over operational envelope

The control requirements specify throttling between 40 bar to 60 bar. Therefore, the same range was chosen for training. The controller achieves accurate control with small steady state offsets around $\bar{e}_{\text{steady}} = 0.2\,\%$ within this range. Outside the trained envelope, the errors start to increase. This is common in machine learning, where models are typically only applicable within the training domain and cannot reliably extrapolate beyond it.

The figure also shows the steady-state valve positions across the operating envelope. When the engine is throttled at a constant mixture ratio, the controller symmetrically opens the turbine valves (TOV and TFV). BPV is continuously closed to maintain a constant fuel injection temperature. FCV is slightly opened to regulate the mass flow in the cooling channels. The valve positions indicate a potential problem, especially for FCV and TOV: The valves are actuated within a very narrow band, not using the full capacity of the valve stroke. This is problematic because the controller requires very high accuracy in valve position. Even small errors of a few percent can cause significant control errors.

6.3.3. Robustness

A Monte Carlo analysis of 1000 simulations with randomized model parameters is performed to test the robustness of the controller. In each simulation, the system characteristics are varied by randomly changing model parameters according to Table A.3. Each parameter is sampled uniformly between its lower and upper bounds. The controller is compared to an open-loop scenario using the nominal valve commands of the controller from Figure 6.3.

Figure 6.5 summarizes the Monte Carlo analysis. Since there is no feedback in **open-loop** control, the system outputs change depending on the amount of parameter variation. Although most parameters are only varied by about 3 % to 5 % around their default values, large deviations occur due to the complex coupling between combustion, heat transfer, and turbopump performance in expander cycle engines. For p_{cc}, the maximum deviation is about ±5 bar or 10 %. Averaging over all simulations, model parameter variations result in an average tracking error of $\bar{e}_{\varnothing} = 4.2\,\%$ and $\bar{e}_{\max} = 11.9\,\%$ for open loop.

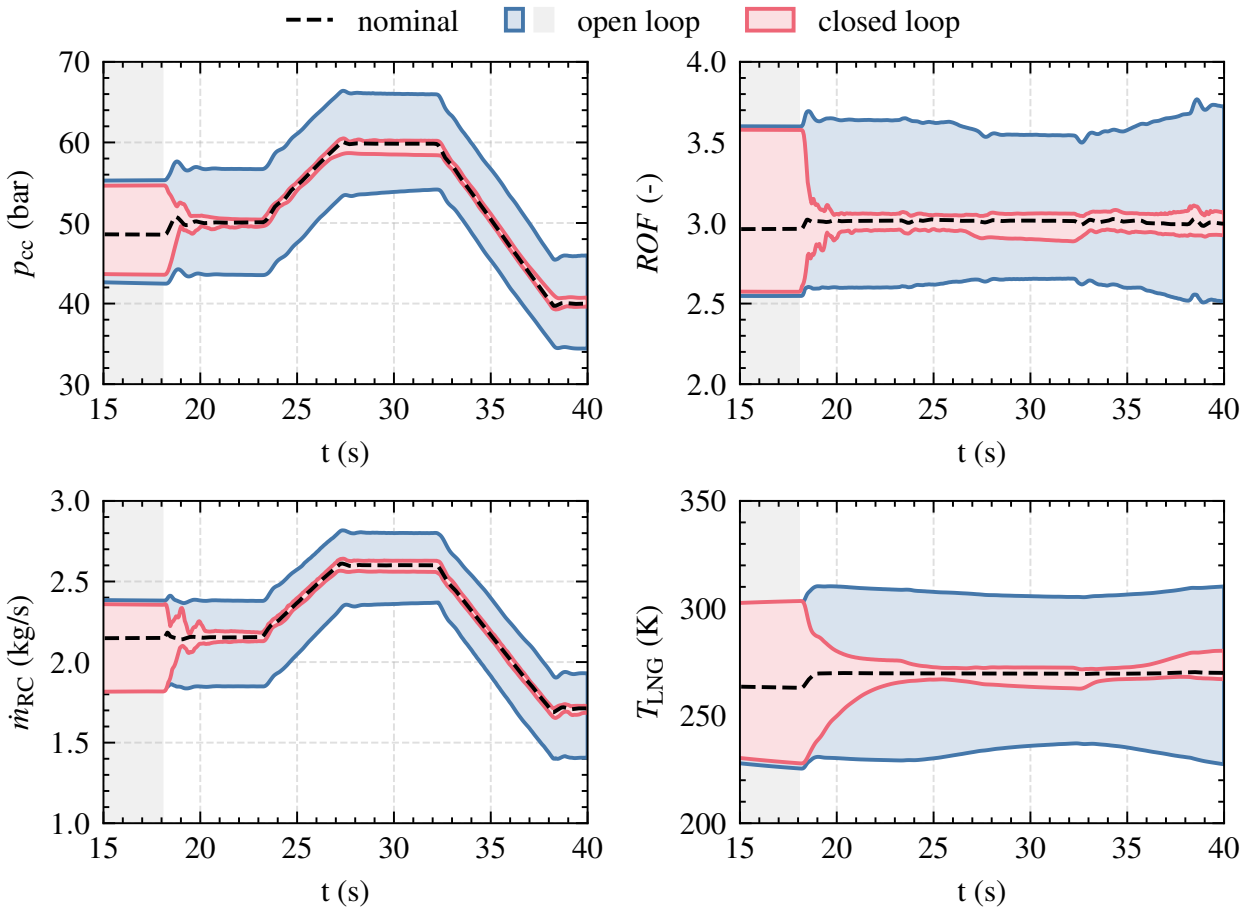


Figure 6.5.: Robustness analysis against modeling errors. Shown are the maximum deviations found in the Monte Carlo analysis. Control starts at $t = 18\,\mathrm{s}$.

The controller, on the other hand, has adapted its control policy to respond to the changing and unknown model parameters. It is therefore able to achieve a significantly lower tracking error with $\bar{e}_{\varnothing} = 0.5\,\%$ and $\bar{e}_{\max} = 2.0\,\%$. Although this is a significant improvement over open loop, the tracking errors are still larger than for the nominal parameter set. This suggests that there are certain parameter combinations that produce worse results than others. To further analyze this, the error distributions are visualized next.

Figure 6.6 compares the error distributions of $\bar{e}_{\varnothing}$ and $\bar{e}_{\max}$ between open-loop and closed-loop control. A large, wide-spread distribution indicates varying control performance depending on the amount of parameter changes. Certain combinations result in episodes with large errors. A narrow, sharp distribution with a low mean value represents a high robustness of the controller, resulting in low tracking errors even for large parameter variations. In **open-loop**, the error distribution is broad with a high mean. It shows that there are some parameter combinations that result in extremely large errors. **Closed-loop** control, on the other hand, results in a very narrow error distribution with no outliers.

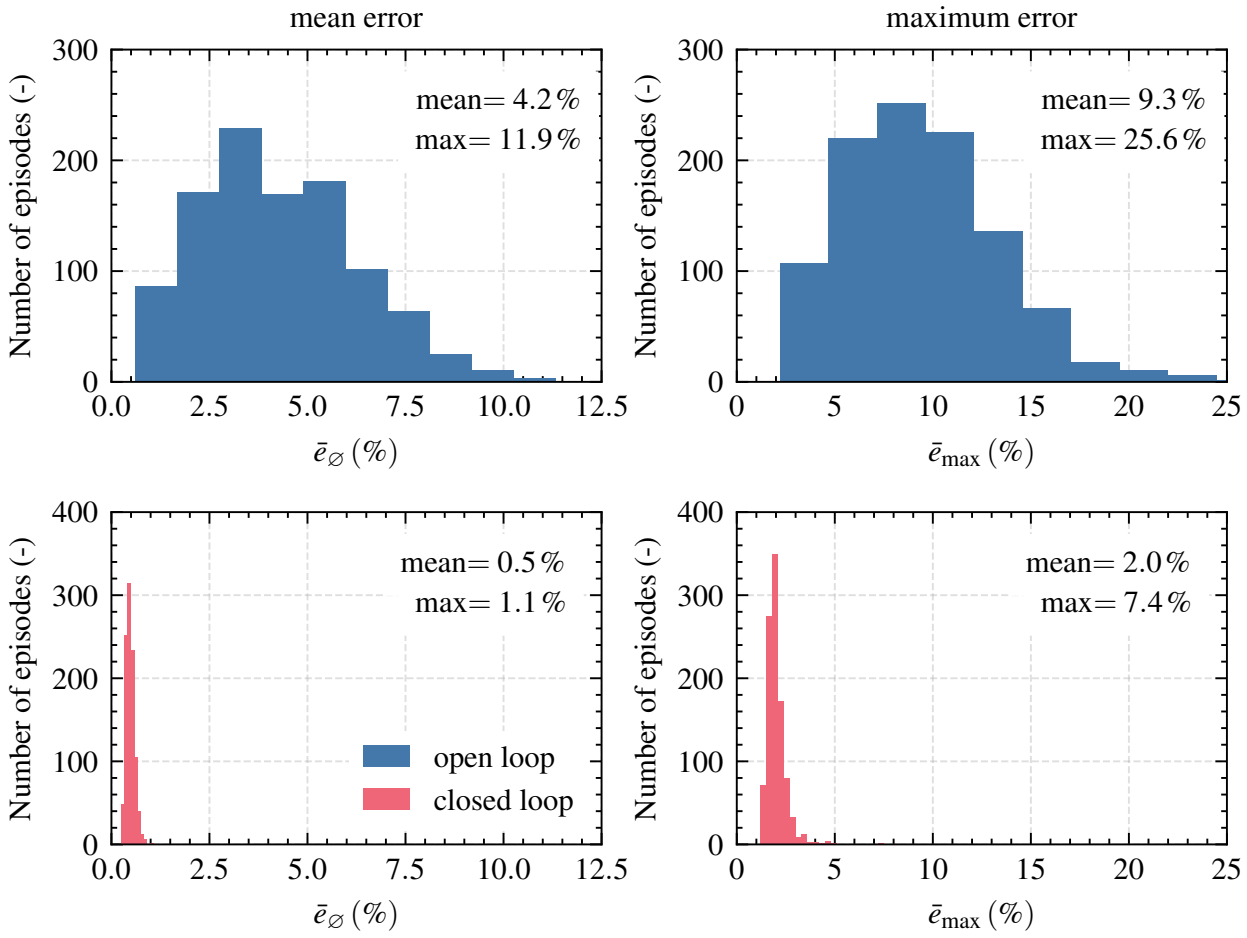


Figure 6.6.: Error distribution for modeling parameter variations

Sensor noise

Figure 6.7 shows the results for the test trajectory when the controller is confronted with Gaussian sensor noise. It is applied to p_{cc}, ROF, and $\dot{m}_{RC}$ with a standard deviation of $\sigma_{noise} = 1.5\,\%$. This is the maximum amount of noise used during training. Overall, the controller is still able to follow the reference trajectory and no large oscillations or instabilities are triggered.

The valve commands are slightly affected by sensor noise, causing a slight jitter in the valve positions. This can be measured by the total control input $\Delta u = \sum_{u \in U} \sum_{t=18}^{40} |u(t) - u(t-1)|$. Without noise, the controller uses a total control input of $\Delta u_{nominal} = 1.00$. For $\sigma_1 = 0.5\,\%$, Δu_{σ_1} increases to 1.94. For the highest noise levels, $\sigma_2 = 1.5\,\%$, Δu_{σ_2} increases to 3.74. When testing in the real world, the effect of sensor noise must be monitored. Possible countermeasures include smoothing the sensor data with an even larger moving average or filtering the command signals.

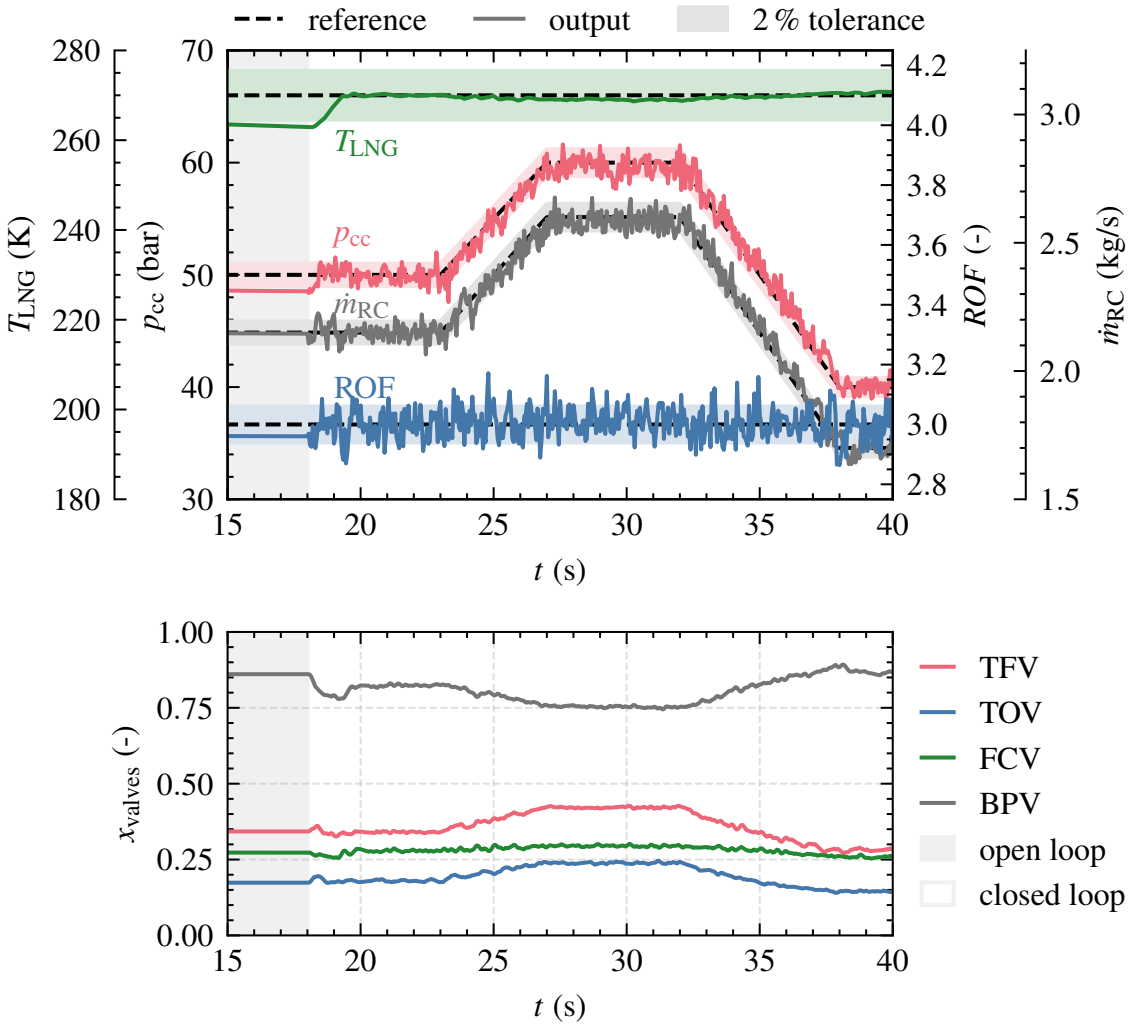


Figure 6.7.: Influence of sensor noise on the tracking performance

6.4. Real-world controller demonstration

Two experimental tests of the neural network controller have been conducted during the DEMO-24 campaign at the P8.3 test facility. This marks the first time at the test facility that closed-loop control is used for the simultaneous control of four variables. In the controller test run, the engine is first ignited and stabilized close to the first reference point in open-loop. Then, closed-loop control is activated. The neural network runs on a notebook directly in Python, exchanging sensor data and valve commands between the test bench and the controller via an Ethernet interface [173] using the User Datagram Protocol (UDP). This setup is more flexible than dedicated real-time hardware and allows the use of the test bench's safety measures, such as an emergency shut-down. In Python, the stacked observation vector is constructed, passed through the neural network, and the resulting control actions are returned to the test bench, which then actuates the control valves. The desired control frequency of 20 Hz was easily achieved with this setup.

In the first test, the controller was unstable, causing large oscillations in the system, as shown in Figure 6.8 and in more details in Figure A.1. After the test, it was discovered that the transient modeling of the flow control valves was erroneous. The response of all four valves to commands was significantly slower than modeled, especially when changing the direction of movement, as shown in Figure A.2. A possible reason for this modeling error is that the transient dynamics of the valves have been determined under ambient conditions with step functions only. When the system is pressurized and large mass flow rates pass through the valves, the valve dynamics might change.

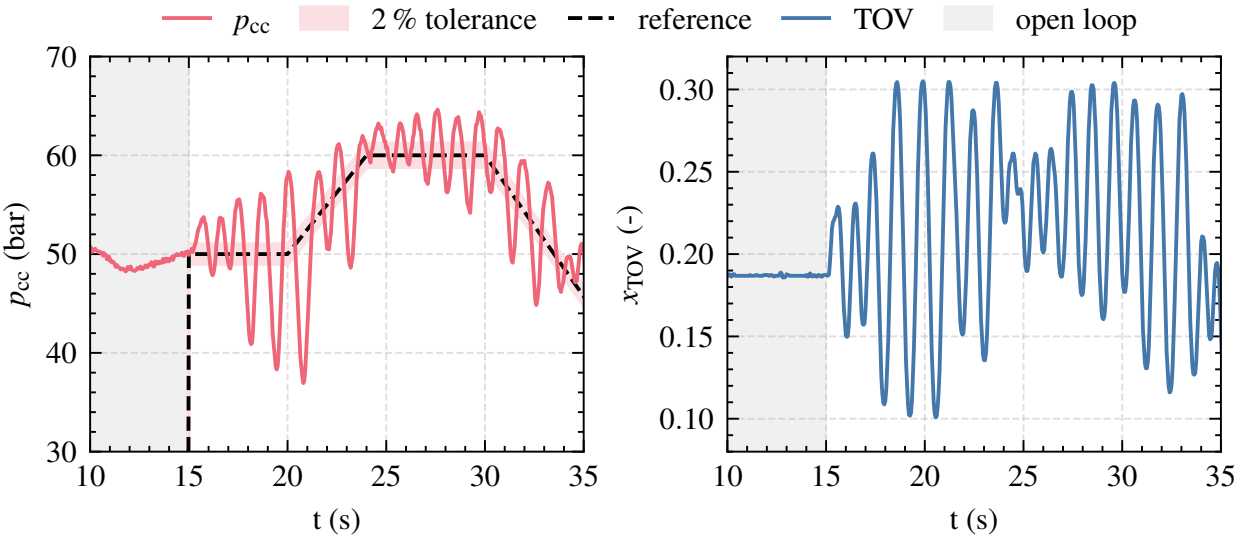


Figure 6.8.: Experimental results of test run 1 (p_{cc} and TOV)

After the first test, the valve modeling was corrected (see Figure A.3). Then, the neural network was retrained from scratch with the new simulation model. After only five days of modeling and training, a second test was conducted.

In the second test, the controller successfully managed to control the system and change the combustion chamber pressure, as shown in Figure 6.9. At $t = 18\,\mathrm{s}$, closed-loop control is activated and successfully maintained for 16.7 s. At $t = 34.7\,\mathrm{s}$, an automated test abort was triggered by a redline in the engine ignitor system, which is unrelated to the controller test.

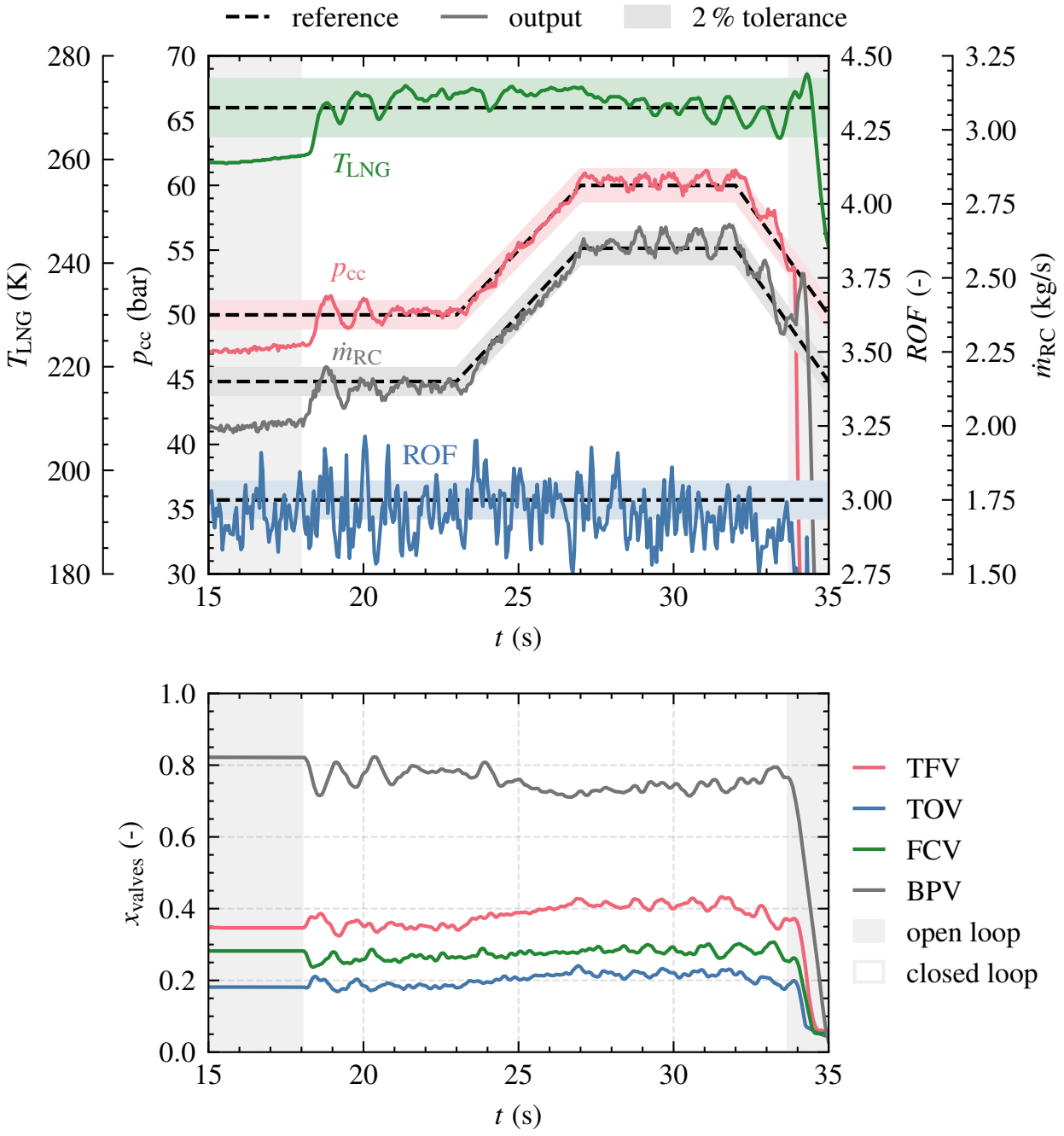


Figure 6.9.: Experimental results of test run 2

Overall, the neural network successfully controls p_{cc} and $\dot{m}_{RC}$ during ramps and steady-state phases while keeping *ROF* and T_{LNG} constant. The mean tracking error over the entire test is $\bar{e}_{\varnothing} = 1.3\,\%$. p_{cc} is controlled with the lowest mean error of 0.7 %. *ROF* has the largest mean error of 2.7 %. $\dot{m}_{RC}$ and T_{LNG} are controlled with mean errors of 1.1 % and 0.8 %, respectively. The maximum error during closed-loop control is 8.5 % for *ROF* due to large sensor noise. The controller manages to keep p_{cc}, $\dot{m}_{RC}$, and T_{LNG} within the 2 % tolerance band for most of the time. The mean *ROF* is slightly outside of the tolerance band at the end of the test, but it is still close to the desired value of 3.0.

When closed-loop control is activated at $t = 18\,s$, the system is outside of the desired reference tolerance band in all quantities. The controller successfully corrects these deviations but causes oscillations, which are damped within approximately 3 s. The controller shows good transient response in following the pressure ramp, starting at $t = 24\,s$. During the pressure ramp, no oscillations are visible. At the highest load point, oscillations of $\pm 2\,\%$ in all controlled variables occur with a frequency of 0.5 Hz.

Table 6.2 summarizes the control performances during the experimental tests and compares them to the simulation results. The control performance is drastically improved from the first to the second test by correcting the transient modeling of the flow control valves. With an average error of $\bar{e}_{\varnothing} = 1.3\,\%$, the second test run successfully demonstrates the feasibility of neural network-based rocket engine control. However, the controller performs slightly worse in the real world compared to the simulation results, despite the use of domain randomization and sensor noise during training. This discrepancy indicates that the LUMEN simulation model still lacks important real-world behaviors, that will be analyzed in the following section.

Table 6.2.: Control performance in simulation and experiment

		$\bar{e}_Y(\%)$				$\bar{e}_{\varnothing}(\%)$	$\bar{e}_{max}(\%)$
		p_{cc}	*ROF*	T_{LNG}	$\dot{m}_{RC}$		
Nominal	Mean	0.5	0.3	0.2	0.4	0.4	2.0
DR - CL	Mean	0.6	0.5	0.3	0.5	0.5	2.0
	Max	1.3	1.9	1.7	1.4	1.1	7.4
DR - OL	Mean	3.1	5.2	4.7	4.0	4.2	9.3
	Max	10.3	21.1	13.8	13.3	11.9	25.6
Experiment	Run 1	6.8	16.7	1.4	6.4	7.8	64.7
	Run 2	0.7	2.7	0.8	1.1	1.3	8.5

6.5. Analysis of the real-world demonstration

The first step of the analysis is to quantify the modeling errors of the EcosimPro model. Table 6.3 shows the mean and maximum modeling errors for the output variables and valve positions. The mean errors are smaller than 5 % for all output variables and smaller than 1 % for the valve positions. However, the maximum error is larger, with errors up to 10.6 %. In total, the modeling errors of the LUMEN model are larger than the individual errors of the subcomponent models as analyzed in Chapter 4.2. Possible reasons are that for the first time the turbines are driven by methane instead of nitrogen. The coupling between all components may also increase the modeling errors.

Table 6.3.: Modeling errors for the test run 2

		Outputs Y				Valves X			
		p_{cc}	ROF	T_{LNG}	$\dot{m}_{RC}$	TFV	TOV	FCV	BPV
Mean	(%)	4.1	2.4	3.9	4.9	0.4	0.8	0.5	0.3
Max	(%)	8.6	7.2	7.7	10.6	2.2	4.3	4.3	1.6

The next step is to test if it is possible to reproduce the experimental results in the simulation by applying domain randomization. In other words, it is tested if the experimental results are within the training domain of the neural network. If successful, this indicates that the agent should have encountered the real system characteristics during training. If the experimental data falls outside the training domain, this could explain why the controller performed slightly worse on the real system compared to the simulation. In such case, the amount of domain randomization could be increased, or even better, the overall accuracy modeling should be improved.

Figure 6.10 compares the time series of the experiment with simulation results generated by inputting the valve commands from the experiment into the simulation model at each time step. The simulation overestimates all four variables during the first half of the test. At the 60 bar load point, the modeling errors decrease in p_{cc} and $\dot{m}_{RC}$, although T_{LNG} is still slightly overestimated. In addition, throughout the entire test run the ROF measurement is significantly noisier in the experiment. The oscillations in T_{LNG} are also more pronounced in the experiment than in the simulation.

The same valve sequence is also simulated 1000 times with domain randomization. In each run, a random set of modeling parameters is sampled according to Table A.3. The shaded area represents the range of minimum and maximum

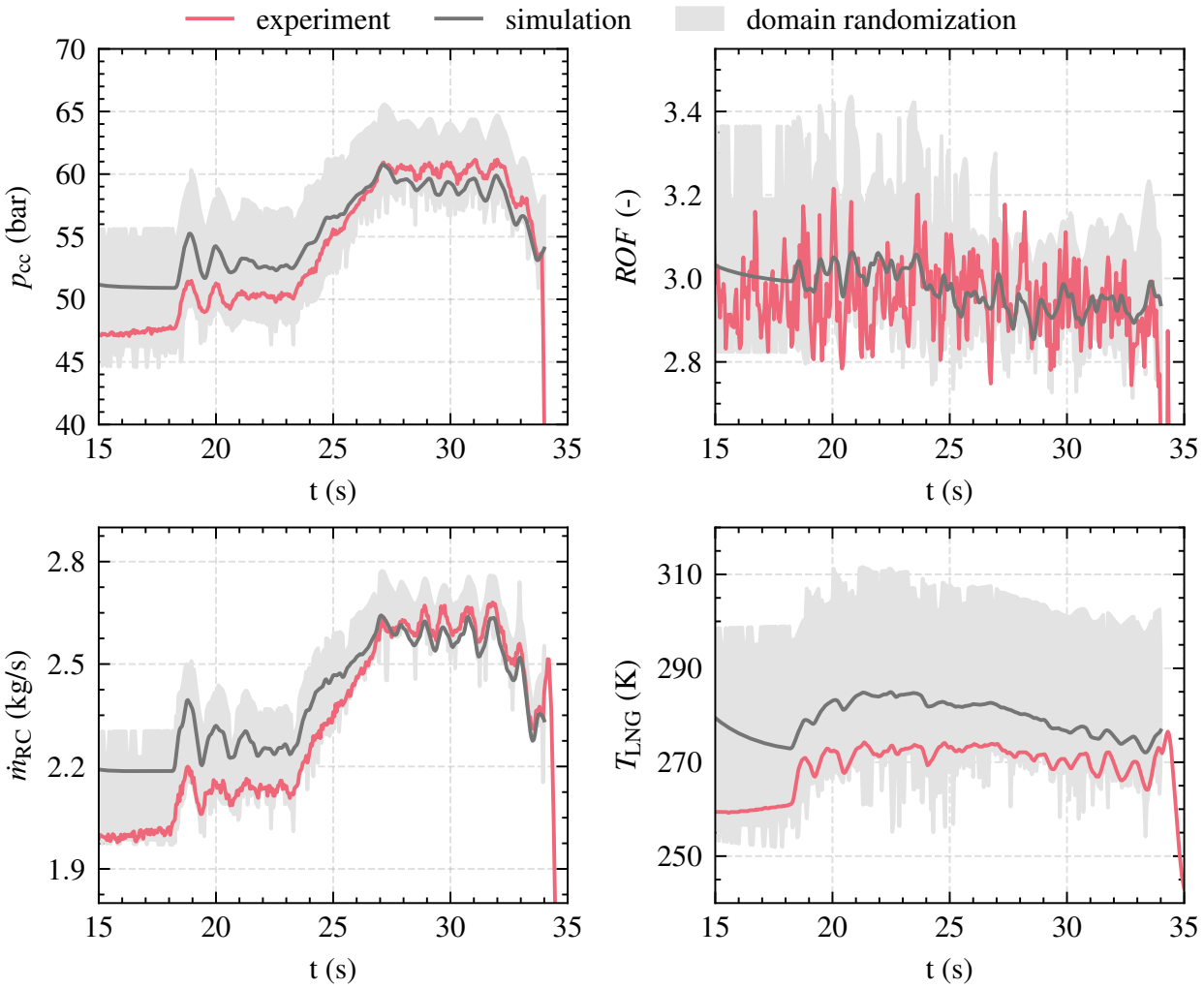


Figure 6.10.: Comparison between simulation and experiment for the valve commands used in the second test run

values obtained.[1] Overall, the experimental results for all four variables are very close to the lower limit of what can be achieved with domain randomization. This is potentially problematic because it means that the agent might have operated at the boundary of its training domain during the experimental test. This might have limited its performance, as we have already seen when examining the operational envelope in Section 6.3.2.

It is important to note that this analysis is only a first step in evaluating the simulation model. Even if the simulation model can roughly predict the system behavior, it is also necessary to study the transient response and analyze the prediction of successive time steps. This is necessary because the agent uses

[1] More specifically, at each time step, the geometric mean offset of all four variables is calculated. The upper and lower bounds of the shaded area are determined by the simulations that yield the most positive and negative geometric mean offset.

observation stacking, i.e. it uses multiple stacked observations at each time step to predict the next action. Thus, to fully replicate the experimental results, the simulation must accurately predict the values and derivatives of all system variables over multiple consecutive time steps. Additional remaining questions include whether the number of stacked observations needs to be improved, or whether other metrics such as derivatives of important quantities or moving averages should be included in the observation state to improve performance. It is also not clear why small oscillations of about 0.5 Hz are still present.

Sensor noise

In the next step, the assumption of a normally distributed sensor noise with a standard deviation of maximum $\sigma_{\text{noise}} = 1.5\,\%$ is examined. Figure 6.11 shows the distribution of the four system outputs measured at a steady state operating point over a period of 5 s. For better comparison, all data points are normalized to the measured average $\bar{Y}$ over this time period.

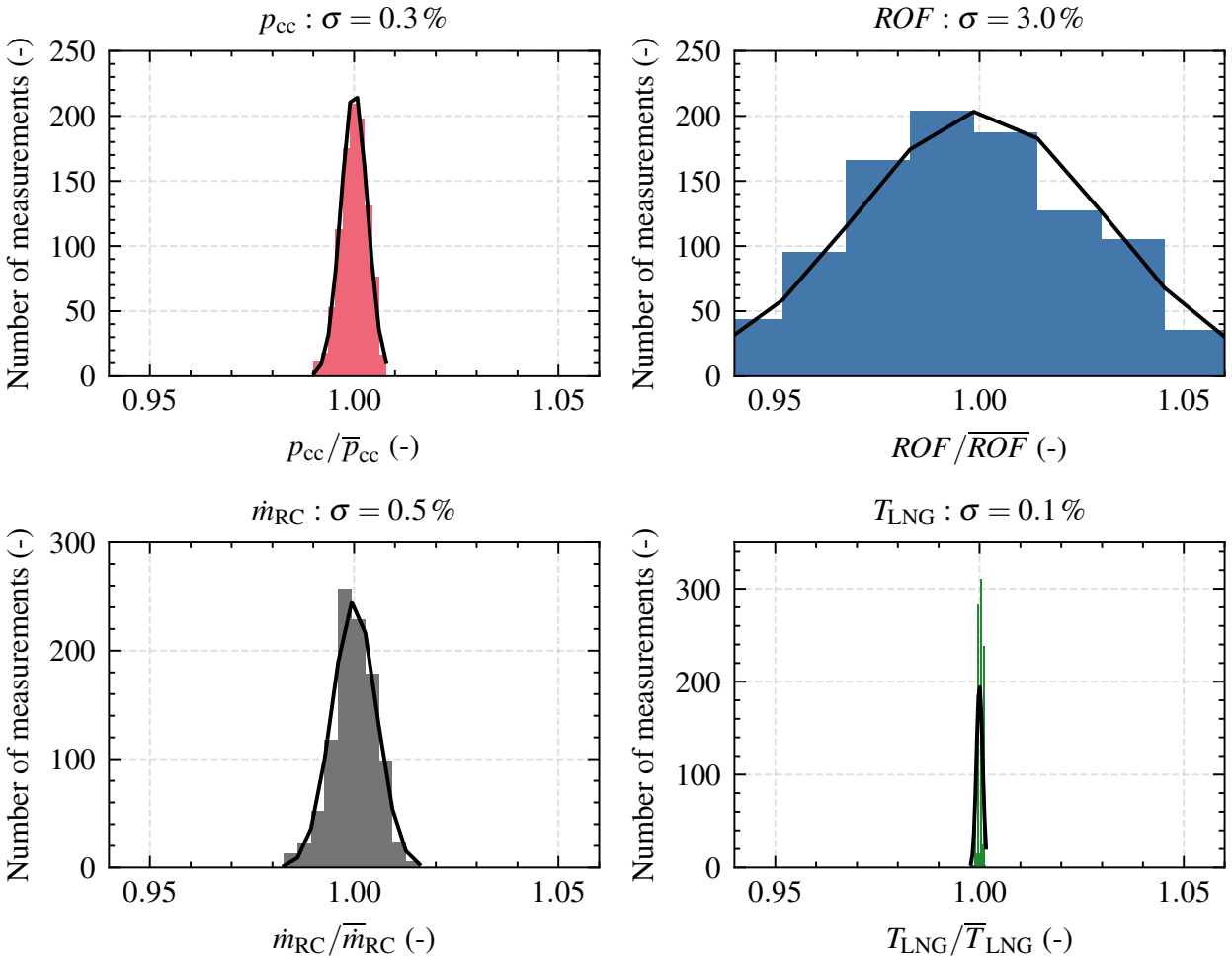


Figure 6.11.: Distribution of sensor noise from steady state

The noise level of the real sensors is comparable to the noise injected during training. The noise distribution follows a normal distribution with a standard deviation of that differs for each sensor. The noise in *ROF* is significant, with $\sigma_{\mathrm{ROF}} = 3.0\,\%$, which is larger than the maximum noise injected during training. The noise in p_{cc} and $\dot{m}_{\mathrm{RC}}$ is below the maximum noise observed during training. The noise in T_{LNG} is negligible.

Two potential problems with sensor noise are identified: First, the significant noise in *ROF* makes controlling this variable extremely difficult, since the noise has the same magnitude as the required 2 % tolerance band. Second, the neural network is trained with the assumption of the same noise level for each sensor. However, in the real world, the noise levels vary significantly depending on the sensor type. In the future, different noise levels should be considered during training depending on the sensor type. In addition, the measurement of *ROF* should be improved to reduce the noise amplitude.

6.6. Conclusion

In this test case, a neural network controller was trained with reinforcement learning for the continuous throttling of LUMEN. The controller is able to change the combustion chamber pressure, while at the same time regulate the mixture ratio, the fuel injection temperature and the cooling channel mass flow by operating four continuous flow control valves. The controller is trained only in simulation and its robustness is enhanced by applying domain randomization of key model parameters and sensor noise during training. The controller shows superior tracking performance in simulation and sufficient robustness to variations in model parameters and sensor noise.

The controller was also tested experimentally and successfully controlled the system for over 16 s. With an average error of 1.3 %, the controller achieved promising results for the first experimental application of neural network-based rocket engine control in the real world. This test shows the feasibility of reinforcement learning for rocket engine control and that it is feasible to only train in simulation and then use the controller directly without fine-tuning.

Despite the promising experimental results, the control performance is slightly worse than in simulation. A first analysis of the simulation model revealed several modeling errors with deviations between simulation and the real engine of up to 10.6 %. Even with domain randomization, the controller seems to barely encounter the real-world characteristics during training and thus operates at the limit of its training domain. The modeling errors and the high sensor noise may explain the slightly worse performance in the real world.

In summary, the following results have been achieved:

1. Simultaneous control of combustion chamber pressure, mixture ratio, cooling channel mass flow, and fuel injection temperature using four flow control valves with a control frequency of 20 Hz.
2. Experimental demonstration for over 16 s with an average error of 1.3 %.
3. Successful control even in the presence of modeling errors of up to 10.6 %, large measurement delays of up to 0.15 s, significant sensor noise with a standard deviation of up to 3.0 %, and a highly coupled system.
4. It has been shown that the controller can be quickly retrained in 5 days if the simulation model needs to be changed. This can be beneficial in engine development when components need to be changed during the test campaign. The controller could be quickly retrained and provide repeatable test results even if the characteristics of the new components are not well known prior to testing.

On the other hand, there are many improvements to be considered for the future to improve the dynamic and steady-state performance of the controller:

1. The EcosimPro modeling needs to be improved with data from the DEMO-24 test campaign.
2. The training procedure needs to be changed by considering distinct noise levels for each sensor.
3. The experimental setup can be improved: The noise in the *ROF* measurements is caused by a high noise level in the Coriolis flowmeter on the fuel side. It is currently unclear why this flow meter produces such a high noise level. One theory is that it is influenced by the flow mixer located directly upstream.
4. The valve flow characteristics should be adjusted so that the valves are actuated over a larger portion of their total stroke, rather than the very narrow range that makes them extremely sensitive.
5. The stability and robustness of the controller to disturbances needs to be tested and improved.

More general improvements, open research areas, and extensions and alternatives to the current setup are discussed in the outlook section of this thesis.

7. Summary

The ongoing shift towards reusability and cost efficiency is creating new challenges in the design and operation of liquid rocket engines. Future engines must be capable of continuous, fast throttling over a wide operating envelope and manage uncertain and changing system properties resulting from engine reuse and additive manufacturing. The transition to more complex engine cycles, such as staged combustion, further complicates operational requirements. Closed-loop control systems offer promising solutions by providing accurate thrust control over a wide range while optimizing engine efficiency. In the long term, these systems could also achieve more advanced control objectives such as fault tolerance, cluster-level control, and damage mitigation.

The objective of this thesis was to evaluate the suitability of reinforcement learning for rocket engine control and compare it to state-of-the-art model predictive control (MPC). A comprehensive literature review provided an in-depth overview of the fundamental challenges in rocket engine control, ranging from engine ignition to fuel-efficient thrust and mixture ratio control. A major challenge identified is the inherent coupling between thrust and mixture ratio control, which requires a complex multiple-input, multiple-output control approach. The review also found that advanced control concepts, such as fault tolerance and cluster control, were already recognized during the Space Shuttle era. However, practical implementation was limited by computational constraints and reliance on slow hydraulic valves. The current shift to more precise all-electric control valves, along with advances in sensor and computing technologies, promises to enable more sophisticated control systems in the near future.

Historically, rocket engine control have relied on proportional-integral (PI) based approaches using linear system models. While these methods are effective for thrust and mixture ratio control within a narrow operational envelope and for improving steady-state accuracy, they are not sufficient for the emerging challenges associated with reusability, propulsive landing, cluster control, and fault tolerance. The reason is that PI-based approaches use only the error term and cannot handle constraints, faults, nor include economic objectives such as maximizing engine efficiency. As these aspects become increasingly important, advanced concepts, such as MPC and RL, are promising alternatives.

All studies of this thesis revolved around the LUMEN expander-bleed engine developed by the Institute of Space Propulsion of the German Aerospace Center (DLR). Therefore, all design parameters and experimental data from several component test campaigns were available for modeling. Within the thesis, a fully transient simulation model was created in EcosimPro using the European Space Propulsion System Simulation (ESPSS) libraries. By relying on components from the well-proven ESPSS libraries and the validation with real test data, the model is a good representation of the characteristics of the real engine, with steady-state modeling errors around 5 %. However, due to modeling uncertainties, maximum transient errors of up to 11 % are present.

Two test cases of rocket engine control were investigated. The first test case is the fuel-efficient thrust and mixture ratio control with constraints. The control objective is to closely follow a predefined reference trajectory and minimize fuel consumption by operating the engine at peak efficiency. The second test case is the continuous throttling between a combustion chamber pressure of 40 bar to 60 bar. The controller is also tasked with controlling the mixture ratio, fuel injection temperature, and cooling channel mass flow rate using four control valves simultaneously. The trained controller was then experimentally tested.

The Soft Actor-Critic (SAC) algorithm was chosen for neural network training because of its sampling efficiency, minimal need for hyperparameter tuning, and inherent robustness due to its stochastic policy. A suitable set of hyperparameters was identified during preliminary work [5, 19, 43] and consistently applied in both test cases, demonstrating that once a suitable reinforcement learning setup is established, it can be adapted to different rocket engine control problems. To enhance the robustness of the controller, domain randomization and sensor noise were employed during the training process.

In summary, the thesis addressed the following research questions:

1. How accurately can a neural network control the transient thrust and mixture ratio of a representative liquid propellant rocket engine? How can the policy be made robust against modeling errors? Is it possible to include advanced control objectives, such as maximizing efficiency or minimizing engine damage, into the control policy?
2. What are the advantages and challenges of using reinforcement learning for rocket engine control compared to state-of-the-art MPC?
3. How does the neural network controller perform on a real rocket engine during experimental tests? Is it feasible to train the neural network only with a simulation model, which may contain modeling errors, and then deploy it directly on the test bench without further fine-tuning?

Research question 1: In both test cases, reinforcement learning demonstrated its potential to train neural networks for rocket engine control, with results comparable to or better than state-of-the-art offset-free MPC. In simulation, the neural network provided accurate transient control of combustion chamber pressure and mixture ratio with a mean error of less than 1.0 %. The controller showed robustness to model parameter variations and sensor noise, and handled constraints without violation. The controller also achieved near-optimal results in maximizing engine efficiency.

Advanced control objectives, such as reducing the damage rate to the combustion chamber wall, can be integrated into the reward function. If a damage model predicts the current damage rate as a function of the system state $\dot{u} = f(x, \dot{x})$, the reward function can be extended to minimize $\dot{u}$. Reinforcement learning could even be applied to more complex scenarios, such as controlling an entire engine cluster or managing faults, by incorporating these objectives into the reward function. The ability of reinforcement learning to use any nonlinear simulation model is beneficial for the space industry, where such models are often available for system analysis or test preparation. By training controllers with these existing resources, development time is reduced. Additionally, as shown by using the same setup in both test cases, reinforcement learning can be easily adapted to new challenges.

Research question 2: The comparison with a state-of-the-art MPC controller revealed similar control performance. The neural network has slightly better tracking performance due to a more aggressive control policy, resulting in faster settling times and lower overshoots. In addition, the neural network provides slightly better constraint handling. The advantage of the neural network is that it is trained with the full nonlinear simulation model, allowing it to learn all system dynamics. The offline nature of the training process also allows for extensive optimization. In contrast, the system model of MPC, constrained by online optimization, is limited in complexity. Consequently, the MPC controller is more conservative to mitigate constraint violations or overshoots that are caused by modeling inaccuracies.

On the other hand, the offset-free formulation of MPC directly compensates for modeling errors. The advantage is that modeling errors do not need to be explicitly accounted for in the controller design, as they are inherently corrected by the offset-free formulation. If model deviations can be quantified, robustness theorems can ensure that MPC remains robust under certain assumptions. In contrast, standard reinforcement learning does not inherently provide robustness to modeling errors. One idea is to improve robustness by learning from real data, or to fine-tune the policy by interacting directly with the real system. Overall, both approaches seem well suited to meet the challenges of future advanced rocket engine control.

Table 7.1 compares the benefits and challenges of reinforcement learning and MPC. Both rely on a cost function, encapsulating control requirements and allowing the integration of advanced goals such as fuel efficiency and damage mitigation. Both approaches can handle constraints, with reinforcement learning doing so implicitly and MPC explicitly. The flexibility of reinforcement learning to learn on any model eliminates the reliance of MPC on a computational efficient model, which may require model order reduction or linearization. Additionally, reinforcement learning has low online deployment costs, making it suitable for scenarios with limited computational resources. In contrast, MPC provides robustness guarantees under certain assumptions, which is a key advantage in safety-critical applications. However, the high online computational cost of MPC for real-time optimization may limit its feasibility on space hardware, but approximate MPC [89] could be a solution to reduce the computational costs. Furthermore, extending MPC to incorporate faults or complex nonlinear damage models presents additional challenges.

Table 7.1.: Comparison between MPC and RL

Reinforcement learning	Model predictive control
∘ continuous inputs and outputs	∘ continuous inputs and outputs
∘ explicit policy (neural network)	∘ implicit policy (optimization)
∘ implicit constraints (reward)	∘ explicit constraints (optimization)
+ utilization of existing models	− (linearized) mathematical model
+ low online costs	− high costs (online optimization)
+ easy to adapt to similar problems	− high effort for new problems
− robustness	+ predictable behavior, robustness
− novel approach	+ state-of-the art, many extensions

Research question 3: The successful experimental demonstration of closed-loop control during a 16 second hot-fire test showed the feasibility of reinforcement learning for rocket engine control. The experiment demonstrated that it is possible to train the controller solely in simulation and apply it directly to the actual engine. The controller achieved promising control accuracy with a mean absolute percentage error of 1.3 %, despite modeling errors of up to 10.6 % and significant sensor noise with a standard deviation of up to 3.0 %.

However, the controller performance in the real-world test was slightly worse compared to the simulation. Analysis indicated that the controller was operating at the edge of its training domain due to modeling errors and possibly insufficient domain randomization. As is typical for most machine learning methods, reinforcement learning generally performs best within its training domain, which likely explains the slight decrease in control performance during

the actual test. This motivates the need for more robust and safer extensions to standard reinforcement learning on the way to using such controllers on flight engines, where stability and robustness are of utmost importance.

In terms of computational requirements, the controller easily met the required control frequency of 20 Hz, implemented in Python without any optimizations, running on a `Intel i7-11850H` notebook processor. Further studies showed that an embedded processor with computational power similar to space grade processors could easily achieve a control frequency of 35 Hz to 50 Hz for the neural network size used in this work (2 layers with 256 neurons) without additional tuning [19]. With further tuning and code optimization, control frequencies of up to 1000 Hz have been achieved with neural networks of similar size [41], suggesting that neural network-based controllers are well suited for the limited computational resources available in space applications.

In summary, this thesis demonstrates that reinforcement learning can effectively train neural networks for rocket engine control, providing accurate transient control of combustion chamber pressure and mixture ratio with performance slightly better than state-of-the-art MPC. The neural network-based controller achieved robust results, handling constraints, sensor noise, and parameter variations while maximizing engine efficiency. The experimental tests confirmed the feasibility of using a controller trained in simulation only directly on a real rocket engine, although performance slightly decreased due to modeling errors. As an outlook, further research is needed to guarantee the safety and robustness of reinforcement learning. This is essential to make such methods viable for future flight engines, where stability and reliability are critical.

7.1. Outlook

Stability, robustness, and safety in reinforcement learning are currently heavily researched topics [68, 127, 141]. As summarized by Osinenko et al. [141], performance and stability proofs are crucial for the acceptance of reinforcement learning in real-world applications, especially in safety-critical domains. Therefore, this work should be extended with new methods to increase robustness and stability. Three promising directions for future research are:

1. Improve system modeling and derive modeling uncertainty estimates.
2. Use more advanced concepts of end-to-end reinforcement learning, such as safe and robust reinforcement learning, to derive performance and safety guarantees under certain assumptions.
3. Investigate hybrid approaches that combine reinforcement learning and classical control methods.

System modeling

Creating a perfect simulation model for liquid rocket engines is not feasible, unlike scenarios such as computer games or board games where the agent learns from a fixed environment with known rules and dynamics. Modeling errors are inevitable due to the complex nonlinear physics involved in combustion processes, heat transfer, and turbopump performance. However, rocket engines undergo extensive ground testing, and once a system is qualified and successfully put into service, it rarely undergoes significant changes. This stability provides an opportunity to refine the simulation model to the point where modeling errors are minimal across the operational envelope. Improving the accuracy of the simulation model can greatly improve the training process by reducing the need for extensive domain randomization. Providing measures of uncertainty and quantifying the modeling errors will help test the controller in simulation and ensure that it will perform reliably when transitioned to real-world applications.

Advanced concepts of end-to-end reinforcement learning

More advanced concepts of end-to-end reinforcement learning should be explored to further increase robustness and safety beyond domain randomization and sensor noise. The long-term goal is to find guarantees that constraints and unsafe states are never violated if the real system behaves similarly to the simulation. The goal is to provide guarantees such as, "If all model parameters of the real system are within $\pm 5\,\%$ of the assumed values, the controller will never violate any safety constraints during operation." Robust and safe reinforcement learning are two promising extensions:

Robust reinforcement learning [127] addresses the shortcomings of classical reinforcement learning in dealing with uncertainties, modeling errors, parameter changes, and structural changes in the environment. The research is driven by the increasing application of reinforcement learning to real-world problems, where the performance of classical reinforcement learning often degrades significantly compared to simulation. One idea of robust reinforcement learning is to increase robustness by confronting the agent with more advanced scenarios during the learning process itself.

One approach is to introduce a destabilizing adversary into the training process. The adversary acts as an opponent to the agent, with the task of degrading the agent's performance by changing specific parts of the simulation. Meanwhile, the agent must learn a robust policy that cannot be exploited by the adversary. The adversary is trained in parallel with an adversarial reward function ($r_{\text{adversary}} = -r_{\text{agent}}$) and can alter the system in up to four different

settings: (1) transition robustness, (2) perturbation robustness, (3) action robustness, and (4) observation robustness.[1]

Transition robust methods focus on the transition probability model of the Markov decision process $s_{t+1} \sim P(\cdot|s_t, a_t)$. These methods define an uncertainty set of possible transition functions $P \in U_P$. At each time step, the adversary takes control of P and tries to find the worst possible transition function for each state-action pair. In disturbance robust methods, modeling errors and parameter changes are treated as external perturbations. Here, the adversary can apply external forces $a_{\mathrm{t}} = f(s_{\mathrm{t}}, a_{\mathrm{t}})$ at each time step to destabilize the agent. These external forces may differ from the agent's action space. For example, if the landing burn of a reusable first stage is controlled by thrust and gimbal angle, the adversary could introduce wind gusts to disrupt the landing maneuver. Action robust methods involve perturbing the actions of the agent itself. Observation robust methods alter the observations the agent receives for decision making.

Safe reinforcement learning: Ensuring safe system operation during learning and deployment is a key goal of safe reinforcement learning [68, 141]. According to Hans et al. [79], an agent is considered safe if it never reaches unsafe states or violates constraints. Due to the nonlinear nature of neural networks and most physical systems, finding such general guarantees for deep reinforcement learning is often extremely challenging. For real-world applications, transferring guarantees beyond simulation to the real system with partially unknown dynamics is one of the most important areas of current research, for example in emerging areas such as autonomous driving [107].

One approach is to learn a supervisor, or reactive shield, that blocks and replaces unsafe actions with safe alternatives at each time step [9, 100, 138]. An alternative from control theory is to use control Lyapunov functions [10, 34] or to learn an additional safety critic that estimates if the agent is operating in a safe manner [78]. Another idea is to introduce constraints on the policy itself, for example by limiting the partial derivatives of the output with respect to the inputs [102], and thereby learn a smoother and safer control policy.

Hybrid approaches

Hybrid approaches that combine reinforcement learning with classical control methods are an interesting alternative to pure end-to-end reinforcement learning. This combination aims to improve performance beyond what classical control methods can achieve, while increasing robustness compared to pure reinforcement learning [22]. Three approaches have been identified:

[1] Figures 5, 6, 7, and 8 of Moos et al. [127] provide excellent illustrations of these concepts

Sequential integration: Reinforcement learning can be used offline before the test or mission to find optimal trajectories. The agent can use the full nonlinear dynamic model to find an optimal trajectory that maximizes high-level objectives, such as maximizing fuel utilization or minimizing engine damage. The resulting trajectory can then be evaluated by human experts. During the flight, the optimized trajectory can be implemented either in open loop or by using classical control methods to keep the system close to the optimized trajectory. This approach was studied for the LOX tank pressure control of the P5 test facility and showed promising results [46].

Parallel integration: Another approach is to use classical control methods and reinforcement learning in parallel, combining their outputs to take advantage of the strengths of both techniques. For example, reinforcement learning can be used to fine-tune the output of a classical PID controller, represented as $a = a_{\text{PID}} + a_{\text{NN}}$. Wu et al. [220] demonstrated that this combined approach can significantly improve transient performance, while also reducing the training time required for reinforcement learning. By limiting the influence of the neural network (for example by ensuring that $|a_{\text{NN}}| \ll |a_{\text{PID}}|$), it becomes easier to provide robustness and stability guarantees compared to standard end-to-end reinforcement learning. From a control theory perspective, the adjustments made by the neural network could be treated as action noise, which allows the use of standard robust methods to proof system stability.

Supervisory control: Reinforcement learning could be used as a supervisory controller that dynamically adjusts PID or MPC parameters in real time to adapt to changing conditions or operating points. Carlucho et al. [29] studied reinforcement learning for PID parameter tuning in a multiple-input multiple-output setting for the control of mobile robots. Bhan et al. [22] extended this idea to fault-tolerant control. The authors propose to estimate fault-related parameters, such as component degradation, and use deep reinforcement learning to adjust PID parameters depending on the fault.

In summary, hybrid approaches offer promising alternatives to pure end-to-end reinforcement learning and may serve as a viable first step in improving current state-of-the-art closed-loop controllers for liquid propellant rocket engines. By combining reinforcement learning with proven techniques such as PI controllers, these hybrid methods may increase industry acceptance. Future studies could explore the effectiveness of such combined controllers, with the possibility of testing them during the next LUMEN test campaign.

Bibliography

[1] M. Abadi, A. Agarwal, P. Barham, et al.: TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. 2016. arXiv: 1603.04467 [cs].

[2] A. Abbaspour, S. Mokhtari, A. Sargolzaei, et al.: A Survey on Active Fault-Tolerant Control Systems. *Electronics*, 9(9), 2020. doi: 10.3390/electronics9091513

[3] A. Abdolmaleki, J. T. Springenberg, J. Degrave, et al.: Relative Entropy Regularized Policy Iteration. 2018. arXiv: 1812.02256 [cs, stat].

[4] M. Adachi, T. Tamura, T. Onga, et al.: Progress Summary of Engineering Model Firing Tests in LE-9 Engine Development. In: *Proceedings of the 68th International Astronautical Congress (IAC)*. Adelaide, Australia, 2017.

[5] A. Adler: Design and Robustness Analysis of Neural Network Based Control of Rocket Engine Turbopumps. Master's Thesis. Germany: JMU Würzburg, 2022.

[6] Airbus Defence and Space: Electronics on Ariane 6 and Vega-C. 2023. url: https://www.airbus.com/en/products-services/space/equipment/launcher-products/launcher-electronics (visited on 08/14/2023).

[7] P. Albertos, A. Sala, and M. Chadli: Multivariable Control Systems - an Engineering Approach. *Automatica*, 41(9), 2005, pp. 1665–1666. doi: 10.1016/j.automatica.2005.04.003

[8] F. Allgöwer and A. Zheng, eds.: Nonlinear Model Predictive Control. Basel: Birkhäuser, 2000. doi: 10.1007/978-3-0348-8407-5

[9] M. Alshiekh, R. Bloem, R. Ehlers, et al.: Safe Reinforcement Learning via Shielding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. doi: 10.1609/aaai.v32i1.11797

[10] A. Anand, K. Seel, V. Gjærum, et al.: Safe Learning for Control Using Control Lyapunov Functions and Control Barrier Functions: A Review. *Procedia Computer Science*, 192, 2021, pp. 3987–3997. doi: 10.1016/j.procs.2021.09.173

[11] J. A. E. Andersson, J. Gillis, G. Horn, et al.: CasADi: A Software Framework for Nonlinear Optimization and Optimal Control. *Mathematical Programming Computation*, 11(1), 2019, pp. 1–36. doi: 10.1007/s12532-018-0139-4

[12] M. Arps: System Identification for Control Applications of Pump Fed Rocket Engines. Bachelor's Thesis. Germany: JMU Würzburg, 2021.

[13] K. Arulkumaran, M. P. Deisenroth, M. Brundage, et al.: Deep Reinforcement Learning: A Brief Survey. *IEEE Signal Processing Magazine*, 34(6), 2017, pp. 26–38. doi: 10.1109/MSP.2017.2743240

[14] H. Asakawa, H. Nanri, I. Masuda, et al.: Study on Combustion Characteristics of LOX/LNG (Methane) Co-axial Type Injector under High Pressure Condition. In: *52nd AIAA/SAE/ASEE Joint Propulsion Conference*. Salt Lake City, UT, 2016. doi: 10.2514/6.2016-5078

[15] K. J. Åström and R. M. Murray: Feedback Systems: An Introduction for Scientists and Engineers. Princeton University Press, 2008.

[16] J. Bellows, R. Brewster, and E. Bekir: OTV Liquid Rocket Engine Control and Health Monitoring. In: *20th Joint Propulsion Conference*. Cincinnati, OH: AIAA, 1984. doi: 10.2514/6.1984-1286

[17] J. W. Bennewitz, D. M. Lineberry, and R. A. Frederick: Investigation of a Single Injector with Applied High Frequency Pressure Disturbances For Applications To Liquid Rocket Engine Combustion Instabilities. In: *49th AIAA/ASME/SAE/ASEE Joint Propulsion Conference*. San Jose, CA: AIAA, 2013. doi: 10.2514/6.2013-3853

[18] H. Benninghoff, K. Borchers, A. Börner, et al.: OBC-NG Concept and Implementation. Tech. rep. DLR, 2016.

[19] P. Berger: Deployment of Rocket Engine Controllers Based on Neural Networks. Master's Thesis. Germany: JMU Würzburg, 2023.

[20] D. P. Bertsekas: Reinforcement Learning and Optimal Control. Belmont, MA: Athena Scientific, 2019.

[21] E. Betts and R. Frederick: A Historical Systems Study of Liquid Rocket Engine Throttling Capabilities. In: *46th AIAA/ASME/SAE/ASEE Joint Propulsion Conference & Exhibit*. Nashville, TN, 2010. doi: 10.2514/6.2010-6541

[22] L. Bhan, M. Quinones-Grueiro, and G. Biswas: Fault Tolerant Control Combining Reinforcement Learning and Model-based Control. In: *2021 5th International Conference on Control and Fault-Tolerant Systems (SysTol)*. Saint-Raphael, France: IEEE, 2021, pp. 31–36. doi: 10.1109/SysTol52990.2021.9595275

[23] R. E. Biggs: Space Shuttle Main Engine, The First Ten Years. In: *History of Liquid Rocket Engine Development in the United States, 1955-1980*. Ed. by S. E. Doyle. Vol. 13. American Astronautical Society, 1992, pp. 69–122.

[24] L. Blackmore: Autonomous Precision Landing of Space Rockets. *National Academy of Engineering*. The Bridge on Frontiers of Engineering, 4(46), 2021, pp. 144–146.

[25] M. Blanke, W. Christian Frei, F. Kraus, et al.: What Is Fault-Tolerant Control? *IFAC Proceedings Volumes*, 33(11), 2000, pp. 41–52. doi: 10.1016/S1474-6670(17)37338-X

[26] M. Börner, J. Martin, J. Hardi, et al.: LUMEN Thrust Chamber: Flame Anchoring For Shear Coaxial Injectors. In: *Space Propulsion Conference*. Virtual: 3AF, 2022.

[27] G. Brockman, V. Cheung, L. Pettersson, et al.: OpenAI Gym. 2016. arXiv: 1606.01540 [cs].

[28] F. C. Bruhn, N. Tsog, F. Kunkel, et al.: Enabling Radiation Tolerant Heterogeneous GPU-based Onboard Data Processing in Space. *CEAS Space Journal*, 12(4), 2020, pp. 551–564. doi: 10.1007/s12567-020-00321-9

[29] I. Carlucho, M. De Paula, and G. G. Acosta: An Adaptive Deep Reinforcement Learning Approach for MIMO PID Control of Mobile Robots. *ISA Transactions*, 102, 2020, pp. 280–294. doi: 10.1016/j.isatra.2020.02.017

[30] M. J. Casiano, J. R. Hulka, and V. Yang: Liquid-Propellant Rocket Engine Throttling: A Comprehensive Review. *Journal of Propulsion and Power*, 26(5), 2010, pp. 897–923. doi: 10.2514/1.49791

[31] E. Chang and K. Kailasanath: Active Control of Combustion Instabilities Using Timed Injection of High Energy Fuels. In: *34th AIAA/ASME/SAE/ASEE Joint Propulsion Conference and Exhibit*. Cleveland, OH: AIAA, 1998. doi: 10.2514/6.1998-3765

[32] M. Chelouati, M. S. Jha, M. Galeotta, et al.: Remaining Useful Life Prediction for Liquid Propulsion Rocket Engine Combustion Chamber. In: *5th International Conference on Control and Fault-Tolerant Systems (SysTol)*. Saint-Raphael, France: IEEE, 2021, pp. 225–230. doi: 10.1109/SysTol52990.2021.9595286

[33] M. Chiappetta: AMD Ryzen Review: Ryzen 7 1800X, 1700X, and 1700 - Zen Brings the Fight Back to Intel. 2021. url: https://hothardware.com/reviews/intel-12th-gen-core-alder-lake-cpu-review (visited on 08/14/2023).

[34] J. Choi, F. Castañeda, C. J. Tomlin, et al.: Reinforcement Learning for Safety-Critical Control under Model Uncertainty, Using Control Lyapunov Functions and Control Barrier Functions, 2020. arXiv: 2004.07584.

[35] H. Cikanek Iii: Characteristics of Space Shuttle Main Engine Failures. In: *23rd Joint Propulsion Conference*. San Diego,CA: AIAA, 1987. doi: 10.2514/6.1987-1939

[36] S. Colas, S. L. Gonidec, P. Saunois, et al.: A Point of View about the Control of a Reusable Engine Cluster. In: *8th European Conference for Aeronautics and Space Sciences (EUCASS)*. Madrid, Spain, 2019. doi: 10.13009/EUCASS2019-234

[37] P. Concio, S. D'Alessandro, M. Aranda-Rosales, et al.: Low-Order Modelling and Validation of Film Cooling in Liquid Rocket Combustion Chambers. In: *Space Propulsion Conference*. Estoril, Portugal: 3AF, 2022.

[38] J.-P. Corriou: Process Control: Theory and Applications. 2nd ed. Cham, Switzerland: Springer Nature, 2018. doi: 10.1007/978-3-319-61143-3

[39] J. Dauer, K. Dresia, and G. Waxenegger-Wilfing: Safe and Robust Reinforcment Learning: How Do You Prevent Catastrophic Behavior of a Neural Network Controller for Rocket Engines. In: *4th Workshop on Advances in Artificial Intelligence for Aerospace Engineering*. Braunschweig, Germany, 2024.

[40] J. Deeken and G. Waxenegger: LUMEN: Engine Cycle Analysis of an Expander-Bleed Demonstrator Engine for Test Bench Operation. In: *Deutscher Luft- Und Raumfahrtkongress*. Braunschweig, Germany, 2016.

[41] J. Degrave, F. Felici, J. Buchli, et al.: Magnetic Control of Tokamak Plasmas through Deep Reinforcement Learning. *Nature*, 602(7897), 2022, pp. 414–419. doi: 10.1038/s41586-021-04301-9

[42] R. C. Dorf and R. H. Bishop: Modern Control Systems. 13th ed. Hoboken: Pearson, 2016.

[43] K. Dresia, A. Adler, A. Saraf, et al.: Rocket Engine Control with Neural Networks: Experimental Results of the LUMEN Turbopump Test Campaign. In: *AIAA Scitech Forum*. National Harbor, MD, 2023. doi: 10.2514/6.2023-0137

[44] K. Dresia, M. Boerner, W. Armbruster, et al.: Design and Control Challenges for the LUMEN LOX/LNG Expander-Bleed Rocket Engine. In: *34th International Symposium on Space Technology and Science (ISTS)*. Fukuoka, Japan, 2023.

[45] K. Dresia, E. Kurudzija, J. Deeken, et al.: Improved Wall Temperature Prediction for the LUMEN Rocket Combustion Chamber with Neural Networks. *Aerospace*, 10(5), 2023. DOI: 10.3390/aerospace10050450

[46] K. Dresia, E. Kurudzija, G. Waxenegger-Wilfing, et al.: Automation of Testing and Fault Detection for Rocket Engine Test Facilities with Machine Learning. *International Journal of Energetic Materials and Chemical Propulsion*, 22(6), 2023, pp. 17–33. DOI: 10.1615/IntJEnergeticMaterialsChemProp.2023047195

[47] K. Dresia, T. Traudt, T. Hörger, et al.: Overview of Future Rocket Engine Control Systems. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[48] K. Dresia, G. Waxenegger-Wilfing, Robson H. Dos Santos Hahn, et al.: Nonlinear Control of an Expander-Bleed Rocket Engine Using Reinforcement Learning. In: *Space Propulsion Conference*. Virtual: 3AF, 2021.

[49] G. Dressler: Summary of Deep Throttling Rocket Engines with Emphasis on Apollo LMDE. In: *42nd AIAA/ASME/SAE/ASEE Joint Propulsion Conference & Exhibit*. Sacramento, CA, 2006. DOI: 10.2514/6.2006-5220

[50] G. Dulac-Arnold, N. Levine, D. J. Mankowitz, et al.: An Empirical Investigation of the Challenges of Real-World Reinforcement Learning, 2020. arXiv: 2003.11881.

[51] K. Dunlap, M. Mote, K. Delsing, et al.: Run Time Assured Reinforcement Learning for Safe Satellite Docking. *Journal of Aerospace Information Systems*, 20(1), 2023, pp. 25–36. DOI: 10.2514/1.I011126

[52] K. Einicke: Mixture Ratio and Combustion Chamber Pressure Control of an Expander-Bleed Rocket Engine with Reinforcement Learning. Diploma Thesis. TU Dresden, 2021.

[53] Emerson Electric Company: Control Valve Handbook. Tech. rep. D101881X012. Marshalltown, IA, 2023.

[54] D. Ernst, M. Glavic, F. Capitanescu, et al.: Reinforcement Learning Versus Model Predictive Control: A Comparison on a Power System Problem. *IEEE Transactions on Systems, Man, and Cybernetics*, 39(2), 2009, pp. 517–529. DOI: 10.1109/TSMCB.2008.2007630

[55] ESA and Empresarios Agrupados: User Manual of the ESPSS EcosimPro Libraries. 3.6.0 Edition. 2022.

[56] B. Eysenbach and S. Levine: Maximum Entropy RL (Provably) Solves Some Robust RL Problems. 2022. arXiv: 2103.06257 [cs].

[57] L. Federici, B. Benedikter, and A. Zavoli: Deep Learning Techniques for Autonomous Spacecraft Guidance During Proximity Operations. *Journal of Spacecraft and Rockets*, 58(6), 2021, pp. 1774–1785. doi: 10.2514/1.A35076

[58] G. F. Franklin, D. J. Powell, M. L. Workman, et al.: Digital Control of Dynamic Systems. 3rd ed. Menlo Park, CA: Addison Wesley Longman, 2002.

[59] G. F. Franklin, J. D. Powell, and A. Emami-Naeini: Feedback Control of Dynamic Systems. 6th ed. Upper Saddle River, NJ: Pearson, 2010.

[60] Y. Fukushima, H. Nakatsuzi, R. Nagao, et al.: Development Status of LE-7A and LE-5B Engines for H-IIA Family. *Acta Astronautica*, 50(5), 2002, pp. 275–284. doi: 10.1016/S0094-5765(01)00165-5

[61] F. Galli, V. Sircoulomb, G. Hoblos, et al.: Remaining Useful Life Estimation Based on Wavelet Decomposition: Application to Bearings in Reusable Liquid Propellant Rocket Engines. In: *Recent Developments in Model-Based and Data-Driven Methods for Advanced Control and Diagnosis*. Ed. by D. Theilliol, J. Korbicz, and J. Kacprzyk. Vol. 467. Cham, Switzerland: Springer Nature, 2023, pp. 107–119. doi: 10.1007/978-3-031-27540-1_10

[62] B. Gaudet and R. Furfaro: Line of Sight Curvature for Missile Guidance Using Reinforcement Meta-Learning. In: *AIAA SCITECH Forum*. National Harbor, MD & Online, 2023. doi: 10.2514/6.2023-1627

[63] M. Ge, M.-S. Chiu, and Q.-G. Wang: Robust PID Controller Design via LMI Approach. *Journal of Process Control*, 12(1), 2002, pp. 3–13. doi: 10.1016/S0959-1524(00)00057-3

[64] S. L. Gonidec, L. Ouakrat, S. Colas, et al.: From the Firsts Engine Control Evaluation to Vinci, the First European Engine Numerically Controlled in Flight. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[65] D. Görges: Relations between Model Predictive Control and Reinforcement Learning. *IFAC-PapersOnLine*, 50(1), 2017, pp. 4920–4928. doi: 10.1016/j.ifacol.2017.08.747

[66] A. Grebe, M. Andrik, and D. Suslov: LUMEN Config File. Tech. rep. DLR-LA-RAK-P8-CF-081-LUMEN_DEMO_24. Lampoldshausen, Germany: DLR, 2024.

[67] E. W. Gretok, E. T. Kain, and A. D. George: Comparative Benchmarking Analysis of Next-Generation Space Processors. In: *IEEE Aerospace Conference*. Big Sky, MT, 2019. DOI: 10.1109/AERO.2019.8741914

[68] S. Gu, L. Yang, Y. Du, et al.: A Review of Safe Reinforcement Learning: Methods, Theory and Applications. 2024. arXiv: 2205.10330 [cs].

[69] M. T. Gulczynski, J. R. Riccius, G. Waxenegger-Wilfing, et al.: Combustion Chamber Fatigue Life Analysis for Reusable Liquid Rocket Engines (LREs). In: *AIAA SCITECH Forum*. National Harbor, MD & Online, 2023. DOI: 10.2514/6.2023-1263

[70] M. T. Gulczyński, J. R. Riccius, E. B. Zametaev, et al.: Turbine Blades for Reusable Liquid Rocket Engines (LRE) - Numerical Fatigue Life Investigation. In: *IEEE Aerospace Conference*. Big Sky, MT, 2023. DOI: 10.1109/AERO55745.2023.10115912

[71] T. Haarnoja, A. Zhou, K. Hartikainen, et al.: Soft Actor-Critic Algorithms and Applications, 2019. arXiv: 1812.05905.

[72] N. F. Haddad, R. D. Brown, R. Ferguson, et al.: Second Generation (200MHz) RAD750 Microprocessor Radiation Evaluation. In: *12th European Conference on Radiation and Its Effects on Components and Systems*. Sevilla, Spain: IEEE, 2011, pp. 877–880. DOI: 10.1109/RADECS.2011.6131320

[73] J. Haemisch, D. Suslov, G. Waxenegger-Wilfing, et al.: LUMEN – Design of the Regenerative Cooling System for an Expander Bleed Cycle Engine Using Methane. In: *Space Propulsion Conference*. Virtual: 3AF, 2021.

[74] M. Hagan and Demuth: Neural Networks for Control. In: *American Control Conference*. San Diego, CA: IEEE, 1999, pp. 1642–1656. DOI: 10.1109/ACC.1999.786109

[75] R. H. S. Hahn, J. Deeken, M. Oschwald, et al.: Turbine Design and Optimization Tool for LUMEN Expander Cycle Demonstrator. *Journal of Physics: Conference Series*, 2021. DOI: 10.1088/1742-6596/1909/1/012053

[76] R. H. S. Hahn, J. Deeken, Tobias Traudt, et al.: LUMEN Turbopump - Preliminary Design of Supersonic Turbine. In: *32nd International Symposium on Space Technology and Science*. 2019.

[77] R. H. S. Hahn and K. Dresia: LUMEN Generic Turbopump Models. Tech. rep. DLR-LA-LM-TP-TB-005-01. Lampoldshausen, Germany: DLR, 2024.

[78] M. Han, Y. Tian, L. Zhang, et al.: Reinforcement Learning Control of Constrained Dynamic Systems with Uniformly Ultimate Boundedness Stability Guarantee. *Automatica*, 129, 2021. doi: 10.1016/j.automatica.2021.109689

[79] A. Hans, D. Schneegass, A. M. Schafer, et al.: Safe Exploration for Reinforcement Learning. In: *Advances in Computational Intelligence and Learning: 16th European Symposium on Artificial Neural Network*. Bruges, Belgium, 2008.

[80] J. Hardi, J. Martin, M. Son, et al.: LUMEN Thrust Chamber – Injector Performance and Stability. In: *Space Propulsion Conference*. Virtual: 3AF, 2022.

[81] J. P. Hathout, M. Fleifil, A. M. Annaswamy, et al.: Combustion Instability Active Control Using Periodic Fuel Injection. *Journal of Propulsion and Power*, 18(2), 2002, pp. 390–399. doi: 10.2514/2.5947

[82] K. Henderson, N. Wiatrek, and P. Saenz: ARMing the Next Generation of Spaceflight Embedded Platforms Through Processor Reusability. In: *IEEE Aerospace Conference*. Big Sky, MT, 2023. doi: 10.1109/AERO55745.2023.10115627

[83] M. Hertneck, J. Kohler, S. Trimpe, et al.: Learning an Approximate Model Predictive Controller With Guarantees. *IEEE Control Systems Letters*, 2(3), 2018, pp. 543–548. doi: 10.1109/LCSYS.2018.2843682

[84] M. Hjorth, M. Åberg, N.-J. Wessman, et al.: GR740: Rad-hard Quad-Core LEON4FT System-on-Chip. In: *Programme and Abstracts Book of the DASIA Conference*. Barcelona, Spain, 2015.

[85] H. Holt, R. Armellin, N. Baresi, et al.: Optimal Q-laws via Reinforcement Learning with Guaranteed Stability. *Acta Astronautica*, 187, 2021, pp. 511–528. doi: 10.1016/j.actaastro.2021.07.010

[86] T. Hörger, K. Dresia, G. Waxenegger-Wilfing, et al.: Preliminary Investigation of Robust Reinforcement Learning for Control of an Existing Green Propellant Thruster. In: *AIAA Propulsion & Energy Forum*. Virtual, 2021. doi: 10.2514/6.2021-3223

[87] T. Hörger, L. Werling, K. Dresia, et al.: Experimental and Simulative Evaluation of a Reinforcement Learning Based Cold Gas Thrust Chamber Pressure Controller. *Acta Astronautica*, 219, 2024, pp. 128–137. doi: 10.1016/j.actaastro.2024.02.039

[88] K. Hornik, M. Stinchcombe, and H. White: Multilayer Feedforward Networks Are Universal Approximators. *Neural Networks*, 2(5), 1989, pp. 359–366. doi: 10.1016/0893-6080(89)90020-8

[89] H. Hose, J. Köhler, M. N. Zeilinger, et al.: Approximate Non-Linear Model Predictive Control with Safety-Augmented Neural Networks. 2023. arXiv: 2304.09575 [cs, eess, math].

[90] F. Hötte, C. v. Sethe, T. Fiedler, et al.: Experimental Lifetime Study of Regeneratively Cooled Rocket Chamber Walls. *International Journal of Fatigue*, 138, 2020. doi: 10.1016/j.ijfatigue.2020.105649

[91] K. Hovell and S. Ulrich: Deep Reinforcement Learning for Spacecraft Proximity Operations Guidance. *Journal of Spacecraft and Rockets*, 58(2), 2021, pp. 254–264. doi: 10.2514/1.A34838

[92] D. H. Huang and D. K. Huzel: Modern Engineering for Design of Liquid-Propellant Rocket Engines. Washington, DC: AIAA, 1992. doi: 10.2514/4.866197

[93] A. Hussein, M. M. Gaber, E. Elyan, et al.: Imitation Learning: A Survey of Learning Methods. *ACM Computing Surveys*, 50(2), 2018. doi: 10.1145/3054912

[94] F. Iandola and K. Keutzer: Small Neural Nets Are Beautiful: Enabling Embedded Systems with Small Deep-Neural-Network Architectures. In: *12th IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis Companion*. Seoul, Korea: ACM, 2017. doi: 10.1145/3125502.3125606

[95] J. Ibarz, J. Tan, C. Finn, et al.: How to Train Your Robot with Deep Reinforcement Learning: Lessons We Have Learned. *The International Journal of Robotics Research*, 40(4-5), 2021, pp. 698–721. doi: 10.1177/0278364920987859

[96] S. Ishimoto, M. Illig, and E. Dumont: Development Status of CALLISTO. In: *33rd ISTS Conference*. Beppu, Japan, 2022.

[97] A. Isselhorst: HM7B Simulation with ESPSS Tool on ARIANE 5 ESC-A Upper Stage. In: *46th AIAA/ASME/SAE/ASEE Joint Propulsion Conference & Exhibit*. Nashville, TN, 2010. doi: 10.2514/6.2010-7047

[98] X. Iturbe, B. Venu, E. Ozer, et al.: A Triple Core Lock-Step (TCLS) ARM® Cortex®-R5 Processor for Safety-Critical and Ultra-Reliable Applications. In: *46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshop*. Toulouse, France, 2016, pp. 246–249. doi: 10.1109/DSN-W.2016.57

[99] A. Janczak: Neural Network Wiener Models. In: *Identification of Nonlinear Systems Using Neural Networks and Polynomial Models*. Vol. 310. Berlin: Springer, 2004, pp. 31–75. doi: 10.1007/978-3-540-31596-4_2

[100] N. Jansen, B. Könighofer, S. Junges, et al.: Safe Reinforcement Learning Using Probabilistic Shields. *LIPIcs, CONCUR*, 171, 2020. doi: 10.4230/LIPICS.CONCUR.2020.3

[101] H. Ji and C. Yin: Application of Soft Actor-Critic Reinforcement Learning to a Search and Rescue Task for Humanoid Robots. In: *2022 China Automation Congress (CAC)*. Xiamen, China: IEEE, 2022, pp. 3954–3960. doi: 10.1109/CAC57257.2022.10056003

[102] M. Jin and J. Lavaei: Stability-Certified Reinforcement Learning: A Control-Theoretic Perspective. *IEEE*, 8, 2020. doi: 10.1109/ACCESS.2020.3045114

[103] M. P. Juniper and R. Sujith: Sensitivity and Nonlinearity of Thermoacoustic Oscillations. *Annual Review of Fluid Mechanics*, 50(1), 2018, pp. 661–689. doi: 10.1146/annurev-fluid-122316-045125

[104] T. Kailath, A. H. Sayed, and B. Hassibi: Linear Estimation. Prentice Hall Information and System Sciences Series. Upper Saddle River, N.J: Prentice Hall, 2000.

[105] S. Kanso, M. S. Jha, M. Galeotta, et al.: Remaining Useful Life Prediction with Uncertainty Quantification of Liquid Propulsion Rocket Engine Combustion Chamber. *IFAC*, 55(6), 2022, pp. 96–101. doi: 10.1016/j.ifacol.2022.07.112

[106] T. Kimura, T. Hashimoto, M. Sato, et al.: Reusable Rocket Engine: Firing Tests and Lifetime Analysis of Combustion Chamber. *Journal of Propulsion and Power*, 32(5), 2016, pp. 1087–1094. doi: 10.2514/1.B35973

[107] H. Krasowski, Y. Zhang, and M. Althoff: Safe Reinforcement Learning for Urban Driving Using Invariably Safe Braking Sets. In: *25th International Conference on Intelligent Transportation Systems (ITSC)*. Macau, China: IEEE, 2022, pp. 2407–2414. doi: 10.1109/ITSC55140.2022.9922166

[108] E. Kurudzija, K. Dresia, J. Martin, et al.: Virtual Sensing for Fault Detection within the LUMEN Fuel Turbopump Test Campaign. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[109] E. Lavretsky and K. A. Wise: Robust and Adaptive Control: With Aerospace Applications. Advanced Textbooks in Control and Signal Processing. London, United Kingdom: Springer, 2013. DOI: 10.1007/978-1-4471-4396-3

[110] C. Legaard, T. Schranz, G. Schweiger, et al.: Constructing Neural Network Based Models for Simulating Dynamical Systems. *ACM Computing Surveys*, 55(11), 2023. DOI: 10.1145/3567591

[111] R. D. Legler and F. V. Bennett: Space Shuttle Mission Summary. Tech. rep. NASA/TM–2011–216142. Johnson Space Center: NASA Scientific and Technical Information Program Office, 2011.

[112] G. Lentaris, K. Maragos, I. Stratakos, et al.: High-Performance Embedded Computing in Space: Evaluation of Platforms for Vision-Based Navigation. *Journal of Aerospace Information Systems*, 15(4), 2018, pp. 178–192. DOI: 10.2514/1.I010555

[113] W. Ley, K. Wittmann, and W. Hallmann, eds.: Handbuch der Raumfahrttechnik. 5th ed. München, Germany: Hanser, 2019.

[114] E. Liang, R. Liaw, R. Nishihara, et al.: RLlib: Abstractions for Distributed Reinforcement Learning. In: *35th International Conference on Machine Learning*. Ed. by J. Dy and A. Krause. Vol. 80. Stockholm, Sweden: PMLR, 2018, pp. 3053–3062.

[115] T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al.: Continuous Control with Deep Reinforcement Learning, 2019. arXiv: 1509.02971.

[116] C. F. Lorenzo: Continuum Fatigue Damage Modeling for Use in Life Extending Control. Technical Memorandum (TM) NASA-TM-106691. Cleveland, OH: NASA Lewis Research Center, 1994.

[117] C. F. Lorenzo, W. C. Merrill, J. L. Musgrave, et al.: Controls Concepts for next Generation Reusable Rocket Engines. Technical Memorandum (TM) NASA-TM-106902. Seattle, WA: NASA Lewis Research Center, 1995.

[118] C. F. Lorenzo and J. L. Musgrave: Overview of Rocket Engine Control. In: *AIP Conference Proceedings*. Vol. 246. Albuquerque, New Mexico (USA): AIP, 1992, pp. 446–455. DOI: 10.1063/1.41807

[119] C. F. Lorenzo, A. Ray, and M. S. Holmes: Nonlinear Control of a Reusable Rocket Engine for Life Extension. *Journal of Propulsion and Power*, 17(5), 2001, pp. 998–1004. DOI: 10.2514/2.5861

[120] A. Lund, Z. A. Haj Hammadeh, P. Kenny, et al.: ScOSA System Software: The Reliable and Scalable Middleware for a Heterogeneous and Distributed on-Board Computer Architecture. *CEAS Space Journal*, 14(1), 2022, pp. 161–171. DOI: 10.1007/s12567-021-00371-7

[121] J. Maciejowski, P. Goulart, and E. Kerrigan: Constrained Control Using Model Predictive Control. In: *Advanced Strategies in Control Systems with Input and Output Constraints*. Ed. by S. Tarbouriech, G. Garcia, and A. H. Glattfelder. Berlin, Germany: Springer, 2007, pp. 273–291. doi: 10.1007/978-3-540-37010-9_9

[122] J. Martin, W. Armbruster, J. Hardi, et al.: Experimental Investigation of Self-Excited Combustion Instabilities in a LOX/LNG Rocket Combustor. *Journal of Propulsion and Power*, 37(6), 2021, pp. 944–951. doi: 10.2514/1.B38289

[123] I. Masaya, T. Masaki, S. Yukari, et al.: Test and Maintenance Technologies of the Facility for LE-9 Engine Firing Test. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[124] D. McComas: Lessons from 30 Years of Flight Software. Tech. rep. NASA Goddard Space Flight Center, 2015.

[125] K. McManus, T. Poinsot, and S. Candel: A Review of Active Control of Combustion Instabilities. *Progress in Energy and Combustion Science*, 19(1), 1993. doi: 10.1016/0360-1285(93)90020-F

[126] V. Mnih, K. Kavukcuoglu, D. Silver, et al.: Human-Level Control through Deep Reinforcement Learning. *Nature*, 518(7540), 2015, pp. 529–533. doi: 10.1038/nature14236

[127] J. Moos, K. Hansel, H. Abdulsamad, et al.: Robust Reinforcement Learning: A Review of Foundations and Recent Advances. *Machine Learning and Knowledge Extraction*, 4(1), 2022, pp. 276–315. doi: 10.3390/make4010013

[128] F. Muratore, F. Ramos, G. Turk, et al.: Robot Learning From Randomized Simulations: A Review. *Frontiers in Robotics and AI*, 9, 2022. doi: 10.3389/frobt.2022.799893

[129] J. Musgrave, T.-H. Guo, E. Wong, et al.: Real-Time Accommodation of Actuator Faults on a Reusable Rocket Engine. *IEEE Transactions on Control Systems Technology*, 5(1), 1997, pp. 100–109. doi: 10.1109/87.553668

[130] J. Musgrave: Linear Quadratic Servo Control of a Reusable Rocket Engine. In: *27th Joint Propulsion Conference*. Sacramento, CA: AIAA, 1991. doi: 10.2514/6.1991-1999

[131] S. Mysore, B. E. Mabsout, R. Mancuso, et al.: Honey. I Shrunk The Actor: A Case Study on Preserving Performance with Smaller Actors in Actor-Critic RL. In: *2021 IEEE Conference on Games (CoG)*. Copenhagen, Denmark: IEEE, 2021. doi: 10.1109/CoG52621.2021.9619008

[132] V. Nair, G. Thampi, S. Karuppusamy, et al.: Loss of Chaos in Combustion Noise as a Precursor of Impending Combustion Instability. *International Journal of Spray and Combustion Dynamics*, 5(4), 2013, pp. 273–290. doi: 10.1260/1756-8277.5.4.273

[133] F. Nasuti and M. Pizzarelli: Pseudo-Boiling and Heat Transfer Deterioration While Heating Supercritical Liquid Rocket Engine Propellants. *The Journal of Supercritical Fluids*, 168, 2021. doi: 10.1016/j.supflu.2020.105066

[134] National Aeronautics and Space Administration (NASA): Space Launch System RS-25 Core Stage Engines. NASA Facts FS-2015-07-064-MSFC. Huntsville, AL: Marshall Space Flight Center, 2015.

[135] E. Nemeth, R. Anderson, J. Ols, et al.: Reusable Rocket Engine Intelligent Control System Framework Design, Phase 2. NASA Contractor Report 187213. Cleveland, OH: Lewis Research Center, 1991.

[136] E. Nemeth, R. Anderson, J. Maram, et al.: An Advanced Intelligent Control System Framework. In: *28th Joint Propulsion Conference and Exhibit*. Nashville, TN: AIAA, 1992. doi: 10.2514/6.1992-3162

[137] O. Notebaert: Research & Technology Activities in On-Board Data Processing Domain. In: *ADCSS 2015 – Avionics Technology Trends Workshop*. Noordwijk, Netherlands: ESA/ESTEC, 2015.

[138] H. Odriozola-Olalde, M. Zamalloa, and N. Arana-Arexolaleiba: Shielded Reinforcement Learning: A Review of Reactive Methods for Safe Learning. In: *IEEE/SICE International Symposium on System Integration*. Atlanta, GA, 2023. doi: 10.1109/SII55687.2023.10039301

[139] K. Ogata: Modern Control Engineering. 5th ed. Electrical Engineering: Instrumentation and Controls Series. Boston, MA: Prentice-Hall, 2010.

[140] OpenAI, I. Akkaya, M. Andrychowicz, et al.: Solving Rubik's Cube with a Robot Hand, 2019. arXiv: 1910.07113.

[141] P. Osinenko, D. Dobriborsci, and W. Aumer: Reinforcement Learning with Guarantees: A Review. *IFAC-PapersOnLine*, 55(15), 2022, pp. 123–128. doi: 10.1016/j.ifacol.2022.07.619

[142] F. Otto: Model-Free Deep Reinforcement Learning—Algorithms and Applications. In: *Reinforcement Learning Algorithms: Analysis and Applications*. Ed. by B. Belousov, H. Abdulsamad, P. Klink, et al. Vol. 883. Cham, Switzerland: Springer International Publishing, 2021, pp. 109–121. doi: 10.1007/978-3-030-41188-6_10

[143] T. Pan, S. Zhang, F. Li, et al.: A Meta Network Pruning Framework for Remaining Useful Life Prediction of Rocket Engine Bearings with Temporal Distribution Discrepancy. *Mechanical Systems and Signal Processing*, 195, 2023. DOI: 10.1016/j.ymssp.2023.110271

[144] X. Pan, X. Chen, Q. Zhang, et al.: Model Predictive Control : A Reinforcement Learning-based Approach. *Journal of Physics: Conference Series*, 2203(1), 2022. DOI: 10.1088/1742-6596/2203/1/012058

[145] G. Pannocchia: Offset-Free Tracking MPC: A Tutorial Review and Comparison of Different Formulations. In: *2015 European Control Conference (ECC)*. Linz, Austria: IEEE, 2015, pp. 527–532. DOI: 10.1109/ECC.2015.7330597

[146] C. Pauly, J. Sender, and M. Oschwald: Ignition of a Gaseous Methane/Oxygen Coaxial Jet. In: *Progress in Propulsion Physics*. Brussels, Belgium: EDP Sciences, 2009, pp. 155–170. DOI: 10.1051/eucass/200901155

[147] X. B. Peng, M. Andrychowicz, W. Zaremba, et al.: Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. In: *International Conference on Robotics and Automation*. Brisbane, Australia: IEEE, 2018, pp. 3803–3810. DOI: 10.1109/ICRA.2018.8460528

[148] S. Perez-Roca: Model-Based Robust Transient Control of Reusable Liquid-Propellant Rocket Engines. PhD thesis. Université Paris-Saclay, France, 2020.

[149] S. Perez-Roca, J. Marzat, H. Piet-Lahanier, et al.: Model-Based Robust Transient Control of Reusable Liquid-Propellant Rocket Engines. *IEEE Transactions on Aerospace and Electronic Systems*, 2020. DOI: 10.1109/TAES.2020.3010668

[150] S. Pérez-Roca, J. Marzat, H. Piet-Lahanier, et al.: A Survey of Automatic Control Methods for Liquid-Propellant Rocket Engines. *Progress in Aerospace Sciences*, 107, 2019, pp. 63–84. DOI: 10.1016/j.paerosci.2019.03.002

[151] T. Poinsot, F. Bourienne, S. Candel, et al.: Suppression of Combustion Instabilities by Active Control. *Journal of Propulsion and Power*, 5(1), 1989, pp. 14–20. DOI: 10.2514/3.23108

[152] L. Prévost, M. Theron, G. Fiore, et al.: CNES Support on LOX/Methane Prometheus Development. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[153] S. Raghu and K. Sreenivasan: Control of Acoustically Coupled Combustion and Fluid Dynamic Instabilities. In: *11th Aeroacoustics Conference*. Sunnyvale, CA: AIAA, 1987. DOI: 10.2514/6.1987-2690

[154] P. Rajendra and V. Brahmajirao: Modeling of Dynamical Systems through Deep Learning. *Biophysical Reviews*, 12(6), 2020, pp. 1311–1320. doi: 10.1007/s12551-020-00776-4

[155] H. C. T. Raposo: Mixture Ratio and Thrust Control of a Liquid-Propellant Rocket Engine. Master's Thesis. Portugal: University of Lisbon & CNES, 2016.

[156] J. Rawlings: Tutorial Overview of Model Predictive Control. *IEEE Control Systems*, 20(3), 2000, pp. 38–52. doi: 10.1109/37.845037

[157] A. Ray, M. S. Holmes, and C. F. Lorenzo: Life Extending Controller Design for Reusable Rocket Engines. *The Aeronautical Journal*, 105(1048), 2001, pp. 315–322. doi: 10.1017/S0001924000012197

[158] A. Ray, X. Dai, M.-K. Wu, et al.: Damage-Mitigating Control of a Reusable Rocket Engine. *Journal of Propulsion and Power*, 10(2), 1994, pp. 225–234. doi: 10.2514/3.23733

[159] M. A. Reitemeyer: Systemanalysis of LOX/LNG Expander Bleed Engine 'LUMEN'. Master's Thesis. Germany: JMU Würzburg, 2023.

[160] W. Retagne, J. Dauer, and G. Waxenegger: Adaptive Satellite Attitude Control for Varying Masses Using Deep Reinforcement Learning. *Frontiers in Robotics and AI*, 11, 2024. doi: 10.3389/frobt.2024.1402846

[161] J. Rhea: BAE Systems Moves into Third Generation Rad-Hard Processors. 2002. url: https://www.militaryaerospace.com/computers/article/16710930/bae-systems-moves-into-third-generation-radhard-processors (visited on 08/20/2024).

[162] M. Riedmiller: 10 Steps and Some Tricks To Set Up Neural Reinforcement Controllers. Tech. rep. Germany: University of Freiburg.

[163] Rocketdyne: Space Shuttle Main Engine Training. Presentation, BV98-04. 1998.

[164] M. Rotondi, P. Concio, S. D'Alessandro, et al.: Development and Validation of Nozzle Erosion Models for Solid and Hybrid Rockets in the ESPSS Libraries. In: *Space Propulsion Conference*. Estoril, Portugal: 3AF, 2022.

[165] J. Ruiz: Simulation of the Complete Electric Propulsion System Associated to HET on ESPSS/EcosimPro. In: *Space Propulsion Conference*. Estoril, Portugal: 3AF, 2022.

[166] M. Sagliano, T. Tsukamoto, J. A. Macés-Hernández, et al.: Guidance and Control Strategy for the CALLISTO Flight Experiment. In: *8th European Conference for Aeronautics and Aerospace Sciences (EUCASS)*. Madrid, Spain, 2018.

[167] T. Salzmann, E. Kaufmann, J. Arrizabalaga, et al.: Real-Time Neural MPC: Deep Learning Model Predictive Control for Quadrotors and Agile Robotic Platforms. *IEEE Robotics and Automation Letters*, 8(4), 2023, pp. 2397–2404. doi: 10.1109/LRA.2023.3246839

[168] S. Sankararaman and K. Goebel: Remaining Useful Life Estimation in Prognosis: An Uncertainty Propagation Problem. In: *Infotech@Aerospace Conference*. Boston, MA: AIAA, 2013. doi: 10.2514/6.2013-4901

[169] A. M. Saraf, T. Traudt, and M. Oschwald: LUMEN Turbopump: Preliminary Thermal Model. *Journal of Physics: Conference Series*, 1909(1), 2021. doi: 10.1088/1742-6596/1909/1/012052

[170] R. Saudemont and S. Le Gonidec: Study of a Robust Control Law Based on (H_∞) for the Vulcain Rocket Engine. Tech. rep. Vernon, France: ArianeGroup, 2000.

[171] A. M. Schäfer: Reinforcement Learning with Recurrent Neural Networks. PhD thesis. Germany: University of Osnabrück.

[172] D. P. Scharf, B. Açikmeşe, D. Dueri, et al.: Implementation and Experimental Demonstration of Onboard Powered-Descent Guidance. *Journal of Guidance, Control, and Dynamics*, 40(2), 2017, pp. 213–229. doi: 10.2514/1.G000399

[173] M. Schrebe: Ethernetschnittstelle Zum P8. Internship Report. Lampoldshausen, Germany: DLR, 2022.

[174] J. Schrittwieser, I. Antonoglou, T. Hubert, et al.: Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. *Nature*, 588(7839), 2020, pp. 604–609. doi: 10.1038/s41586-020-03051-4

[175] J. Schulman, F. Wolski, P. Dhariwal, et al.: Proximal Policy Optimization Algorithms, 2017. arXiv: 1707.06347.

[176] M. Schwenzer, M. Ay, T. Bergs, et al.: Review on Model Predictive Control: An Engineering Perspective. *The International Journal of Advanced Manufacturing Technology*, 117(5-6), 2021, pp. 1327–1349. doi: 10.1007/s00170-021-07682-3

[177] A. Scorsoglio, A. D'Ambrosio, L. Ghilardi, et al.: Image-Based Deep Reinforcement Meta-Learning for Autonomous Lunar Landing. *Journal of Spacecraft and Rockets*, 59(1), 2022, pp. 153–165. doi: 10.2514/1.A35072

[178] P. F. Seitz and R. F. Searle: Space Shuttle Main Engine Control System. In: *National Aerospace Engineering and Manufacturing Meeting*. Los Angeles, CA: Society of Automotive Engineers, 1973. doi: 10.4271/730927

[179] D. Silver, T. Hubert, J. Schrittwieser, et al.: A General Reinforcement Learning Algorithm That Masters Chess, Shogi, and Go through Self-Play. *Science*, 362(6419), 2018, pp. 1140–1144. DOI: 10.1126/science.aar6404

[180] D. Silver, J. Schrittwieser, K. Simonyan, et al.: Mastering the Game of Go without Human Knowledge. *Nature*, 550(7676), 2017, pp. 354–359. DOI: 10.1038/nature24270

[181] D. Sima: ARM Processors Line. Lecture. Hungary: Obuda University Budapest, 2018.

[182] C. Simon, D. Theilliol, D. Sauter, et al.: Remaining Useful Life Assessment via Residual Generator Approach - A SOH Virtual Sensor Concept. In: *Conference on Control Applications*. Juan-les-Pins, France: IEEE, 2014, pp. 1202–1207. DOI: 10.1109/CCA.2014.6981492

[183] D. Simon: Optimal State Estimation: Kalman, H∞, and Nonlinear Approaches. Hoboken, NJ: John Wiley & Sons, 2006. DOI: 10.1002/0470045345

[184] P. Simontacchi, A. Remy, E. Humbert, et al.: PROMETHEUS: Precursor of New Low-Cost Rocket Engine Family. In: *10th European Conference for Aeronautics and Space Sciences (EUCASS)*. Lausanne, Switzerland, 2023.

[185] B. Singh, R. Kumar, and V. P. Singh: Reinforcement Learning in Robotic Applications: A Comprehensive Survey. *Artificial Intelligence Review*, 55(2), 2022, pp. 945–990. DOI: 10.1007/s10462-021-09997-9

[186] SpaceX: Falcon User's Guide. 2021. URL: https://www.spacex.com/media/falcon-users-guide-2021-09.pdf (visited on 08/20/2024).

[187] STMicroelectronics: STM32H742xI/G STM32H743xI/G. Datasteeht DS12110 Rev 10. Plan-les-Ouates, Switzerland, 2023.

[188] H. Sunakawa, T. Kobayashi, K. Okita, et al.: Development Status of Electrical Valve Control System for LE-9 Engine. In: *53rd AIAA/SAE/ASEE Joint Propulsion Conference*. Atlanta, GA, 2017. DOI: 10.2514/6.2017-4671

[189] H. Sunakawa, A. Kurosu, K. Okita, et al.: Automatic Thrust and Mixture Ratio Control of LE-X. In: *44th AIAA/ASME/SAE/ASEE Joint Propulsion Conference & Exhibit*. Hartford, CT, 2008. DOI: 10.2514/6.2008-4666

[190] D. I. Suslov, W. Armbruster, J. Haemisch, et al.: Experimental Investigation of Start Transients in LOX/LNG 25 kN Regenerative Cooled Subscale Thrust Chamber. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[191] D. I. Suslov, J. Haemisch, E. B. Zametaev, et al.: Development of a 25-kN Regeneratively Cooled LOX/LNG Thrust Chamber for LUMEN. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[192] G. P. Sutton and O. Biblarz: Rocket Propulsion Elements. 7th ed. New York: John Wiley & Sons, 2001.

[193] R. S. Sutton and A. G. Barto: Reinforcement Learning: An Introduction. Adaptive Computation and Machine Learning Series. Cambridge, MA: The MIT Press, 2018.

[194] X. Tang, B. Huang, T. Liu, et al.: Highway Decision-Making and Motion Planning for Autonomous Driving via Soft Actor-Critic. *IEEE Transactions on Vehicular Technology*, 71(5), 2022, pp. 4706–4717. doi: 10.1109/TVT.2022.3151651

[195] T.-T. Tay, I. Mareels, and J. B. Moore: High Performance Control. Systems & Control. Boston: Birkhäuser, 1998.

[196] A. Taylor: What Is Control Valve Authority | Flo Control. 2020. url: https://www.flocontrol.ltd.uk/what-is-control-valve-authority/ (visited on 03/02/2024).

[197] M. Tipaldi, R. Iervolino, and P. R. Massenio: Reinforcement Learning in Spacecraft Control Applications: Advances, Prospects, and Challenges. *Annual Reviews in Control*, 54, 2022, pp. 1–23. doi: 10.1016/j.arcontrol.2022.07.004

[198] J. Tobin, R. Fong, A. Ray, et al.: Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World, 2017. arXiv: 1703.06907.

[199] B. D. Tracey, A. Michi, Y. Chervonyi, et al.: Towards Practical Reinforcement Learning for Tokamak Magnetic Control. 2023. arXiv: 2307.11546 [physics].

[200] T. Traudt, W. Armbruster, C. Groll, et al.: LUMEN, the Test Bed for Rocket Engine Components: Results of the Acceptance Tests and Overview on the Engine Test Preparation. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[201] T. Traudt, J. Deeken, M. Oschwald, et al.: Liquid Upper Stage Demonstrator Engine (LUMEN): Overview and Project Progress. In: *International Astronautical Congress (IAC)*. Dubai, Ireland, 2021.

[202] T. Traudt, R. H. Dos Santos Hahn, T. Mason, et al.: LUMEN Turbopump - Design and Manufacturing of the LUMEN LOX and LNG Turbopump Components. In: *32nd International Symposium on Space Technology and Science (ISTS)*. Fukui, Japan, 2019.

[203] T. Traudt, M. Oschwald, and S. Schlechtriem: LUMEN Turbopump - Status of the Development and Testing. In: *Space Propulsion Conference*. Virtual: 3AF, 2022.

[204] T. Troudet and W. Merrill: A Real-Time Neural Net Estimator of Fatigue Life. NASA Technical Memorandum NASA-TM-103117. Cleveland, OH: Lewis Research Center, 1990.

[205] G. Valchev: Model Predictive Control of Transiently Operating Liquid-Propellant Rocket Engines. Master's Thesis. Germany: RWTH Aachen, 2022.

[206] Verein Deutscher Ingenieur (VDI): Fluidic characteristic quantities of control valves and their determination. Tech. rep. VDI/VDE 2173. Düsseldorf, Germany, 2007.

[207] O. Vinyals, I. Babuschkin, W. M. Czarnecki, et al.: Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning. *Nature*, 575(7782), 2019, pp. 350–354. doi: 10.1038/s41586-019-1724-z

[208] A. Wächter and L. T. Biegler: On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming. *Mathematical Programming*, 106(1), 2006, pp. 25–57. doi: 10.1007/s10107-004-0559-y

[209] D. Wang, W. Zheng, Z. Wang, et al.: Comparison of Reinforcement Learning and Model Predictive Control for Building Energy System Optimization. *Applied Thermal Engineering*, 228, 2023. doi: 10.1016/j.applthermaleng.2023.120430

[210] J. Wang, M. Wei, and X. Xing: Static Gain Estimation for Nonlinear Dynamic Systems from Steady-State Values Hidden in Historical Data. *ISA Transactions*, 120, 2022, pp. 78–88. doi: 10.1016/j.isatra.2021.03.007

[211] L. Y. Wang and J.-F. Zhang: Fundamental Limitations and Differences of Robust and Adaptive Control. In: *American Control Conference*. Vol. 6. Arlington, VA: IEEE, 2001, pp. 4802–4807. doi: 10.1109/ACC.2001.945742

[212] C. J. C. H. Watkins and P. Dayan: Q-Learning. *Machine Learning*, 8(3-4), 1992, pp. 279–292. doi: 10.1007/BF00992698

[213] G. Waxenegger-Wilfing, K. Dresia, J. Deeken, et al.: A Reinforcement Learning Approach for Transient Control of Liquid Rocket Engines. *IEEE Transactions on Aerospace and Electronic Systems*, 57(5), 2021, pp. 2938–2952. doi: 10.1109/TAES.2021.3074134

[214] G. Waxenegger-Wilfing, U. Sengupta, J. Martin, et al.: Early Detection of Thermoacoustic Instabilities in a Cryogenic Rocket Thrust Chamber Using Combustion Noise Features and Machine Learning. *Chaos: An Interdisciplinary Journal of Nonlinear Science*, 31(6), 2021. doi: 10.1063/5.0038817

[215] A. R. Weiss: Dhrystone Benchmark: History, Analysis, "Scores" and Recommendations. White Paper. Embedded Microprocessor Benchmark Consortium, 2002.

[216] D. Welberg, B. Determann, C. Hessel, et al.: The Astris Kickstage Propulsion System Development Status & Outlook. In: *Space Propulsion Conference*. Estoril, Portugal: 3AF, 2022.

[217] R. Welch, D. Limonadi, and R. Manning: Systems Engineering the Curiosity Rover: A Retrospective. In: *8th International Conference on System of Systems Engineering*. Maui, HI: IEEE, 2013, pp. 70–75. doi: 10.1109/SYSoSE.2013.6575245

[218] P. J. Werbos: A Menu of Designs for Reinforcement Learning Over Time. In: *Neural Networks for Control*. Ed. by W. T. Miller, R. S. Sutton, and P. J. Werbos. The MIT Press, 1991, pp. 67–96. doi: 10.7551/mitpress/4939.003.0007

[219] Reinforcement Learning and Markov Decision Processes. In: *Reinforcement Learning: State-of-the-Art*. Ed. by M. Wiering and M. Van Otterlo. Vol. 12. Berlin, Germany: Springer, 2012, pp. 3–42. doi: 10.1007/978-3-642-27645-3_1

[220] Y. Wu, L. Xing, F. Guo, et al.: On the Combination of PID Control and Reinforcement Learning: A Case Study with Water Tank System. In: *16th Conference on Industrial Electronics and Applications*. Chengdu, China: IEEE, 2021, pp. 1877–1882. doi: 10.1109/ICIEA51954.2021.9516140

[221] W. Yu, J. Tan, C. Karen Liu, et al.: Preparing for the Unknown: Learning a Universal Policy with Online System Identification. In: *Robotics: Science and Systems XIII*. Cambridge, MA: Robotics: Science and Systems Foundation, 2017. doi: 10.15607/RSS.2017.XIII.048

[222] R. Zhang, Y. Han, M. Su, et al.: Robust Reinforcement Learning with UUB Guarantee for Safe Motion Control of Autonomous Robots. *Science China Technological Sciences*, 67(1), 2024, pp. 172–182. doi: 10.1007/s11431-023-2435-3

A. Appendix

SAC hyperparameters

Tab. A.1 lists the SAC parameters that differ from the original paper [71].

Table A.1.: SAC hyperparameters

Parameter	Value
discount factor (γ)	0.9
replay buffer size	500 000
number of workers	10
number of GPUs	1
target entropy	-8

Configuration of test case 1

Tab. A.2 lists the reinforcement learning setup for test case 1.

Table A.2.: Reinforcement learning configuration for test case 1

Parameter	Value
$\beta(ROF, J, T_{\text{turbine}}, n_{\text{OTP}}, n_{\text{FTP}})$	0.5
$\beta(\dot{m}_{\text{RC}})$	0.2
γ	0.3
σ	12
N_f	4
N_p	3
Δt	0.1 s

Domain randomization in test case 2

Tab. A.3 lists the variables that are changed during domain randomization.

Table A.3.: Domain randomization parameters for test case 2

Component	Parameter	Unit	Min	Max
Valves	K_v	%	0.95	1.05
	a_max	%	0.94	1.06
	v_max	%	0.92	1.08
Thrust chamber	k_1	%	0.96	1.04
	k_3	%	0.98	1.02
	$r_\mathrm{RC}, r_\mathrm{NE}$	%	0.95	1.05
	η_{c^*}	%	0.99	1.01
	A_t	%	0.99	1.01
Injector	ζ_oxy, ζ_red	%	0.97	1.03
Turbopumps	η_FTP	%	0.97	1.03
	η_OTP	%	0.96	1.04
Interfaces	p_LOX, p_LNG	%	0.90	1.00
	T_LOX, T_LNG	%	0.97	1.03
Sensors delay	$\Delta T(p_\mathrm{cc})$	s	0.05	0.15
	$\Delta T(T_\mathrm{LNG})$	s	0.15	0.25
	$\Delta T(\dot{m}_\mathrm{RC})$	s	0.05	0.15
	$\Delta T(\dot{m}_\mathrm{LOX})$	s	0.1	0.25
	$\Delta T(\dot{m}_\mathrm{LNG})$	s	0.1	0.25
Valve delay	T_d	s	0.03	0.08

Experimental result of test run 1

Figure A.3 shows the result of test run 1. The controller is unstable due to a modeling error in the valve transfer function.

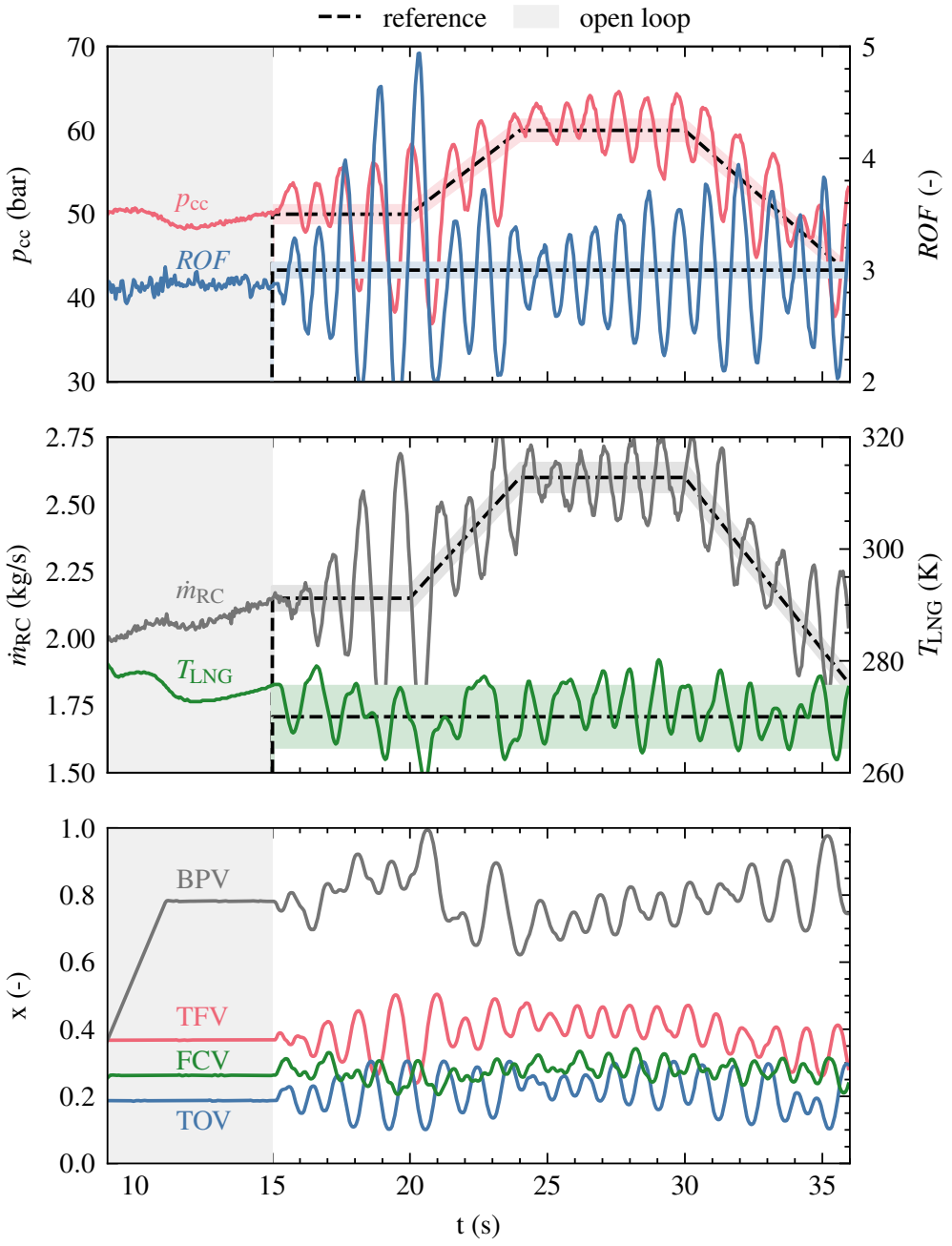


Figure A.1.: Experimental results of test run 1

Valve modeling

After the first test, it was discovered that the transient modeling of the flow control valves was erroneous. The response of all four valves to commands was significantly slower than modeled, especially when changing the direction of movement, as shown in Figure A.2.

After the first test, the valve modeling was corrected. The new model accuractely predicts the valve response to commands, as shown in Figure A.3 for test case 1 and Figure A.4 for test case 2.

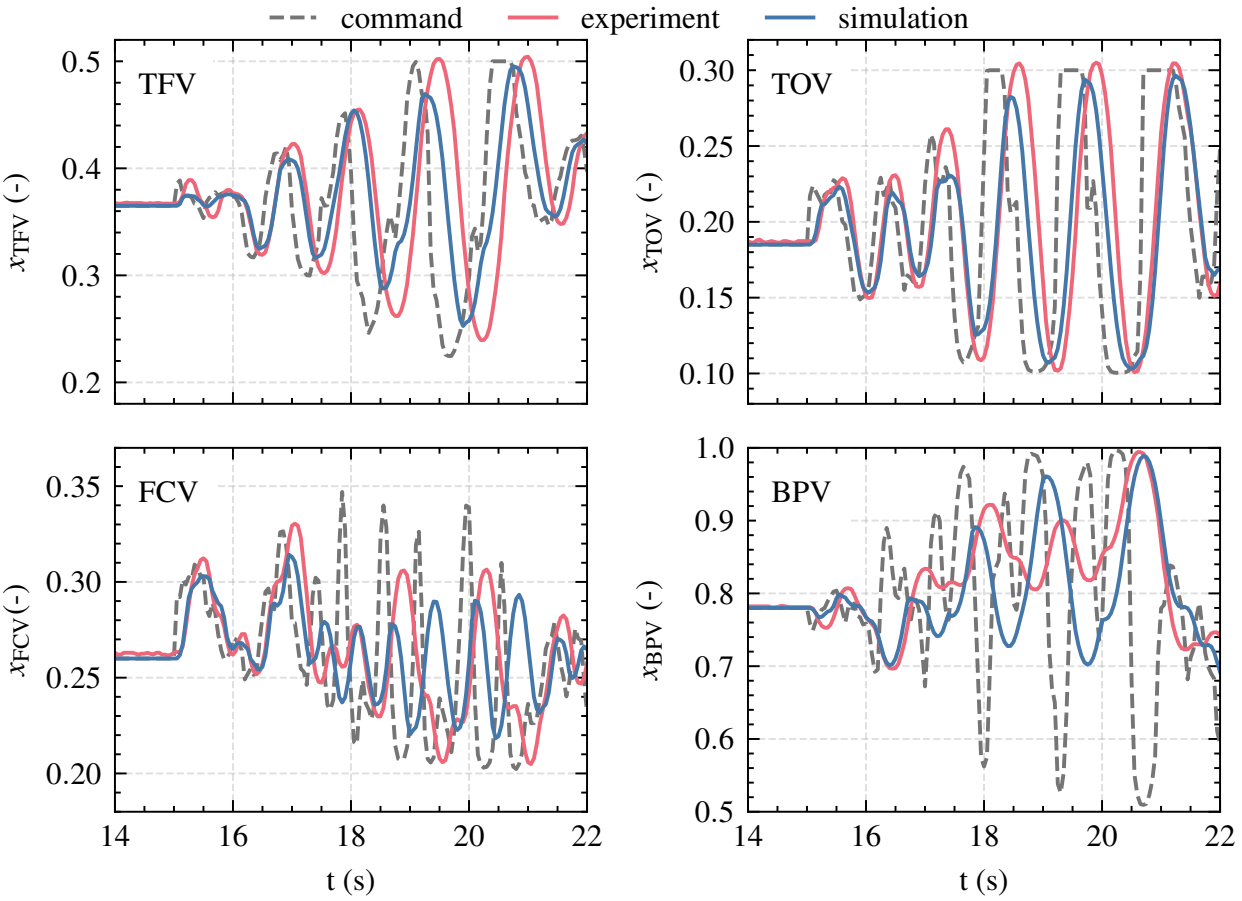


Figure A.2.: Baseline valve modeling for test run 1

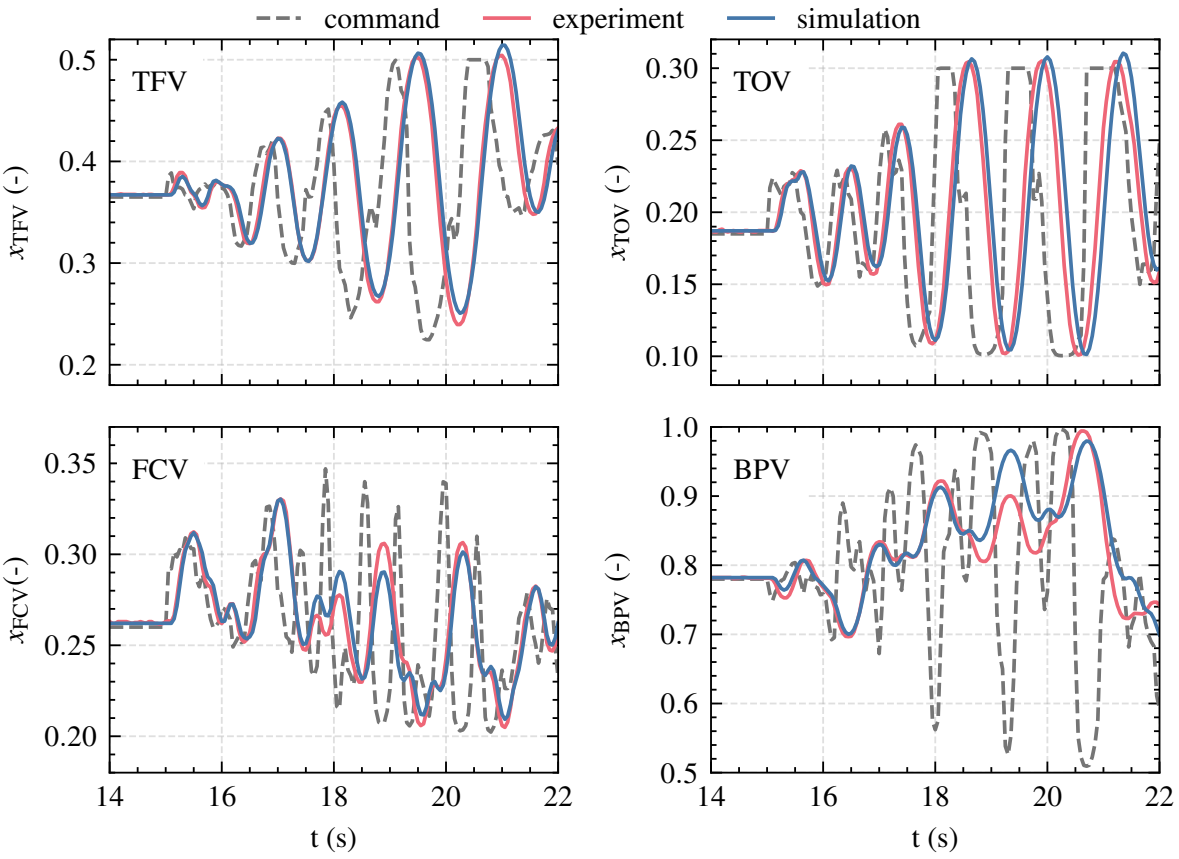


Figure A.3.: Improved valve modeling for test run 1

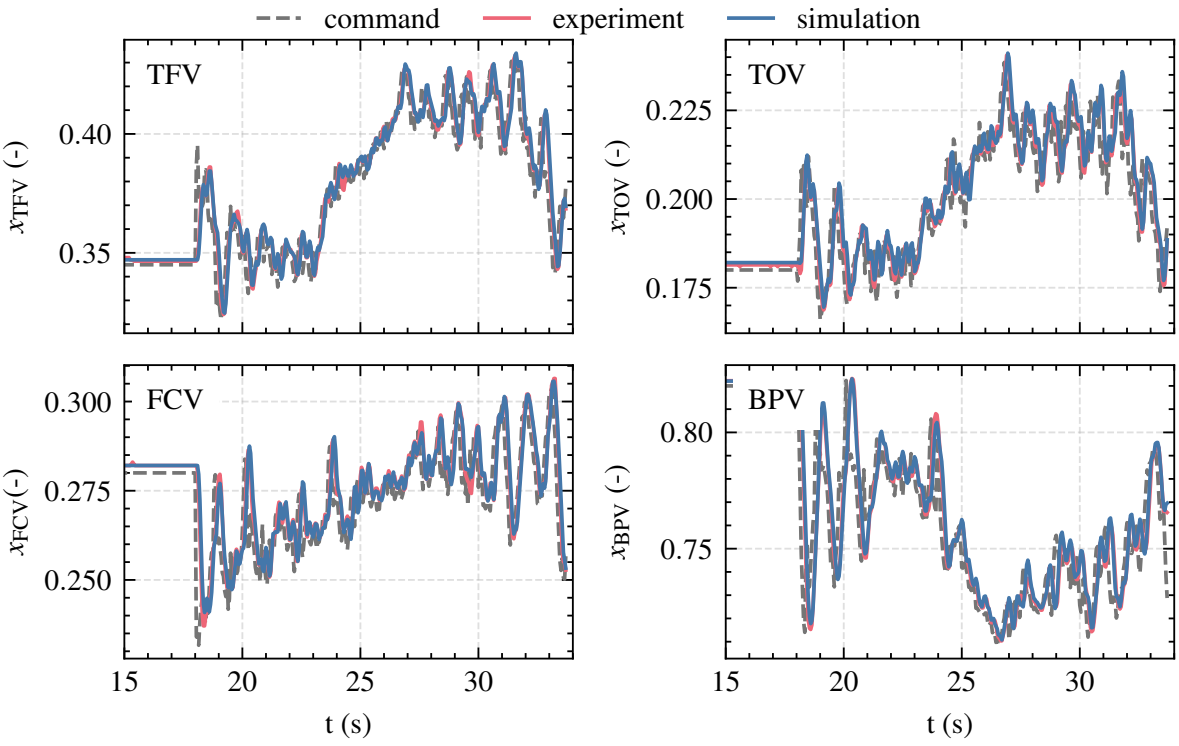


Figure A.4.: Improved valve modeling for test run 2

B. List of publications in relation to this thesis

Journal publications

[1] K. Dresia, E. Kurudzija, G. Waxenegger-Wilfing, et al.: Automation of Testing and Fault Detection for Rocket Engine Test Facilities with Machine Learning. *International Journal of Energetic Materials and Chemical Propulsion*, 22(6), 2023, pp. 17–33. DOI: 10.1615/IntJEnergeticMaterialsChemProp.2023047195

[2] T. Hörger, L. Werling, K. Dresia, et al.: Experimental and Simulative Evaluation of a Reinforcement Learning Based Cold Gas Thrust Chamber Pressure Controller. *Acta Astronautica*, 219, 2024, pp. 128–137. DOI: 10.1016/j.actaastro.2024.02.039

[3] G. Waxenegger-Wilfing, K. Dresia, J. Deeken, et al.: A Reinforcement Learning Approach for Transient Control of Liquid Rocket Engines. *IEEE Transactions on Aerospace and Electronic Systems*, 57(5), 2021, pp. 2938–2952. DOI: 10.1109/TAES.2021.3074134

Conference publications

[1] K. Dresia, A. Adler, A. Saraf, et al.: Rocket Engine Control with Neural Networks: Experimental Results of the LUMEN Turbopump Test Campaign. In: *AIAA Scitech Forum*. National Harbor, MD, 2023. DOI: 10.2514/6.2023-0137

[2] K. Dresia, M. Boerner, W. Armbruster, et al.: Design and Control Challenges for the LUMEN LOX/LNG Expander-Bleed Rocket Engine. In: *34th International Symposium on Space Technology and Science (ISTS)*. Fukuoka, Japan, 2023.

[4] K. Dresia, T. Traudt, T. Hörger, et al.: Overview of Future Rocket Engine Control Systems. In: *Space Propulsion Conference*. Glasgow, Scotland: 3AF, 2024.

[5] K. Dresia, G. Waxenegger-Wilfing, Robson H. Dos Santos Hahn, et al.: Nonlinear Control of an Expander-Bleed Rocket Engine Using Reinforcement Learning. In: *Space Propulsion Conference*. Virtual: 3AF, 2021.

[6] T. Hörger, K. Dresia, G. Waxenegger-Wilfing, et al.: Preliminary Investigation of Robust Reinforcement Learning for Control of an Existing Green Propellant Thruster. In: *AIAA Propulsion & Energy Forum*. Virtual, 2021. DOI: 10.2514/6.2021-3223

[7] G. Waxenegger-Wilfing, K. Dresia, J. Deeken, et al.: Machine Learning Methods for the Design and Operation of Liquid Rocket Engines – Research Activities at the DLR Institute of Space Propulsion. In: *Space Propulsion Conference*. Virtual: 3AF, 2021.